

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ BỘ VI XỬ LÝ 32 BIT

NGÀNH : CNKT ĐIỆN TỬ - VIỄN THÔNG

SVTH : NGUYỄN BẢO TÍNH

MSSV: 20161383

TRẦN NHẬT HẢI

MSSV: 20161311

Tp. Hồ Chí Minh, 06/2024

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ BỘ VI XỬ LÝ 32 BIT

NGÀNH : CNKT ĐIỆN TỬ - VIỄN THÔNG

SVTH: NGUYỄN BẢO TÍNH

MSSV: 20161383

TRẦN NHẬT HẢI

MSSV: 20161311

GVHD : ThS. LÊ MINH THÀNH

Tp. Hồ Chí Minh, 06/2024

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

Độc lập – Tự do – Hạnh phúc

----***----

Tp. Hồ Chí Minh, ngày 03 tháng 06 năm 2024

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Nguyễn Bảo Tính **MSSV:** 20161383

Trần Nhật Hải **MSSV:** 20161311

Ngành: Công nghệ kỹ thuật điện tử- viễn thông **Lớp:** 2016VMVT1

Giảng viên hướng dẫn: ThS. Lê Minh Thành

Ngày nhận đề tài: 19/02/2024

Ngày nộp đề tài: 03/06/2024

1. Tên đề tài: Thiết kế bộ vi xử lý 32 bit

2. Các số liệu, tài liệu ban đầu:

- Kiến thức về các môn: Mạch điện, Điện tử cơ bản, Kỹ thuật số, Kỹ thuật vi xử lý, Thiết kế mạch tích hợp VLSI.
- Các tài liệu về phần mềm Vivado.
- Các kiến thức về lập trình ngôn ngữ phân cứng Verilog.

3. Nội dung thực hiện đề tài:

- Vẽ sơ đồ khối tổng quát cho hệ thống.
- Thiết kế chi tiết hệ thống.
- Lập trình từng mô đun bằng phần mềm Vivado.
- Mô phỏng chương trình bằng phần mềm Vivado.
- Viết báo cáo.
- Bảo vệ đồ án tốt nghiệp.

4. Sản phẩm:

Kết quả mô phỏng chạy các lệnh của hệ thống trên Vivado.

LỜI CẢM ƠN

Đầu tiên, nhóm chúng tôi xin gửi lời cảm ơn sâu sắc đến Thầy Lê Minh Thành, người đã tận tình hướng dẫn và chia sẻ kiến thức cùng kinh nghiệm quý báu. Sự động viên và hỗ trợ không ngừng của Thầy đã giúp nhóm vượt qua mọi khó khăn trong suốt quá trình thực hiện đồ án này.

Nhóm xin bày tỏ lòng biết ơn chân thành đến Ban Giám hiệu, các thầy cô giáo và toàn thể cán bộ nhân viên trong trường, những người đã tạo điều kiện thuận lợi và cung cấp môi trường học tập, nghiên cứu lý tưởng giúp chúng tôi hoàn thành đồ án.

Nhóm cũng cảm ơn các bạn bè đã luôn sát cánh, chia sẻ những khó khăn và đóng góp nhiều ý kiến giá trị trong suốt thời gian thực hiện đồ án.

Dù đã cố gắng hết sức, nhưng do kiến thức và kinh nghiệm còn hạn chế, đồ án của nhóm không tránh khỏi những thiếu sót. Chúng tôi mong nhận được sự góp ý và chỉ bảo từ các thầy cô và bạn bè để hoàn thiện hơn nữa.

Xin chân thành cảm ơn!

Nguyễn Bảo Tính, Trần Nhật Hải

TÓM TẮT

Đề tài nghiên cứu phát triển bộ vi xử lý 32 bit dựa trên kiến trúc RISC-V và Harvard để tối ưu hóa hiệu suất cho hệ thống nhúng. Công việc bao gồm thiết kế và mô phỏng các thành phần cơ bản của vi xử lý, sử dụng kỹ thuật đường ống và phần mềm Vivado Design Suite. Đề tài không bao gồm thiết kế vi mạch chi tiết hay tích hợp vi mạch và không liên quan đến các công nghệ hoặc giao thức khác trong hệ thống nhúng.

Đề tài này nhằm thiết kế và xây dựng mô hình vi xử lý RISC-V 5 tầng sử dụng Verilog HDL, mô phỏng các khối chức năng chính như lấy lệnh, giải mã, thực thi, truy cập bộ nhớ và ghi lại. Công việc mô phỏng và kiểm thử nhằm xác minh tính chính xác và hiệu quả hoạt động của mô hình. Kết quả bao gồm mô hình hoàn chỉnh và đánh giá hiệu quả, có ứng dụng trong giáo dục, nghiên cứu và thiết kế vi mạch, góp phần hiểu sâu về kiến trúc RISC-V và kỹ thuật đường ống.

Từ khóa: RISC-V, Kỹ thuật đường ống, Vi xử lý, Verilog HDL.

MỤC LỤC

LỜI CẢM ƠN.....	i
TÓM TẮT.....	ii
MỤC LỤC	iii
DANH MỤC HÌNH.....	v
DANH MỤC BẢNG	vii
CÁC TỪ VIẾT TẮT.....	viii
CHƯƠNG 1 : TỔNG QUAN	1
1.1. TỔNG QUAN	1
1.2. MỤC TIÊU ĐỀ TÀI.....	3
1.3. GIỚI HẠN ĐỀ TÀI.....	4
1.4. BỐ CỤC ĐỒ ÁN	4
CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT	5
2.1. GIỚI THIỆU VỀ KIẾN TRÚC RISC.....	5
2.1.1. Tổng quan về kiến trúc máy tính.....	5
2.1.2. Sự phát triển của kiến trúc RISC	11
2.1.3. Đặc điểm chính của kiến trúc RISC	11
2.1.4. Ưu điểm và nhược điểm kiến trúc RISC	12
2.2. KIẾN TRÚC VI XỬ LÝ	13
2.2.1. Tổng quan về kiến trúc vi xử lý.....	13
2.2.2. Kiến trúc máy tính Von Neumann.....	13
2.2.3. Kiến trúc máy tính Havard	15
2.2.4. Kiến trúc máy tính MIPS.....	16
2.3. KIẾN TRÚC TẬP LỆNH RISC-V	17
2.3.1. Tổng quan.....	17
2.3.2. Lịch sử và phát triển của RISC-V	17
2.3.3. Nguyên tắc thiết kế của RISC-V	18
2.3.4. Kiến trúc tập lệnh số nguyên cơ sở RV32I.....	19
2.3.5. Phân mở rộng tiêu chuẩn của RISC-V	25
2.3.6. Ưu điểm, nhược điểm của RISC-V	26
2.4. KỸ THUẬT ĐƯỜNG ỐNG	27

2.4.1. Tổng quan về kỹ thuật đường ống.....	27
2.4.2. Hoạt động của đường ống năm tầng.....	29
2.4.3. Xung đột đường ống.....	30
CHƯƠNG 3 : THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG.....	34
3.1. YÊU CẦU HỆ THỐNG.....	34
3.2. THIẾT KẾ SƠ ĐỒ KHỐI VÀ CHỨC NĂNG MỖI KHỐI.....	34
3.2.1. Sơ đồ khối hệ thống.....	34
3.2.2. Hoạt động hệ thống	36
3.2.3. Khối lấy lệnh	37
3.2.4. Khối giải mã lệnh	39
3.2.5. Khối thực thi.....	42
3.2.6. Khối truy cập bộ nhớ.....	43
3.2.7. Khối ghi lại	44
3.2.8. Khối xử lý xung đột.....	45
CHƯƠNG 4 : KẾT QUẢ.....	47
4.1. KẾT QUẢ MÔ PHỎNG DẠNG SÓNG CÁC KHỐI	47
4.1.1. Khối lấy lệnh	47
4.1.2. Khối giải mã lệnh	48
4.1.3. Khối thực thi.....	53
4.1.4. Khối truy cập bộ nhớ.....	59
4.1.5. Khối ghi lại	62
4.1.6. Khối xử lý xung đột.....	62
4.2. KẾT QUẢ MÔ PHỎNG TOÀN HỆ THỐNG.....	64
CHƯƠNG 5 : KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	76
5.1. KẾT LUẬN.....	76
5.2. HƯỚNG PHÁT TRIỂN	76
TÀI LIỆU THAM KHẢO	77
PHỤ LỤC	79

DANH MỤC HÌNH

Hình 2.1: Máy tính đầu tiên ENIAC ra đời năm 1946.....	6
Hình 2.2: Mô hình kiến trúc Von Neumann.....	7
Hình 2.3: Ram động và Transistor.....	7
Hình 2.4: Sơ đồ khối chức năng của một hệ thống máy tính.....	8
Hình 2.5: CPU của hãng Intel: 8086 và Core 2 Duo.....	9
Hình 2.6: Bộ nhớ ROM và RAM.....	10
Hình 2.7: Kiến trúc máy tính Von Neumann nguyên thủy.....	13
Hình 2.8: Kiến trúc máy tính Von Neumann hiện đại.....	14
Hình 2.9: Kiến trúc máy tính Harvard.....	15
Hình 2.10: Sự phát triển của RISC-V.....	17
Hình 2.11: Trạng thái kiến trúc hiển thị cho RV32I.....	19
Hình 2.12: Định dạng lệnh của RV32I.....	20
Hình 2.13: Cấu trúc lệnh loại R.....	21
Hình 2.14: Cấu trúc lệnh loại I.....	21
Hình 2.15: Cấu trúc lệnh loại S.....	22
Hình 2.16: Cấu trúc lệnh loại U.....	22
Hình 2.17: Cấu trúc lệnh loại B.....	22
Hình 2.18: Cấu trúc lệnh loại J.....	23
Hình 2.19: Danh sách các lệnh trong RV32M.....	25
Hình 2.20: Thực thi chu kỳ đơn, thực thi không đường ống (phía trên) so với dùng kỹ thuật đường ống (phía dưới).....	28
Hình 2.21: Thực hiện chu trình lệnh đường ống số học.....	29
Hình 2.22: Sự xung đột trong các lệnh dùng kỹ thuật đường ống.....	31
Hình 2.23: Các lệnh phụ thuộc kết quả ADD với đường dẫn chuyển tiếp.....	31
Hình 2.24: Lệnh LD bỏ qua kết quả tới AND và OR, không tới SUB.....	32
Hình 2.25: Đường ống trước và sau khi chậm trễ.....	32
Hình 3.1: Sơ đồ tổng quan hệ thống.....	35
Hình 3.2: Sơ đồ chi tiết hệ thống.....	36

Hình 3.3: Sơ đồ Khối lấy lệnh.....	37
Hình 3.4: Sơ đồ chi tiết Khối lấy lệnh.....	38
Hình 3.5: Sơ đồ Khối giải mã lệnh	39
Hình 3.6: Sơ đồ chi tiết Khối giải mã lệnh	39
Hình 3.7: Sơ đồ Khối thực thi	42
Hình 3.8: Sơ đồ chi tiết Khối thực thi	42
Hình 3.9: Sơ đồ Khối truy cập bộ nhớ.....	43
Hình 3.10: Sơ đồ chi tiết Khối truy cập bộ nhớ.....	44
Hình 3.11: Sơ đồ Khối ghi lại	44
Hình 3.12: Sơ đồ chi tiết Khối ghi lại	45
Hình 3.13: Sơ đồ Khối xử lý xung đột.....	45
Hình 3.14: Sơ đồ chi tiết Khối xử lý xung đột.....	46
Hình 4.1: Sơ đồ RTL của Khối lấy lệnh.....	47
Hình 4.2: Dạng sóng của Khối lấy lệnh.....	47
Hình 4.3: Sơ đồ RTL của Khối giải mã lệnh.....	49
Hình 4.4: Dạng sóng của Khối giải mã lệnh	49
Hình 4.5: Sơ đồ RTL của Khối thực thi	54
Hình 4.6: Dạng sóng của Khối thực thi	54
Hình 4.7: Sơ đồ RTL của Khối truy cập bộ nhớ	60
Hình 4.8: Dạng sóng của Khối truy cập bộ nhớ.....	60
Hình 4.9: Sơ đồ RTL của Khối ghi lại	62
Hình 4.10: Dạng sóng của Khối ghi lại	62
Hình 4.11: Sơ đồ RTL của Khối xử lý xung đột.....	63
Hình 4.12: Dạng sóng của Khối xử lý xung đột.....	63
Hình 4.13: Sơ đồ RTL của Vi xử lý 32 bit.....	65
Hình 4.14: Công suất hoạt động của Bộ vi xử lý 32 bit.....	65
Hình 4.15: Dạng sóng của bộ vi xử lý 32 bit (từ 0 – 160ns).....	73
Hình 4.16: Dạng sóng của bộ vi xử lý 32 bit (từ 160 – 320ns).....	74
Hình 4.17: Dạng sóng của bộ vi xử lý 32 bit (từ 320 – 480ns).....	75

DANH MỤC BẢNG

<i>Bảng 2.1: Sơ đồ mã hoạt động chính RV32I</i>	20
<i>Bảng 2.2: Tổng thời gian của từng lệnh tính theo thời gian từng giai đoạn</i>	28
<i>Bảng 3.1: Bảng trạng thái của tín hiệu điều khiển với từng loại lệnh</i>	41
<i>Bảng 3.2: Bảng trạng thái của tín hiệu điều khiển ALU</i>	41
<i>Bảng 3.3: Mã hóa ImmSrc</i>	41
<i>Bảng 3.4: Các giá trị điều khiển cho các bộ ghép kênh chuyển tiếp</i>	46
<i>Bảng 4.1: Mã Assembly dùng mô phỏng chương trình</i>	66
<i>Bảng 4.2: Tóm tắt kết quả</i>	71

CÁC TỪ VIẾT TẮT

Ký hiệu	Từ tiếng Anh	Nghĩa tiếng Việt
ALU	Arithmetic Logic Unit	Đơn vị luận lý số học
ARM	Advanced RISC Machine	Máy tính với tập lệnh đơn giản tiên tiến
CISC	Complex Instruction Set Computer	Máy tính với tập lệnh phức tạp
CU	Control Unit	Đơn vị điều khiển
CPU	Central Processing Unit	Bộ xử lý trung tâm
FPGA	Field-programmable gate array	Mảng cổng lập trình được dạng trường
HDL	Hardware Description Language	Ngôn ngữ mô tả phần cứng
IoT	Internet of Things	Internet vạn vật
ISA	Instruction Set Architecture	Kiến trúc tập lệnh
MIPS	Microprocessor without Interlocked Pipeline Stages	Bộ vi xử lý không có giai đoạn đường ống đan xen
PC	Program Counter	Bộ đếm chương trình
RAM	Random Access Memory	Bộ nhớ truy xuất ngẫu nhiên
RAW	Read After Write	Đọc sau ghi
RISC	Reduced Instruction Set Computing	Máy tính với tập lệnh đơn giản
RISC-V	Reduced Instruction Set Computing - V	Máy tính với tập lệnh đơn giản -V
ROM	Read Only Memory	Bộ nhớ chỉ đọc
WAR	Write After Read	Ghi sau đọc
WAW	Write after Write	Ghi sau ghi

CHƯƠNG 1 : TỔNG QUAN

1.1. TỔNG QUAN

Thế giới hiện đại đang chứng kiến một giai đoạn phát triển công nghệ đầy mạnh mẽ, đem lại những đổi mới kỹ thuật số đặc biệt quan trọng. Các tiến bộ này không chỉ ảnh hưởng đến các ngành công nghiệp và cấu trúc xã hội mà còn có sự thay đổi đáng kể nhiều khía cạnh của cuộc sống. Sự tiến bộ không ngừng trong lĩnh vực công nghệ không chỉ làm thay đổi bề mặt của các ngành công nghiệp, mà còn tạo ra sự hòa nhập và tương tác giữa nhiều xu hướng công nghệ khác nhau chứng kiến những đột phá đáng kể hơn nữa trong lĩnh vực này. Điều đặc biệt về cuộc cách mạng công nghiệp 4.0 là sự kết hợp và tương tác giữa nhiều xu hướng công nghệ, tạo nên một sức mạnh mới mẻ và đột phá [1].

Trong những năm gần đây, ngành công nghệ bán dẫn đã trở thành một ngành công nghiệp đóng vai trò quan trọng trong nền kinh tế của nhiều quốc gia trên thế giới, bao gồm cả Việt Nam. Sự thiếu hụt chip trên toàn cầu đã trở thành một vấn đề nóng được nhiều quốc gia quan tâm. Trong một tọa đàm về xu hướng phát triển công nghệ mạch tích hợp trên thế giới và tại Việt Nam, ông Phil Hoàng, một quản lý cấp cao tại Skyworks Solutions, đã chia sẻ rằng vi mạch bán dẫn được coi là nền tảng của tính toán hiện đại và đóng vai trò vô cùng quan trọng trong bối cảnh thế giới đang trải qua giai đoạn bùng nổ nghiên cứu và sản xuất chip [2]. Công nghiệp vi mạch bán dẫn đang đóng vai trò ngày càng quan trọng trong cuộc cách mạng chuyển đổi số và xanh, cũng như trong việc thúc đẩy kinh tế tri thức thay vì dựa vào tài nguyên thiên nhiên. Việt Nam với nguồn nhân lực chăm chỉ và ham học hỏi, cùng với đội ngũ các nhà khoa học nỗ lực không ngừng cho sự phát triển của đất nước [3]. Xu hướng công nghệ RISC đang ngày càng trở nên phổ biến trong lĩnh vực thiết kế vi mạch và máy tính. Bằng cách tập trung vào việc giảm số lượng các lệnh máy cần thiết và tối ưu hóa hiệu suất xử lý, RISC đem lại nhiều lợi ích đáng kể [4]. Đặc biệt, việc tiêu thụ năng lượng thấp làm cho RISC trở thành sự lựa chọn ưa thích cho các ứng dụng di động và IoT. Sự đơn giản của các lệnh RISC cũng thuận lợi cho việc tối ưu hóa phần cứng, giúp giảm kích thước, chi phí sản

xuất và tăng hiệu suất của các hệ thống. Với những ưu điểm này, RISC đang ngày càng được ứng dụng rộng rãi trong các thiết bị điện tử và có tiềm năng phát triển mạnh mẽ trong tương lai [4].

Ở đề tài “Design of 32-bit Processor for Embedded Systems” của tác giả Hyun Woo Oh, Kwon Neung Cho và Seung Eun Lee đã áp dụng kiến trúc MIPS cho bộ vi xử lý 32 bit cho hệ thống nhúng [5]. Đề tài đã đạt được những ưu điểm như cung cấp diện tích ít hơn và công suất tiêu thụ thấp hơn, tuy nhiên đề tài vẫn còn một số hạn chế như khả năng xử lý đa nhiệm còn hạn chế, đặc biệt là so với kiến trúc tập lệnh RISC hiện đại khác. Điều này làm giảm hiệu suất của hệ thống khi cần xử lý nhiều tác vụ. Ở đề tài “A Single Clock Cycle MIPS RISC Processor Design using VHDL” của Mamun Bin Ibne Reaz, Md. Shabiul Islam và Mohd. S. Sulaiman [6], đề tài “Hardware Modelling of a 32-bit Single Cycle RISC Processor using VHDL” của tác giả Safaa S. Omran và Hadeel S. Mahmood [7] đã sử dụng một chu kỳ xung duy nhất cho bộ vi xử lý MIPS RISC. Đề tài đã đạt được những ưu điểm như một lệnh được thực thi trong một chu kỳ xung duy nhất, làm cho thiết kế đơn giản hơn và không có sự xung đột dữ liệu. Tuy nhiên còn một số hạn chế như một số lệnh có thể chỉ mất một phần nhỏ của chu kỳ xung để thực thi, trong khi các lệnh khác có thể mất nhiều chu kỳ. Điều này làm tăng thời gian thực thi trung bình của mỗi lệnh dẫn đến hiệu suất thấp và một số tài nguyên như thanh ghi, đường dẫn dữ liệu có thể không được sử dụng hiệu quả gây nên lãng phí tài nguyên [6].

Dựa trên những ưu điểm của công nghệ RISC, đồ án tốt nghiệp này tập trung phát triển một bộ vi xử lý mã nguồn mở, khả năng mở rộng, tiết kiệm năng lượng và hiệu suất cao. RISC-V tuân thủ nguyên tắc kiến trúc tập lệnh đơn giản hóa và hoạt động như một tập lệnh mã nguồn mở, hoàn toàn truy cập miễn phí cho cá nhân và các công ty [9]. RISC-V đã được phát triển bởi một nhóm dẫn đầu bởi Giáo sư David Patterson tại Đại học California, Berkeley. Ý tưởng này xuất phát từ nhu cầu trong cộng đồng công nghiệp và nghiên cứu về một kiến trúc tập lệnh

mở, linh hoạt, có thể mở rộng và hỗ trợ các không gian địa chỉ 32 bit, 64 bit và 128 bit [9].

RISC-V được thiết kế để làm cho việc phát triển và triển khai các bộ vi xử lý trở nên dễ dàng hơn bằng cách cung cấp một kiến trúc linh hoạt, tiêu chuẩn và không bị ràng buộc. Nhằm phát triển một bộ vi xử lý dựa trên kiến trúc tập lệnh RISC-V và kiến trúc Harvard. Trong quá trình thực hiện nhóm sẽ tập trung vào việc thiết kế các thành phần chính của bộ vi xử lý bao gồm : Bộ điều khiển, bộ nhớ, bộ xử lý lệnh và bộ xử lý dữ liệu để tối ưu hóa hiệu suất và tính hiệu quả. Sau đó, để đảm bảo tính đúng đắn và hiệu suất của bộ vi xử lý, nhóm sẽ sử dụng phần mềm mô phỏng Vivado Design Suite để kiểm tra các mô-đun của bộ vi xử lý. Thừa hưởng từ các đề tài đã nghiên cứu như kiến trúc MIPS, chu kỳ xung duy nhất... Nhóm đưa ra một số cải tiến như sử dụng kiến trúc Harvard để sử dụng hai bộ nhớ riêng biệt cho lệnh và dữ liệu giúp tránh xung đột khi truy cập và sử dụng kỹ thuật đường ống cho phép nhiều lệnh thực hiện cùng một lúc , từ đó tăng tốc độ xử lý , giúp giảm thời gian chờ đợi và tối ưu hóa hiệu suất của bộ vi xử lý [8]. Kết quả dự kiến của đề tài này là một bộ vi xử lý 32 bit dựa trên kiến trúc tập lệnh RISC-V và kiến trúc Harvard có khả năng thực thi nhiều lệnh cùng một lúc, tối ưu hóa hiệu suất và tính hiệu quả trong các hệ thống nhúng và ứng dụng nhúng khác.

1.2. MỤC TIÊU ĐỀ TÀI

Mục tiêu của đề tài là phát triển một bộ vi xử lý 32 bit dựa trên kiến trúc tập lệnh RISC-V và kiến trúc Harvard, nhằm tối ưu hóa hiệu suất và tính hiệu quả trong các hệ thống nhúng :

- Tận dụng kiến trúc với tập lệnh đơn giản hóa để giảm độ phức tạp và mã nguồn mở hoàn toàn có thể truy cập linh hoạt để bất kỳ cá nhân nào sử dụng mà không chịu bất kỳ chi phí nào.
- Tận dụng kỹ thuật đường ống để tăng cường hiệu suất xử lý thông qua việc thực thi đồng thời các giai đoạn khác nhau của một lệnh trong mỗi chu kỳ xung.

- Tận dụng ưu điểm của kiến trúc Harvard để phân biệt bộ nhớ lệnh và bộ nhớ dữ liệu để cải thiện việc truy cập bộ nhớ, tăng tốc độ thực thi chương trình và tránh xung đột giữa việc đọc và ghi dữ liệu.

1.3. GIỚI HẠN ĐỀ TÀI

Đề tài sẽ tập trung vào việc thiết kế và phát triển một bộ vi xử lý 32 bit dựa trên kiến trúc tập lệnh RISC-V và kiến trúc Harvard. Tuy nhiên, để đảm bảo tính khả thi và hiệu quả, đề tài sẽ giới hạn vào các khía cạnh như:

- Hạn chế đi sâu vào vấn đề thiết kế vi mạch hay tích hợp ở cấp độ chip. Thay vào đó là thiết kế các thành phần cơ bản của bộ vi xử lý.
- Sử dụng phần mềm mô phỏng để kiểm tra tính đúng đắn và hiệu suất của bộ vi xử lý, nhưng không đặt mục tiêu vào việc tạo ra một mô hình mô phỏng hoàn chỉnh và chi tiết của toàn bộ bộ vi xử lý.
- Không tích hợp bộ vi xử lý với các công nghệ, giao thức khác trong hệ thống nhúng và nhấn mạnh vào việc phát triển một vi xử lý độc lập.

1.4. BỐ CỤC ĐỒ ÁN

Đề tài gồm 5 chương được phân bổ như sau:

- Chương 1: Tổng quan: Tổng quan về đề tài, mục tiêu đặt ra của đề tài, giới hạn đề tài cũng như bố cục của đề tài.
- Chương 2: Cơ sở lý thuyết: Lý thuyết liên quan đến đề tài, những khái niệm và nội dung cần tìm hiểu khi xây dựng đề tài như vi xử lý, bộ nhớ, kiến trúc máy tính...
- Chương 3: Thiết kế và xây dựng hệ thống: Mô tả yêu cầu của hệ thống, sơ đồ khối hệ thống, thiết kế chi tiết từng khối và thiết kế phần mềm.
- Chương 4: Kết quả: Đánh giá chức năng từng khối, kết quả mô phỏng và đánh giá khả năng đọc/ghi bộ nhớ.
- Chương 5: Kết luận và hướng phát triển.

CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT

2.1. GIỚI THIỆU VỀ KIẾN TRÚC RISC

2.1.1. Tổng quan về kiến trúc máy tính

2.1.1.1. Giới thiệu kiến trúc máy tính

Kiến trúc máy tính rất quan trọng trong thiết kế hệ thống máy tính hiện đại. Nó xác định cách các thành phần tương tác và cách thông tin được xử lý, lưu trữ. Kiến trúc này là cơ sở để phát triển và tối ưu hóa ứng dụng mới. Kiến trúc RISC và CISC là hai loại kiến trúc cung cấp các phương pháp khác nhau cho việc thiết kế và tối ưu hóa hệ thống, mỗi loại có ưu và nhược điểm riêng, cho phép sự linh hoạt trong lựa chọn kiến trúc phù hợp với dự án.

Kiến trúc máy tính nghiên cứu việc chọn và lắp ráp phần cứng để xây dựng máy tính đáp ứng yêu cầu về chi phí, hiệu suất và chức năng. Yêu cầu chức năng đòi hỏi máy tính có nhiều tính năng thực tiễn, yêu cầu hiệu suất đòi hỏi xử lý nhanh, chi phí đòi hỏi giá rẻ hơn. Đáp ứng cả ba mục tiêu này rất khó, nhưng nhờ phát triển công nghệ vi xử lý, máy tính hiện đại trở nên tính năng hơn, nhanh hơn và rẻ hơn.

Kiến trúc máy tính bao gồm:

- Kiến trúc tập lệnh
- Vi kiến trúc
- Thiết kế hệ thống

Kiến trúc tập lệnh là giao diện giữa phần cứng và phần mềm. Nó xác định các lệnh mà phần cứng có thể thực hiện, cùng với các kiểu dữ liệu, các thanh ghi, các phương thức địa chỉ, và mô hình bộ nhớ mà lập trình viên và trình biên dịch có thể sử dụng để điều khiển phần cứng. ISA đóng vai trò như một hợp đồng, đảm bảo rằng bất kỳ chương trình nào được viết cho một ISA cụ thể sẽ chạy trên bất kỳ loại phần cứng nào tuân theo ISA đó. Một số ví dụ về ISA bao gồm x86, ARM, và MIPS.

Vi kiến trúc là cách mà một bộ xử lý cụ thể được thiết kế để thực thi ISA. Cấp độ này đối phó với các quyết định thiết kế nội bộ như số lượng và loại thanh ghi,

số lượng và kiểu các đơn vị thực thi, cách thức triển khai kỹ thuật đường ống, quản lý bộ nhớ đệm, và tối ưu hóa cho các hoạt động đồng thời. Các ví dụ về vi kiến trúc bao gồm Intel's Core, AMD's Zen, và ARM's Cortex series. Mỗi vi kiến trúc cung cấp một cách để tối ưu hóa thực thi của ISA cụ thể trên phần cứng.

Thiết kế hệ thống liên quan đến tổng thể của máy tính bao gồm không chỉ CPU mà còn các thành phần khác như bộ nhớ, các thiết bị vào ra và hệ thống giao tiếp. Thiết kế hệ thống quyết định cách các thành phần này kết nối và tương tác với nhau, từ các đường nối và kết nối mạng nội bộ đến các giải pháp như hệ thống bộ nhớ phân tán hoặc tính toán đám mây. Thiết kế hệ thống cũng bao gồm tính toán các yêu cầu về nguồn điện, làm mát và quản lý năng lượng, đảm bảo rằng toàn bộ hệ thống có thể hoạt động hiệu quả và ổn định.

2.1.1.2. Lịch sử và phát triển của kiến trúc máy tính

Lịch sử và phát triển của kiến trúc máy tính bao gồm nhiều giai đoạn chuyển biến từ máy tính đầu tiên đến các thiết bị hiện đại ngày nay. Dưới đây là các mốc quan trọng và các đột phá trong lịch sử thiết kế máy tính:

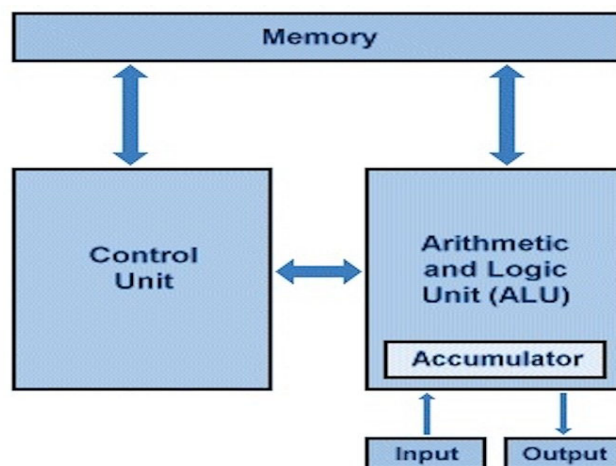
❖ Thời kỳ đầu tiên của Máy Tính (1940-1950)

- **Máy Tính Điện Tử Đầu Tiên:** Máy tính ENIAC ra đời vào năm 1945 là máy tính điện tử đầu tiên. ENIAC sử dụng hàng nghìn ống điện tử, chiếm diện tích rất lớn và tiêu thụ nhiều điện năng.



Hình 2.1: Máy tính đầu tiên ENIAC ra đời năm 1946

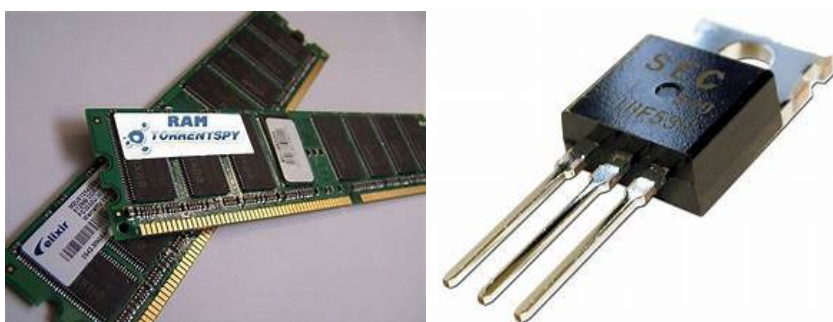
- Von Neumann Architecture: John Von Neumann đề xuất mô hình kiến trúc máy tính vào năm 1945, được biết đến là kiến trúc Von Neumann, nơi mà chương trình và dữ liệu cùng được lưu trữ trong bộ nhớ.



Hình 2.2: Mô hình kiến trúc Von Neumann

❖ Phát triển của Bộ nhớ và Lưu trữ (1950-1960)

- Transistor: Phát minh transistor vào năm 1947 và sự áp dụng của chúng trong máy tính vào những năm 1950 đã giúp máy tính nhỏ gọn và tiết kiệm điện hơn nhiều.
- RAM Động: Phát minh RAM động vào cuối những năm 1960 đã cho phép bộ nhớ máy tính nhanh và hiệu quả hơn.

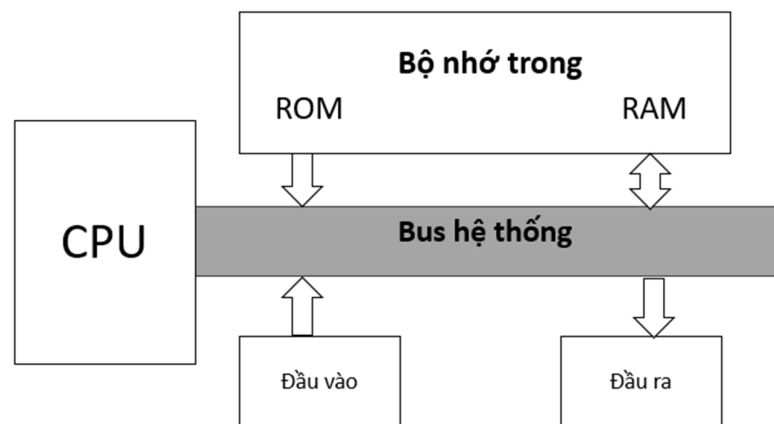


Hình 2.3: Ram động và Transistor

❖ **Đổi mới trong Kiến Trúc Máy Tính (1970-1980)**

- Kiến trúc CISC và RISC: Sự ra đời và phát triển của các kiến trúc máy tính phức hợp (CISC) và đơn giản (RISC), với RISC tập trung vào việc tối ưu hóa các lệnh đơn giản để tăng hiệu suất.
- Bộ xử lý đa lõi: Các nghiên cứu và phát triển về bộ xử lý đa lõi bắt đầu, đặt nền móng cho sự phát triển của các CPU hiện đại có nhiều lõi xử lý.

2.1.1.3. Cấu trúc và chức năng của kiến trúc máy tính



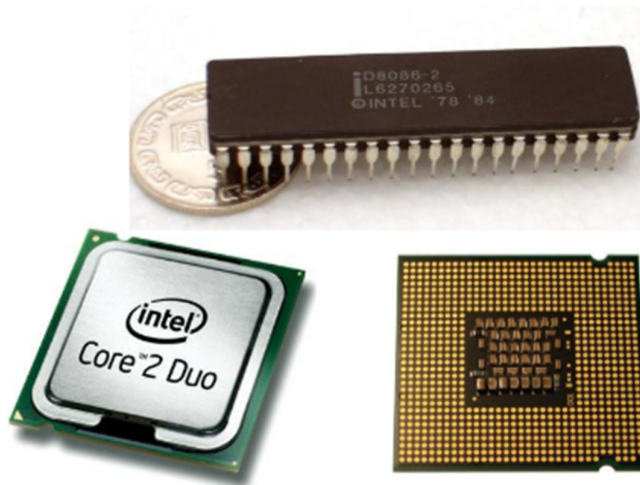
Hình 2.4: Sơ đồ khối chức năng của một hệ thống máy tính

Hình 2.4 mô tả sơ đồ cấu trúc chức năng của một hệ thống máy tính, gồm các thành phần chính: CPU, bộ nhớ trong (ROM và RAM), bus hệ thống và các thiết bị đầu vào/ra. CPU, được coi là bộ não của máy tính, chịu trách nhiệm thực hiện các lệnh từ phần mềm và liên kết với các thành phần khác qua bus hệ thống. Bộ nhớ trong bao gồm ROM, nơi lưu trữ chương trình khởi động và các chương trình cơ bản không mất dữ liệu khi tắt máy, và RAM, nơi lưu trữ dữ liệu và chương trình đang chạy nhưng sẽ mất dữ liệu khi tắt máy. Bus hệ thống là kênh giao tiếp nội bộ giữa CPU, bộ nhớ trong và các thiết bị đầu vào/ra, bao gồm các bus dữ liệu, địa chỉ và điều khiển để truyền tải thông tin. Các thiết bị đầu vào như bàn phím, chuột cho phép người dùng nhập dữ liệu vào máy tính, trong khi các thiết bị đầu ra như màn hình, máy in giúp xuất dữ liệu ra cho người dùng. Cấu trúc này minh họa sự

tương tác và phối hợp giữa các thành phần chính của hệ thống máy tính để thực hiện các nhiệm vụ xử lý dữ liệu.

❖ Khối xử lý trung tâm

Hình 2.5 giới thiệu một số CPU của hãng Intel, bộ phận quan trọng nhất thường được gọi là "bộ não" của máy tính. Đây là bộ xử lý trung tâm, nơi thực hiện các tác vụ chính. Cách phổ biến để diễn đạt nhu cầu của hệ thống và người dùng là thông qua các chương trình máy tính, mỗi chương trình bao gồm một tập hợp các lệnh của CPU. CPU có trách nhiệm đọc, giải mã và thực thi các lệnh chương trình từ bộ nhớ. Bằng cách thực hiện các lệnh này, CPU giúp máy tính cung cấp các chức năng hữu ích cho người dùng.



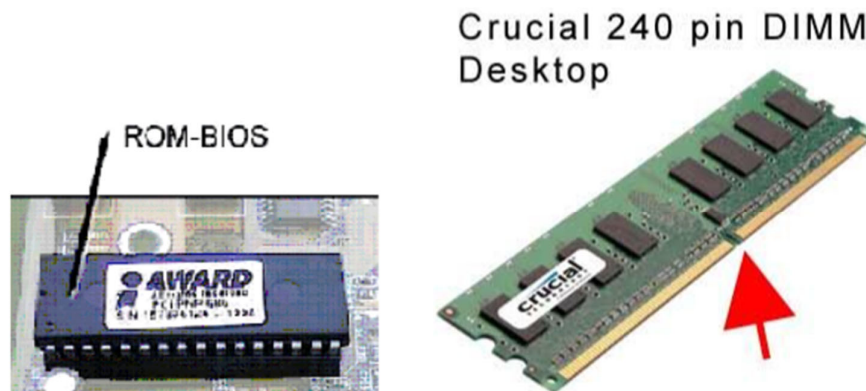
Hình 2.5: CPU của hãng Intel: 8086 và Core 2 Duo

CPU là một vi mạch tích hợp cao gồm bốn thành phần chính: Đơn vị điều khiển, Đơn vị luận lý số học, Thanh ghi, và Bus nội bộ. Đơn vị điều khiển chịu trách nhiệm đọc, giải mã và quản lý việc thực thi các lệnh. Ngoài việc thực hiện các phép toán số học như cộng, trừ, nhân và chia, Đơn vị luận lý số học còn có thể thực hiện các phép toán luận lý như AND, OR, NOT và các phép dịch chuyển [22]. Các không gian lưu trữ tạm thời cho dữ liệu và lệnh mà CPU xử lý được gọi là các thanh ghi. Bus Nội bộ kết nối với hệ thống bus bên ngoài và truyền tín hiệu giữa các thành phần của CPU.

❖ Bộ nhớ trong

Thường được gọi là bộ nhớ chính, bộ nhớ trong là không gian lưu trữ cho dữ liệu người dùng và hệ thống cũng như các lệnh hỗ trợ hoạt động của CPU. Thường được chế tạo dựa trên chất bán dẫn, bộ nhớ trong có hai loại: Bộ nhớ truy xuất ngẫu nhiên và Bộ nhớ chỉ đọc. Dữ liệu và lệnh hệ thống thường được lưu trữ trong ROM. ROM được nạp thông tin trong quá trình sản xuất và thường chỉ có thể đọc được khi thiết bị đang sử dụng[22]. Hơn nữa, dữ liệu lưu trữ trong ROM vẫn tồn tại ngay cả khi không có điện.

Khác với ROM, RAM thường được sử dụng để lưu trữ dữ liệu và lệnh cho cả người dùng và hệ thống. Thông thường, RAM có dung lượng lớn hơn nhiều so với ROM. Tuy nhiên, dữ liệu trong RAM chỉ tồn tại khi có nguồn điện. Các mạch bộ nhớ ROM và RAM được lắp đặt trên mô-đun RAM, như minh họa trong Hình 2.6.



Hình 2.6: Bộ nhớ ROM và RAM

❖ Bus hệ thống

Các dây dẫn kết nối CPU với các phần khác của máy tính được gọi là Bus hệ thống. Bus địa chỉ, Bus dữ liệu và Bus điều khiển là ba con bus phụ thường tạo nên Bus hệ thống.

- Tín hiệu địa chỉ được gửi từ CPU đến bộ nhớ và các thiết bị ngoại vi qua Bus địa chỉ.
- Tín hiệu dữ liệu được truyền đi hai chiều giữa CPU và các thành phần khác qua Bus dữ liệu.

- Tín hiệu trạng thái từ các thành phần khác và tín hiệu điều khiển từ CPU đến các thành phần khác đều được gửi qua Bus điều khiển.

2.1.2. Sự phát triển của kiến trúc RISC

Kiến trúc RISC xuất hiện vào những năm 1980 với mục tiêu đơn giản hóa tập lệnh để tăng hiệu suất xử lý. Các nhà nghiên cứu nhận ra rằng phần lớn các lệnh phức tạp trong máy tính CISC không được sử dụng thường xuyên, do đó, việc đơn giản hóa tập lệnh giúp cải thiện tốc độ xử lý và hiệu quả sử dụng tài nguyên. Kiến trúc RISC đặc trưng bởi tập lệnh đơn giản, hỗ trợ kỹ thuật đường ống để thực hiện nhiều lệnh đồng thời, và tăng cường sử dụng các thanh ghi thay vì phụ thuộc vào bộ nhớ. Sự phát triển của RISC đã tạo ra các vi xử lý nổi tiếng như ARM, MIPS, và SPARC, được ứng dụng rộng rãi trong thiết bị điện tử tiêu dùng, máy chủ, và hệ thống nhúng. Kiến trúc RISC đã trở thành nền tảng cho nhiều tiến bộ trong công nghệ vi xử lý, đóng góp quan trọng vào sự phát triển của ngành công nghiệp máy tính hiện đại [8].

2.1.3. Đặc điểm chính của kiến trúc RISC

Kiến trúc RISC là một trong những tiến bộ quan trọng trong lĩnh vực công nghệ máy tính. Với sự tập trung vào việc đơn giản hóa và tối ưu hóa hiệu suất của CPU, kiến trúc RISC đã đem lại những đặc điểm đáng chú ý như sau:

Tập lệnh đơn giản là một trong những đặc trưng quan trọng của kiến trúc RISC. Kiến trúc RISC sử dụng một tập hợp nhỏ các lệnh đơn giản và đồng nhất, giúp giảm thời gian giải mã và thực thi lệnh, từ đó tăng hiệu suất xử lý [16].

Thiết kế của RISC tối ưu cho kỹ thuật đường ống lệnh. Điều này cho phép nhiều lệnh được xử lý đồng thời ở các giai đoạn khác nhau, cải thiện hiệu suất tổng thể của hệ thống[18].

RISC tăng cường sử dụng các thanh ghi để lưu trữ tạm thời dữ liệu thay vì phụ thuộc nhiều vào bộ nhớ ngoài. Việc này giúp giảm thời gian truy cập dữ liệu và tăng tốc độ xử lý, làm cho hệ thống nhanh hơn và hiệu quả hơn.

Một đặc điểm khác của kiến trúc RISC là các lệnh tải và lưu trữ được tách riêng. Điều này giúp đơn giản hóa việc thiết kế bộ giải mã lệnh và bộ điều khiển, làm cho hệ thống dễ dàng phát triển và duy trì.

Hầu hết các lệnh trong RISC được thiết kế để hoàn thành trong một chu kỳ máy, giúp dự đoán hiệu suất dễ dàng hơn và tăng tính ổn định của hệ thống. Điều này cũng giúp giảm độ phức tạp trong việc thiết kế và quản lý hệ thống. Kiến trúc RISC không chỉ được biết đến với tính đơn giản mà còn với sự dễ dàng trong việc tối ưu hóa và mở rộng. Điều này cho phép các nhà sản xuất CPU tùy chỉnh và cải thiện hiệu suất của các sản phẩm dựa trên RISC một cách dễ dàng hơn. Bằng cách tùy chỉnh phần cứng và phần mềm, họ có thể tạo ra các phiên bản CPU với hiệu suất cao hơn và tính linh hoạt trong thiết kế, đáp ứng nhanh chóng các yêu cầu của thị trường và công nghệ mới. Điều này giúp kiến trúc RISC trở thành một trong những lựa chọn phổ biến và hiệu quả trong thế giới của công nghệ máy tính ngày nay.

2.1.4. Ưu điểm và nhược điểm kiến trúc RISC

❖ Ưu điểm

- Lệnh đơn giản hơn: Bộ xử lý RISC sử dụng tập lệnh đơn giản hơn, giúp giải mã và thực thi nhanh chóng dễ dàng hơn. Điều này dẫn đến quá trình xử lý thời gian nhanh hơn.
- Thực thi nhanh hơn: Vì bộ xử lý RISC có lệnh đơn giản hơn nên chúng có thể thực thi các lệnh nhanh hơn bộ xử lý CISC.
- Tiêu thụ điện năng thấp hơn: Bộ xử lý RISC tiêu thụ ít năng lượng hơn bộ xử lý CISC, khiến chúng trở nên lý tưởng cho các thiết bị di động.

❖ Nhược điểm

- Yêu cầu nhiều hướng dẫn hơn: Bộ xử lý RISC yêu cầu nhiều hướng dẫn hơn để thực hiện các tác vụ phức tạp hơn bộ xử lý CISC.
- Tăng cường khả năng sử dụng bộ nhớ: Bộ xử lý RISC được yêu cầu nhiều bộ nhớ hơn để lưu trữ các tiện ích bổ sung hướng dẫn cần thiết để thực hiện các tác vụ phức tạp.

- Chi phí cao hơn: Việc phát triển và sản xuất bộ xử lý RISC có thể cân nhắc hơn bộ xử lý CISC.

2.2. KIẾN TRÚC VI XỬ LÝ

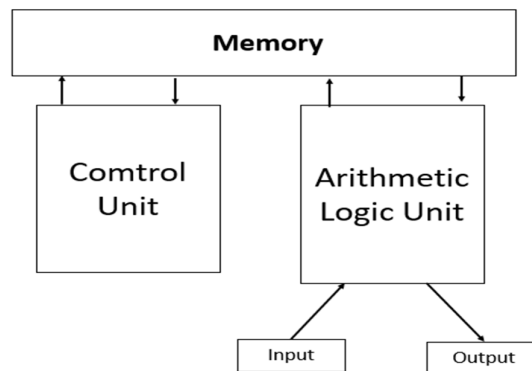
2.2.1. Tổng quan về kiến trúc vi xử lý

Kiến trúc vi xử lý là thiết kế và cấu trúc của bộ xử lý trung tâm trong máy tính và các thiết bị điện tử. Vi xử lý là bộ não của hệ thống máy tính, chịu trách nhiệm thực hiện các lệnh từ phần mềm và điều khiển các hoạt động của hệ thống. Kiến trúc vi xử lý bao gồm các thành phần chính như đơn vị luận lý số học, các thanh ghi, bộ điều khiển và các bus để kết nối với bộ nhớ và các thiết bị ngoại vi.

Sự phát triển của kiến trúc vi xử lý đã mang lại nhiều cải tiến về hiệu suất, tiết kiệm năng lượng, và khả năng xử lý đa nhiệm, đóng góp quan trọng vào sự tiến bộ của công nghệ thông tin và truyền thông. Vi xử lý hiện đại không chỉ được sử dụng trong máy tính cá nhân, mà còn trong các thiết bị di động, hệ thống nhúng và các thiết bị thông minh khác, đáp ứng nhu cầu ngày càng cao của người dùng và các ứng dụng công nghệ.

2.2.2. Kiến trúc máy tính Von Neumann

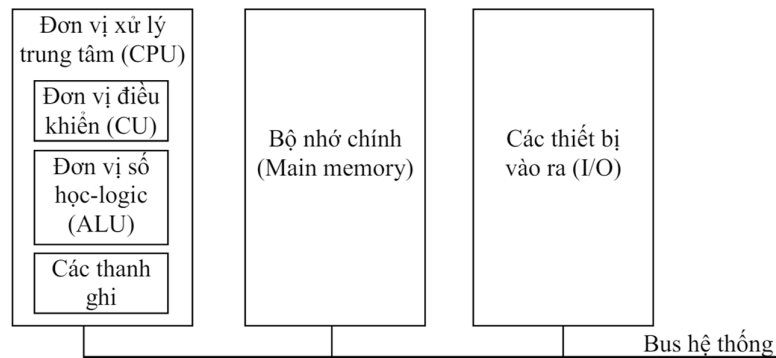
Kiến trúc Von Neumann, được đề xuất bởi nhà toán học và nhà khoa học máy tính John Von Neumann vào giữa thế kỷ 20, là một trong những mô hình cơ bản nhất của máy tính.



Hình 2.7: Kiến trúc máy tính Von Neumann nguyên thủy

Hình 2.7 mô tả kiến trúc máy tính Von Neumann nguyên thủy, cho thấy tất cả các vi xử lý máy tính đều dựa trên kiến trúc này vì chúng có các thành phần cơ bản

chung gồm: Bộ điều khiển, Đơn vị số học và logic, Bộ nhớ và các thiết bị đầu vào/ra. Bộ điều khiển quản lý tất cả các tín hiệu điều khiển của vi xử lý. Nó điều khiển luồng dữ liệu vào và ra, tìm nạp lệnh, và kiểm soát cách thức dữ liệu được chuyển xung quanh hệ thống. Đơn vị lưu trữ số học là một thành phần của CPU chịu trách nhiệm xử lý các phép toán cần thiết như phép cộng, phép trừ và các phép so sánh. ALU thực hiện các thao tác toán học, logic và dịch chuyển bit theo yêu cầu. Bộ nhớ dùng để lưu trữ chương trình và dữ liệu. Các thiết bị đầu vào và đầu ra liên quan đến việc đọc và ghi dữ liệu từ các thiết bị ngoại vi.



Hình 2.8: Kiến trúc máy tính Von Neumann hiện đại

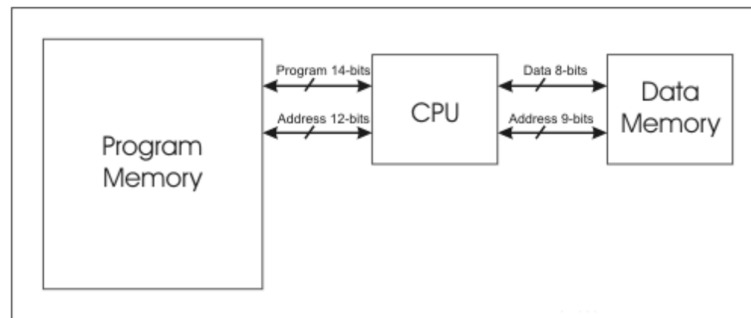
Hình 2.8 cho ta thấy một biến thể cải tiến của kiến trúc máy tính Von Neumann gốc, đã trở thành tiêu chuẩn cho các máy tính hiện đại ngày nay. Ba ý tưởng chính tạo nên cơ sở của kiến trúc này là: bộ nhớ chung đọc-ghi, vị trí xác định cách truy cập vào bộ nhớ và các chỉ thị chương trình được thực hiện theo thứ tự tuần tự.

Trong kiến trúc máy tính Von Neumann hiện đại, dữ liệu và các chỉ thị được lưu trữ cùng nhau trong một bộ nhớ duy nhất, cho các hai mục đích đọc và ghi. Điều này giúp tăng cường hiệu suất và linh hoạt trong việc truy cập dữ liệu và chương trình. Vị trí xác định cách truy cập vào bộ nhớ, không phải là nội dung của nó, giúp đơn giản hóa quá trình truy cập và quản lý bộ nhớ.

Các chỉ thị chương trình được thực hiện theo thứ tự tuần tự, từ đầu đến cuối, giúp đảm bảo sự tuân thủ và dễ dàng theo dõi luồng thực thi của chương trình. Quá trình thực thi lệnh được chia thành ba giai đoạn chính: CPU lấy chỉ thị từ bộ nhớ, sau đó đọc dữ liệu từ bộ nhớ nếu cần, giải mã và thực thi chỉ thị. Khi chỉ thị được

thực thi, CPU ghi kết quả trở lại bộ nhớ (nếu có), hoàn thành quá trình thực thi của lệnh. Điều này tạo ra một quy trình mạch lạc và đảm bảo tính chính xác trong thực thi chương trình.

2.2.3. Kiến trúc máy tính Harvard



Hình 2.9: Kiến trúc máy tính Harvard

Trong một máy tính thông thường theo kiến trúc von Neumann, các lệnh và dữ liệu đều được lưu trữ trong cùng một bộ nhớ và cùng một bus được sử dụng để lấy lệnh và truyền dữ liệu. Điều này có nghĩa là CPU không thể thực hiện cả hai việc cùng lúc đọc lệnh và đọc/ghi dữ liệu. Ngược lại, Kiến trúc Harvard có thể lưu trữ và bus riêng biệt cho lệnh và dữ liệu. Nó được phát triển để khắc phục nút cổ chai của kiến trúc von Neumann. Ưu điểm chính của việc có bus riêng biệt cho lệnh và dữ liệu là CPU có thể truy cập lệnh và đọc/ghi dữ liệu đồng thời, tăng hiệu suất hoạt động.

Hình 2.9 mô tả về kiến trúc máy tính Harvard chia bộ nhớ thành hai loại riêng biệt: bộ nhớ chương trình và bộ nhớ dữ liệu. Bộ nhớ chương trình và bộ nhớ dữ liệu được kết nối với CPU thông qua hai hệ thống bus khác nhau. Mỗi hệ thống bus gồm ba phần: tín hiệu điều khiển, tín hiệu dữ liệu và tín hiệu địa chỉ. Vì kiến trúc máy tính Harvard cho phép hai hệ thống bus riêng biệt với dung lượng lớn hơn, máy tính được xây dựng trên kiến trúc này có thể đạt được tốc độ xử lý nhanh hơn so với máy tính dựa trên kiến trúc von máy tính Neumann. Ngoài ra, hệ thống bộ nhớ của kiến trúc máy tính Harvard cho phép thực hiện nhiều lệnh truy cập bộ nhớ cùng một lúc, giảm xung đột truy cập bộ nhớ, đặc biệt khi CPU sử dụng kỹ thuật đường ống. Điều này có thể thực hiện được nhờ hai hệ thống bus độc lập.

2.2.4. Kiến trúc máy tính MIPS

2.2.4.1. Giới thiệu về MIPS

Kiến trúc xử lý theo phong cách RISC được gọi là MIPS, đã được phát triển bởi MIPS Technologies. Vào đầu những năm 1980, chương trình nghiên cứu VLSI của Đại học Stanford đã tham gia vào việc phát triển các bộ xử lý MIPS. Phiên bản đầu tiên của MIPS, R2000, đã được phát hành vào năm 1985, và sau đó là phiên bản MIPS R3000 vào năm 1988.

Kiến trúc MIPS dựa trên thanh ghi là nền tảng của thiết kế này. Các thanh ghi, còn được gọi là các hằng số được lưu trữ như một phần của các lệnh, là cơ sở của tất cả các phép toán số học và logic. Một số lệnh cơ bản để đọc dữ liệu từ bộ nhớ vào các thanh ghi và ghi dữ liệu từ các thanh ghi vào bộ nhớ cũng được bao gồm trong kiến trúc MIPS.

Từ MIPS I ISA đến MIPS V ISA, ISA MIPS đã trải qua sự tiến hóa. Vào cuối những năm 1990, kiến trúc MIPS dựa trên hai kiến trúc cơ bản:

- ❖ MIPS32 cho kiến trúc 32 bit và MIPS64 cho kiến trúc 64 bit.
- ❖ Kiến trúc MIPS 32 bit bổ sung các lệnh từ các ISA MIPS III đến MIPS V vào ISA MIPS II, trong khi tập lệnh của ISA MIPS V là nền tảng cho kiến trúc MIPS 64 bit.

2.2.4.2. Thành phần cơ bản của kiến trúc máy tính MIPS

Kiến trúc MIPS bao gồm các thành phần sau:

- **Kiến trúc tập lệnh MIPS:** Định nghĩa một tập các câu lệnh 32 bit và 64 bit để bảo đảm tương thích với một bộ vi xử lý áp dụng kiến trúc MIPS. Tập lệnh MIPS bao gồm tổng cộng 111 câu lệnh ở hệ 32 bit, ví dụ như một câu lệnh MIPS dưới đây:
add \$r12, \$r7, \$r8 000000_00111_01000_01100_00000_010100
- **Kiến trúc tài nguyên phân quyền:** Đưa ra một tập các môi trường đảm bảo cho ISA hoạt động. PRA cung cấp các cơ chế cần thiết để quản lý các tài nguyên của bộ vi xử lý: bộ nhớ ảo, cache...

- **Các thành phần mở rộng tùy chọn cho ứng dụng:** Cung cấp các chức năng nâng cao được thiết kế riêng biệt cho các chức năng khác nhau.

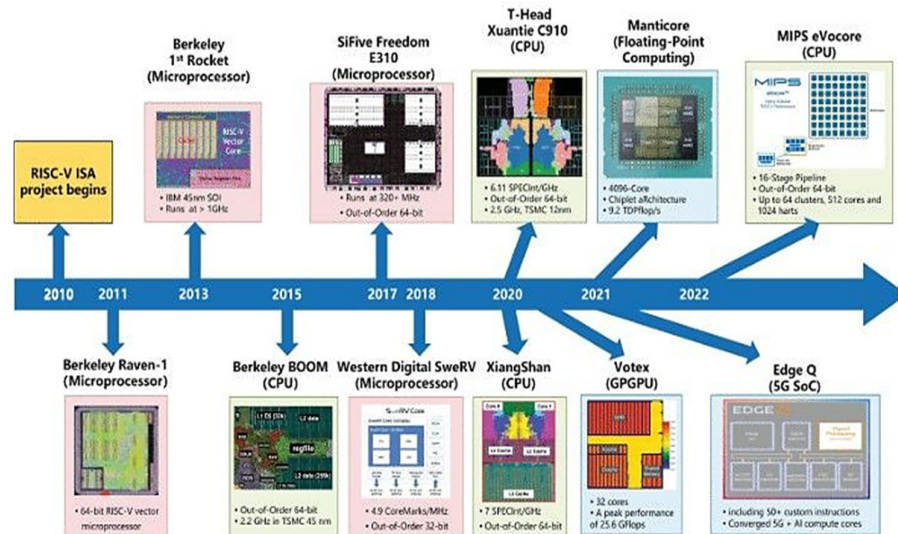
Trên thực tế, máy tính hoạt động bằng cách lấy các lệnh ngôn ngữ máy từ bộ nhớ, giải mã và thực hiện chúng. Ngôn ngữ máy và các giá trị sẽ được tính toán và mã hóa thành mã nhị phân. Máy tính xây dựng một kiến trúc máy tính sử dụng kiến trúc MIPS thì các thành phần cơ bản cần bao gồm: Bộ đếm chương trình, bộ nhớ, tập hợp các lệnh thanh ghi, tập hợp các thanh ghi, khối luận lý số học và khối điều khiển.

2.3. KIẾN TRÚC TẬP LỆNH RISC-V

2.3.1. Tổng quan

RISC-V là một kiến trúc tập lệnh mã nguồn mở được sử dụng để phát triển bộ xử lý tùy chỉnh cho nhiều ứng dụng khác nhau, từ thiết kế nhúng đến siêu máy tính. Nó là một ISA nguồn mở và đã trở nên rất phổ biến trong những năm gần đây. RISC-V được phát triển ban đầu tại Đại học California, Berkeley và hiện được quản lý bởi RISC-V International, một tổ chức có nhiều thành viên và đã có hàng tỷ chip chứa lõi RISC-V được sản xuất. Có nhiều triển khai RISC-V, bao gồm cả lõi nguồn mở và sản phẩm sở hữu trí tuệ thương mại[10].

2.3.2. Lịch sử và phát triển của RISC-V



Hình 2.10: Sự phát triển của RISC-V

ISA độc quyền do các công ty kiểm soát chặt chẽ, hạn chế quyền truy cập và áp đặt chi phí cấp phép đã làm chậm sự đổi mới và cạnh tranh. Để khắc phục điều này, RISC-V, một ISA mã nguồn mở, đã được phát minh vào năm 2010 tại Đại học California, Berkeley, nhằm giải quyết các hạn chế của ISA độc quyền và cung cấp nền tảng cho thiết kế bộ xử lý trong tương lai [11].

Phiên bản RV32I của RISC-V, tập lệnh cơ bản đầu tiên, ra mắt năm 2011, nhấn mạnh sự đơn giản và hiệu quả theo nguyên lý RISC. Năm 2015, Hiệp hội RISC-V được thành lập với hơn 200 tổ chức thành viên, thúc đẩy sự chấp nhận và tiêu chuẩn hóa ISA RISC-V. ISA RISC-V đã được nhiều công ty áp dụng cho các ứng dụng từ vi xử lý vi mạch và hệ thống nhúng đến máy tính đa năng và bộ xử lý trung tâm dữ liệu. Sự phát triển của RISC-V được thúc đẩy bởi nhu cầu về tính tùy chỉnh, giảm phụ thuộc vào ISA độc quyền và các giải pháp tiết kiệm năng lượng. Bằng cách cung cấp một ISA mở, linh hoạt và mở rộng, RISC-V đã mở ra một kỷ nguyên mới về đổi mới trong thiết kế bộ xử lý, có tiềm năng thay đổi cảnh quan ngành công nghiệp bán dẫn [11][12].

2.3.3. Nguyên tắc thiết kế của RISC-V

ISA RISC-V dựa trên nguyên tắc đơn giản hóa tập lệnh, mô đun và mở rộng, tạo điều kiện cho việc tạo ra bộ xử lý tùy chỉnh, hiệu suất cao và linh hoạt cho nhiều ứng dụng khác nhau.

Theo nguyên tắc "Máy tính với tập lệnh đơn giản hóa", RISC-V cung cấp một tập lệnh đơn giản và tổng quát, tối ưu hóa hiệu suất và tiêu thụ năng lượng. So với CISC, RISC có ít lệnh mỗi chu kỳ, lệnh đơn giản và chuẩn hóa, thực thi nhanh hơn và tiêu thụ năng lượng thấp hơn.

ISA RISC-V được tổ chức thành các thành phần độc lập, có khả năng kết hợp để tạo ra bộ xử lý tùy chỉnh, đáp ứng các yêu cầu về hiệu suất, tiêu thụ năng lượng và thiết kế khác nhau. Khả năng mở rộng cho phép thêm các chỉ thị và tính năng mới mà không ảnh hưởng đến tính tương thích phần mềm, mang lại tính linh hoạt và tùy chỉnh cao cho các ứng dụng đặc biệt [12].

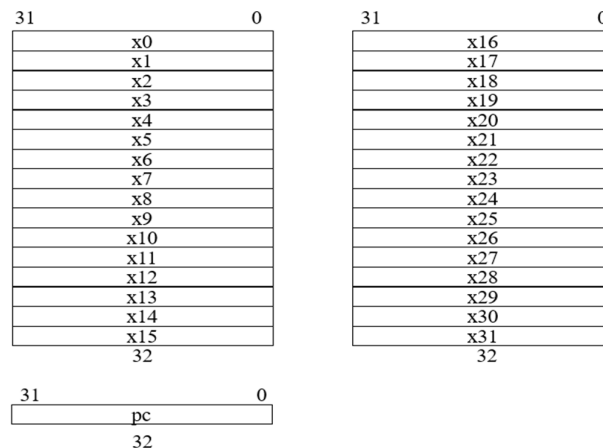
2.3.4. Kiến trúc tập lệnh số nguyên cơ sở RV32I

2.3.4.1. Đặc điểm và cấu trúc

RV32I là ISA 32 bit đơn giản cho số nguyên với 47 lệnh, gồm 8 lệnh hệ thống và 39 lệnh tính toán, điều khiển, truy cập bộ nhớ. Kiến trúc load-store của RISC-V sử dụng lệnh load và store để chuyển dữ liệu vào và ra khỏi bộ nhớ, còn lệnh tính toán hoạt động trên thanh ghi.

2.3.4.2. Thanh ghi

Kiến trúc RISC-V sử dụng 31 thanh ghi số nguyên 32 bit từ x1 đến x31 Hình 2.11. Thanh ghi **x0** là hằng số 0, dùng làm thanh ghi đích. Thanh ghi **ra** lưu địa chỉ trở về sau khi gọi hàm. Thanh ghi **sp** là con trỏ ngăn xếp, quản lý vị trí trên ngăn xếp. Thanh ghi **gp** là con trỏ toàn cục, truy cập dữ liệu toàn cục. Thanh ghi **tp** là con trỏ luồng, hỗ trợ chương trình đa luồng. Thanh ghi **t0-t6** là thanh ghi tạm thời cho dữ liệu tạm thời. Thanh ghi **s0/fp** là con trỏ khung, dùng như con trỏ cơ sở cho khung ngăn xếp hiện tại. Thanh ghi **s1-s11** dùng lưu trữ dữ liệu qua các lời gọi hàm. Các thanh ghi **a0-a7** (tương ứng với x10-x17) là các thanh ghi đối số. Ngoài ra, có một thanh ghi **pc** bổ sung là bộ đếm chương trình, giữ địa chỉ byte của lệnh hiện tại. Tổng cộng, trạng thái kiến trúc mà người dùng có thể nhìn thấy có độ dài 1024 bit[13].



Hình 2.11: Trạng thái kiến trúc hiển thị cho RV32I

2.3.4.3. Mã hóa lệnh

Bảng 2.1 mô tả phân bố mã hoạt động chính của RV32I. Các mã hoạt động chính có độ dài 7 bit, nhưng trong ISA cơ bản, hai bit thấp nhất được đặt thành 11. Khoảng $\frac{3}{4}$ không gian mã hóa còn lại được sử dụng cho các ISA mở rộng nhằm cải thiện mật độ mã. RV32I sử dụng 11 trong số 32 mã hoạt động chính còn lại. Các ISA cơ bản khác sử dụng thêm 4 mã hoạt động chính, và các mở rộng cơ bản sử dụng 8 mã hoạt động chính. Các chi tiết chi tiết về các lệnh RV32I và cách mã hóa chúng có thể được tìm thấy trong Phụ lục của tài liệu.

Bảng 2.1: Sơ đồ mã hoạt động chính RV32I

inst[4:2]	000	001	010	011	100	101	110	111
inst[6:5]								
00	Tải	F Ext		Fences	Số học	AUIPC	RV64I	
01	Ghi	F Ext		A Ext	Số học	LUI	RV64I	
10	F Ext	F Ext	F Ext	F Ext	F Ext		RV128I	
11	Nhánh	JALR		JAL	Hệ thống		RV128I	

Trong RV32I, lệnh có độ dài 32 bit và được lưu trữ trong bộ nhớ theo căn chỉnh tự nhiên và thứ tự byte little-endian. Có sáu định dạng lệnh mô tả trong Hình 2.12, bao gồm 47 lệnh với 4 định dạng chính: R, I, S, U và 2 biến thể B và J. Các lệnh trong các định dạng này sử dụng tối đa hai toán hạng thanh ghi được xác định bởi rs1 và rs2 và tạo ra tối đa một kết quả thanh ghi được xác định bởi rd.

31	27 26 25 24	20 19	15 14	12 11	7 6	0	
funct7	rs2	rs1	funct3	rd	opcode		R-type
imm[11:0]		rs1	funct3	rd	opcode		I-type
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode		S-type
imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode		B-type
imm[31:12]				rd	opcode		U-type
imm[20 10:1 11 19:12]				rd	opcode		J-type

Hình 2.12: Định dạng lệnh của RV32I

Các loại lệnh của RV32I được tạo nên bởi các trường:

Các trường quan trọng của lệnh trong RV32I bao gồm:

- Funct7: 7 bit, xác định thêm thông tin về phép toán.
- rs2, rs1, rd: Mỗi trường gồm 5 bit, đại diện cho hai thanh ghi nguồn (rs1, rs2) và thanh ghi đích (rd).
- Funct3: 3 bit, xác định loại phép toán cụ thể.
- Opcode: 7 bit, phân biệt các loại lệnh.
- Imm: Chứa giá trị tạm thời, thường là hằng số hoặc địa chỉ lưu trong bộ nhớ.

Một đặc điểm quan trọng của mã hóa là các chỉ định thanh ghi luôn chiếm cùng một vị trí trong lệnh, cho phép việc lấy thanh ghi được thực hiện song song với việc giải mã lệnh, giảm thiểu đường dẫn quan trọng trong nhiều thiết kế [13].

❖ Lệnh loại R:

31	27 26 25 24	20 19	15 14	12 11	7 6	0
funct7		rs2	rs1	funct3	rd	opcode
7		5	5	3	5	7
0000000		rs2	rs1	ADD/SLT/SLTU	rd	op-R
0000000		rs2	rs1	AND/OR/XOR	rd	op-R
0000000		rs2	rs1	SLL/SRL	rd	op-R
0100000		rs2	rs1	SUB/SRA	rd	op-R

Hình 2.13: Cấu trúc lệnh loại R

Lệnh loại R trong RV32I được sử dụng cho các hoạt động từ thanh ghi đến thanh ghi, bao gồm hai thanh ghi nguồn và một thanh ghi đích. Các lệnh gồm :

- Phép toán số học và luận lý: ADD, SUB, OR, XOR, AND.
- Phép so sánh: SLT, SLTU.
- Phép dịch: SLL, SRL, SRA.

❖ Lệnh loại I:

31	20 19	15 14	12 11	7 6	0
imm[11:0]		rs1	funct3	rd	opcode
12		5	3	5	7
immediate[11:0]		rs1	ADDI/SLTI[U]	rd	op-I
immediate[11:0]		rs1	ANDI/ORI/XORI	rd	op-I
0000000	shamt[4:0]	rs1	SLLI/SRLI	rd	op-I
0100000	shamt[4:0]	rs1	SRAI	rd	op-I
immediate[11:0]		rs1	LB[U]/LH[U]/LW	rd	op-LOAD
immediate[11:0]		rs1	000	rd	op-JALR

Hình 2.14: Cấu trúc lệnh loại I

Lệnh loại I trong RV32I được sử dụng cho các phép toán tức thời, bao gồm hai toán hạng thanh ghi và một giá trị tức thời 12 bit. Các lệnh này bao gồm:

- Phép toán số học và luận lý với giá trị tức thời: ADDI, ANDI, ORI, XORI.
- Phép so sánh với giá trị tức thời: SLTI, SLTIU.
- Phép dịch theo giá trị tức thời: SLLI, SRLI, SRAI.
- Lệnh tải: LW, LB, LH.
- Lệnh nhảy không điều kiện: JALR

❖ Lệnh loại S

31	27 26 25 24	20 19	15 14	12 11	7 6	0
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
7		5	5	3	5	7
immediate[11:5]		rs2	rs1	SB/SH/SW	immediate[4:0]	op-S

Hình 2.15: Cấu trúc lệnh loại S

Lệnh loại S được sử dụng cho các hoạt động lưu trữ, lưu trữ dữ liệu từ thanh ghi vào bộ nhớ, bao gồm hai toán hạng thanh ghi và giá trị tức thời 12 bit cho phần bù địa chỉ bộ nhớ[13].

❖ Lệnh loại U

31	12 11	7 6	0
imm[31:12]		rd	opcode
20		5	7
immediate[31:12]		rd	op-LUI
immediate[31:12]		rd	op-AUIPC

Hình 2.16: Cấu trúc lệnh loại U

Lệnh loại U trong kiến trúc RISC-V được sử dụng cho các hoạt động có giá trị tức thời 20 bit, như tải hằng số 20 bit vào một thanh ghi hoặc thiết lập 20 bit trên của một thanh ghi. Hai lệnh loại U phổ biến là:

- LUI : Lưu ngay giá trị 20 bit vào thanh ghi đích.
- AUIPC : Thêm phần bù vào địa chỉ PC hiện có.

❖ Lệnh loại B

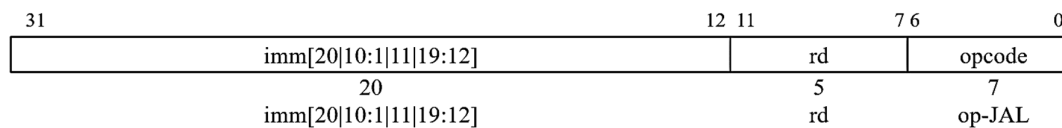
31	30	27 26 25 24	20 19	15 14	12 11	8	7	6	0
imm[12 10:5]		rs2	rs1	funct3	imm[4:1 11]	opcode			
7		5	5	3	5	7			
imm[12 10:5]		rs2	rs1	BEQ/BNE	imm[4:1 11]	op-B			
imm[12 10:5]		rs2	rs1	BLT[U]	imm[4:1 11]	op-B			
imm[12 10:5]		rs2	rs1	BGE[U]	imm[4:1 11]	op-B			

Hình 2.17: Cấu trúc lệnh loại B

Lệnh loại B được sử dụng cho các hoạt động nhánh có điều kiện, chuyển điều khiển sang một lệnh khác dựa trên một điều kiện. Chúng bao gồm hai toán hạng thanh ghi và giá trị tức thời 12 bit cho địa chỉ đích nhánh[13].

- BEQ, BNE: So sánh bằng, nhảy tới một địa chỉ nhất định nếu hai thanh ghi bằng hoặc khác nhau.
- BLT, BLTU: So sánh bé hơn, nhảy tới một địa chỉ nhất định nếu giá trị trong một thanh ghi bé hơn giá trị trong một thanh ghi khác.
- BGE, BGEU: So sánh lớn hơn hoặc bằng, nhảy tới một địa chỉ nhất định nếu giá trị trong một thanh ghi lớn hơn hoặc bằng giá trị trong một thanh ghi khác.

❖ Lệnh loại J



Hình 2.18: Cấu trúc lệnh loại J

Lệnh loại J được sử dụng cho các hoạt động nhảy vô điều kiện, chuyển điều khiển sang lệnh khác một cách vô điều kiện. Chúng bao gồm một toán hạng thanh ghi và giá trị tức thời 20 bit cho địa chỉ đích nhảy[13].

2.3.4.4. Lệnh tính toán

RV32I bao gồm 21 lệnh tính toán, bao gồm các phép toán số học, luận lý và so sánh. Các lệnh này hoạt động trên các thanh ghi số nguyên và một số lệnh có thể sử dụng toán hạng tạm thời. Chúng hỗ trợ cả số nguyên có dấu và không dấu, với số nguyên có dấu sử dụng biểu diễn bù hai. Tất cả toán hạng tạm thời đều được mở rộng dấu, ngay cả khi đại diện cho giá trị không dấu, giúp giảm độ phức tạp của ISA và cải thiện hiệu suất trong một số trường hợp[13].

2.3.4.5. Lệnh truy cập bộ nhớ

RV32I cung cấp năm lệnh để tải giá trị từ bộ nhớ vào thanh ghi số nguyên và ba lệnh để lưu giá trị từ thanh ghi vào bộ nhớ. Tất cả các lệnh này sử dụng địa chỉ byte để xác định vị trí bộ nhớ, tạo địa chỉ bằng cách cộng giá trị trong thanh ghi rs1 với toán hạng tạm thời 12 bit được mở rộng dấu. Ngoài ra, còn có hai lệnh

fence để đồng bộ hóa các truy cập đồng thời vào bộ nhớ từ các luồng phân cứng khác nhau[13].

2.3.4.6. *Lệnh điều khiển*

Trong RISC-V, có sáu lệnh nhánh để thay đổi luồng điều khiển và hai lệnh nhảy vô điều kiện. Các lệnh nhánh sử dụng định dạng SB và so sánh số học giữa hai thanh ghi, cho phép chuyển quyền kiểm soát đến một địa chỉ mới trong phạm vi ± 4 KiB. Địa chỉ mới được tính bằng cách cộng giá trị toán hạng 12 bit mở rộng dấu với giá trị hiện tại của bộ đếm chương trình.

Khe trễ nhánh trong kiến trúc RISC cho phép thực thi lệnh ngay sau lệnh nhánh, dù nhánh có thực hiện hay không. Tuy nhiên, trong các đường ống sâu và máy tính với phân phối lệnh siêu tổng hợp, khe trễ trở nên phức tạp và kém hiệu quả, thường phải điền bằng lệnh no-op, làm tăng kích thước mã lệnh.

2.3.4.7. *Lệnh hệ thống*

Trong ISA RISC-V RV32I, có tám lệnh hệ thống. Lệnh ECALL gọi hệ điều hành thực hiện cuộc gọi hệ thống, còn lệnh EBREAK gọi bộ gỡ lỗi. RISC-V không xác định quy ước truyền tham số cho các cuộc gọi hệ thống; thông tin mã siêu dữ liệu không được mã hóa trong lệnh EBREAK mà được lưu trữ trong cấu trúc phụ trợ, chỉ mục bởi bộ đếm chương trình.

Các lệnh CSRRW, CSRRC, CSRRS, CSRRWI, CSRRCI và CSRRSI cho phép sao chép, xóa, hoặc thiết lập các bit trong CSR, và có thể nhận giá trị từ một nguồn nguyên không đặt bit. Một số CSR cấp người dùng trong RV32I bao gồm Cycle Counter, Instret Counter, và Time Register sử dụng cho chẩn đoán hiệu năng.

2.3.4.8. *Kiến trúc tập lệnh số nguyên cơ sở RV32E, RV64I, RV128I*

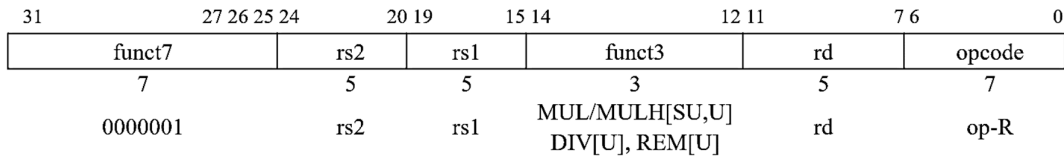
RV32E là một phiên bản đơn giản hóa của kiến trúc RV32I, được thiết kế cho các hệ thống nhúng với yêu cầu về diện tích và năng lượng thấp. Nó giảm số lượng thanh ghi số nguyên từ 32 xuống 16 thanh ghi 32 bit, vẫn tương thích với tập lệnh cơ bản của RISC-V, phù hợp cho các ứng dụng nhúng như IoT.

RV64I là một phiên bản mở rộng của RV32I, hỗ trợ các thanh ghi 64 bit. Điều này cho phép xử lý dữ liệu lớn hơn và địa chỉ bộ nhớ rộng hơn, phù hợp với các hệ thống cần khả năng tính toán và bộ nhớ cao hơn.

RV128I là một phần của dự án RISC-V mở rộng, hướng tới việc hỗ trợ các thanh ghi 128 bit. Điều này cho phép xử lý dữ liệu cực kỳ lớn và địa chỉ bộ nhớ với không gian rất rộng, phù hợp với các hệ thống cần khả năng xử lý dữ liệu và bộ nhớ cực kỳ cao[14].

2.3.5. Phần mở rộng tiêu chuẩn của RISC-V

2.3.5.1. Nhân và chia số nguyên “M”



Hình 2.19: Danh sách các lệnh trong RV32M

Phần mở rộng M của RISC-V là một mô-đun mở rộng cho kiến trúc RISC-V cơ bản, bao gồm các lệnh số học đa dạng hơn, đặc biệt là các lệnh nhân và chia cho số nguyên. Mục tiêu của phần mở rộng này là tối ưu hóa hiệu suất cho các phép tính số nguyên nặng. Hình 2.19 hiển thị các lệnh mà RV32M cung cấp cho mục đích này [15].

2.3.5.2. Nguyên tử “A”

Phần mở rộng chuẩn "A" của RISC-V bao gồm các lệnh nguyên tử dùng để thực hiện các hoạt động đồng bộ hóa và giao tiếp hiệu quả giữa các luồng xử lý, không cần sự can thiệp của hệ điều hành, giúp nâng cao khả năng của RISC-V trong xây dựng các hệ thống nhúng, đám mây và lõi đa xử lý, đặc biệt hữu ích cho các ứng dụng đa luồng và hệ thống phân tán[15].

2.3.5.3. Dấu phẩy động độ chính xác đơn “F”

Phần mở rộng F của RISC-V thêm các lệnh dấu phẩy động vào kiến trúc cơ bản RISC-V, cho phép thực hiện các phép tính dấu phẩy động một cách hiệu quả. Điều này giúp hệ thống xử lý các tác vụ khoa học và kỹ thuật yêu cầu độ chính xác cao và tính toán số học phức tạp mà không phải hy sinh hiệu suất tổng thể.

2.3.5.4. Dấu phẩy động độ chính xác đôi “D”

Phần mở rộng D của RISC-V là mô-đun mở rộng cho kiến trúc cơ bản RISC-V, bổ sung các lệnh dấu phẩy động chính xác kép. Điều này cho phép thực hiện các phép tính dấu phẩy động với độ chính xác cao hơn, hỗ trợ các ứng dụng khoa học và kỹ thuật cần đến tính toán số học phức tạp và độ chính xác cao. RV32D mở rộng khả năng của phần mở rộng F, cho phép các hệ thống RISC-V xử lý các tác vụ tính toán dấu phẩy động chính xác kép một cách hiệu quả[15].

2.3.5.5. Các phần mở rộng tiêu chuẩn khác

RISC-V cung cấp các phần mở rộng M, A, F, và D để bổ sung tính năng tính toán đa mục đích, được gọi chung là G. Ngoài ra, có các phần mở rộng Q và L, tương tự như F và D, nhưng hỗ trợ lệnh dấu phẩy động 128 bit hoặc có độ chính xác gấp bốn lần. RISC-V cũng cho phép thiết kế các phần mở rộng khác nhằm tăng khả năng tùy chỉnh của hệ thống, bao gồm lệnh nén, thao tác bit, hỗ trợ ngôn ngữ dịch động, bộ nhớ giao dịch, SIMD đóng gói và hoạt động vector. Mỗi phần mở rộng này được thiết kế để đáp ứng các yêu cầu cụ thể và có thể được triển khai hoặc bỏ qua tùy thuộc vào mục tiêu thiết kế của hệ thống, giúp RISC-V trở thành một kiến trúc linh hoạt và mạnh mẽ[13][15].

2.3.6. Ưu điểm, nhược điểm của RISC-V

2.3.6.1. Ưu điểm

- **Tính mở và miễn phí bản quyền:** RISC-V là một ISA mở, khuyến khích sự đổi mới và ý tưởng đa dạng từ nhiều nguồn.
- **Tính tùy biến cao:** Khả năng tùy chỉnh cao của RISC-V giúp tối ưu hóa hiệu suất và tiêu thụ năng lượng cho các ứng dụng cụ thể.
- **Cộng đồng mở rộng:** RISC-V có một cộng đồng cộng tác rộng lớn, tạo điều kiện cho việc chia sẻ thông tin và đổi mới ý tưởng.
- **Linh hoạt và đa dạng ứng dụng:** RISC-V có tính linh hoạt cao và có thể được sử dụng trong nhiều dự án khác nhau, từ hệ thống nhúng đến trung tâm dữ liệu.

- **Đáng tin cậy và dễ xác minh:** Các tính năng của RISC-V được đánh giá công khai, giúp kiến trúc trở nên dễ hiểu, xác minh và đáng tin cậy.
- **Sử dụng trong nghiên cứu và giảng dạy:** RISC-V được sử dụng rộng rãi trong lĩnh vực nghiên cứu học thuật và giảng dạy, tạo ra những ý tưởng mới trong cộng đồng[16].

2.3.6.2. Nhược điểm

- **Sự chấp nhận thấp:** RISC-V vẫn chưa nhận được mức độ chấp nhận rộng rãi như các ISA đã được thành lập như x86 và ARM.
- **Hệ sinh thái phần mềm đang phát triển:** Hệ sinh thái phần mềm của RISC-V đang phát triển nhanh chóng nhưng vẫn đang tụt hậu so với sự khả dụng của phần mềm.
- **Hệ sinh thái phần cứng đòi hỏi thời gian phát triển:** Xây dựng một hệ sinh thái phần cứng trưởng thành đòi hỏi nhiều thời gian hơn.
- **Khả năng tương thích giữa các triển khai và phần mở rộng:** Việc đạt được khả năng tương thích hoàn toàn giữa các triển khai và phần mở rộng khác nhau của RISC-V là khó khăn.
- **Không tương thích với phần mềm cũ:** RISC-V không tương thích với phần mềm hiện có được thiết kế cho các ISA khác như x86, có thể yêu cầu việc biên dịch lại phần mềm để tương thích[16].

2.4. KỸ THUẬT ĐƯỜNG ỐNG

2.4.1. Tổng quan về kỹ thuật đường ống

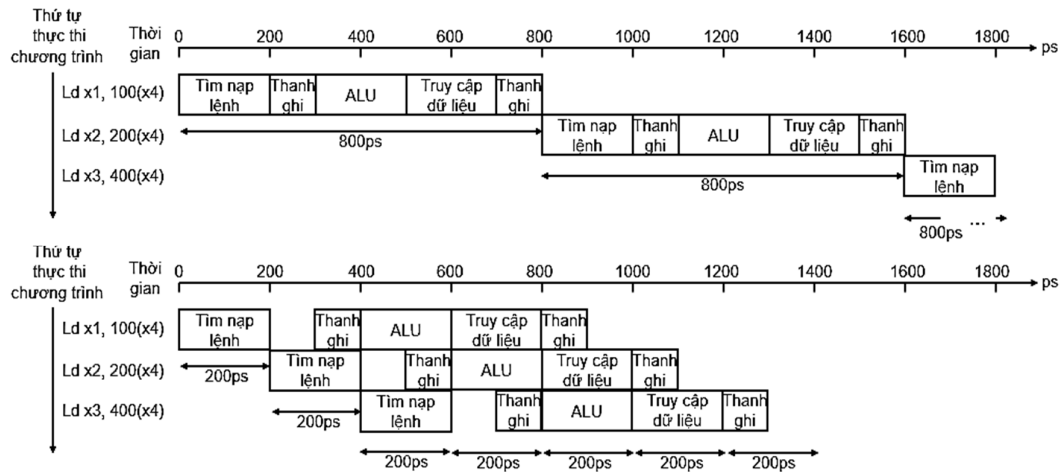
Kỹ thuật đường ống là việc chồng chéo nhiều lệnh trong quá trình thực hiện, giúp tăng hiệu suất CPU bằng cách giảm thiểu thời gian chờ và xử lý nhiều lệnh trong một đơn vị thời gian. Trong kiến trúc máy tính, kỹ thuật này cho phép thực hiện đồng thời nhiều lệnh ở các giai đoạn khác nhau của quá trình xử lý. Mỗi giai đoạn thực hiện một nhiệm vụ cụ thể, và khi một giai đoạn hoàn thành, lệnh tiếp theo có thể bắt đầu giai đoạn đó mà không cần chờ đợi lệnh trước kết thúc. Đây là một phần quan trọng trong thiết kế các bộ vi xử lý hiện đại như Intel Core i7 và ARM Cortex-A53. Trong kiến trúc RISC-V, một lệnh thường trải qua năm bước:

lấy lệnh từ bộ nhớ, đọc và giải mã lệnh, thực hiện phép toán hoặc tính toán địa chỉ, truy cập dữ liệu trong bộ nhớ, và ghi kết quả vào thanh ghi[17].

Bảng 2.2: Tổng thời gian của từng lệnh tính theo thời gian từng giai đoạn

Loại lệnh	Tìm nạp lệnh	Đọc thanh ghi	Hoạt động ALU	Truy cập dữ liệu	Ghi thanh ghi	Tổng thời gian
Loadword (lw)	200ps	100ps	200ps	200ps	100ps	800ps
Storeword (sw)	200ps	100ps	200ps	200ps		700ps
R-format (add, sub, AND, OR, slt)	200ps	100ps	200ps		100ps	600ps
Branch (beq)	200ps	100ps	200ps			500ps

Kỹ thuật đường ống được minh họa qua ví dụ ở Hình 2.20 về một bộ xử lý với 8 lệnh cơ bản. Giả sử các thời gian thực hiện cho các hoạt động như truy xuất bộ nhớ, tính toán ALU, đọc và ghi thanh ghi như Bảng 2.2. Trong mỗi chu kỳ xung nhịp, mỗi lệnh có đúng một chu kỳ để hoàn thành, do đó, chu kỳ xung nhịp phải kéo dài để phù hợp với lệnh chậm nhất, ví dụ như lệnh lw (800ps).



*Hình 2.20: Thực thi chu kỳ đơn, thực thi không đường ống (phía trên)
so với dùng kỹ thuật đường ống (phía dưới)*

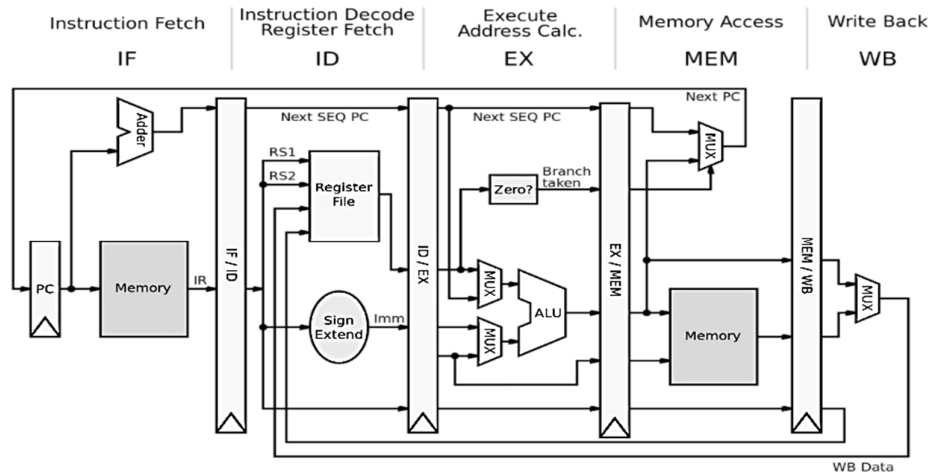
Trong trường hợp không sử dụng kỹ thuật đường ống, thời gian giữa lệnh đầu tiên và lệnh thứ ba là 1600ps. Nhưng nếu áp dụng kỹ thuật đường ống, thời gian

này được cải thiện gấp 4 lần, chỉ còn lại 400ps. Trong trường hợp lý tưởng, tốc độ tăng từ kỹ thuật đường ống gần bằng số lượng giai đoạn của đường ống. Mặc dù có những hạn chế và tác động phụ, kỹ thuật đường ống vẫn cải thiện hiệu suất bằng cách xử lý nhiều lệnh cùng một lúc, đặc biệt là trong các chương trình thực tế[18].

Kỹ thuật đường ống được chia làm 2 loại: Đường ống số học dùng cho phép toán số học như phép nhân và phép toán dấu phẩy động và đường ống lệnh dùng cho phép thực thi đồng thời các giai đoạn tìm nạp, giải mã và thực thi lệnh, tăng hiệu quả xử lý[19].

2.4.2. Hoạt động của đường ống năm tầng

Kỹ thuật đường ống trong xử lý dữ liệu giúp tối ưu hóa thời gian chờ và tăng hiệu suất bằng cách cho phép bộ xử lý thực hiện nhiều lệnh cùng một lúc. Quá trình này tương tự như một dây chuyền sản xuất, với mỗi phân đoạn thực hiện một bước cụ thể như Hình 2.21.



Hình 2.21: Thực hiện chu trình lệnh đường ống số học.

- Lấy lệnh: Lấy lệnh tiếp theo từ bộ nhớ và cập nhật bộ đếm chương trình để trở đến lệnh tiếp theo.
- Giải mã: Giải mã lệnh và đọc các thanh ghi cần thiết, kiểm tra điều kiện nhảy và tính toán địa chỉ nhảy.
- Thực thi: Thực hiện phép toán đã được chuẩn bị từ trước, bao gồm các phép toán tham chiếu bộ nhớ hoặc phép toán ALU.

- Truy nhập bộ nhớ: Nếu có lệnh đọc hoặc ghi dữ liệu, truy cập vào bộ nhớ để thực hiện thao tác đọc hoặc ghi.
- Ghi lại: Kết quả của phép toán được ghi vào thanh ghi tương ứng, bao gồm cả dữ liệu từ bộ nhớ hoặc từ phép toán ALU[20].

2.4.3. Xung đột đường ống

2.4.3.1. Khái niệm và phân loại

Kỹ thuật đường ống giúp giảm độ trễ của lệnh bằng cách chạy chúng đồng thời, nhưng cũng gây ra xung đột đường ống[19]. Có ba loại xung đột gồm xung đột cấu trúc, xung đột dữ liệu và xung đột điều khiển.

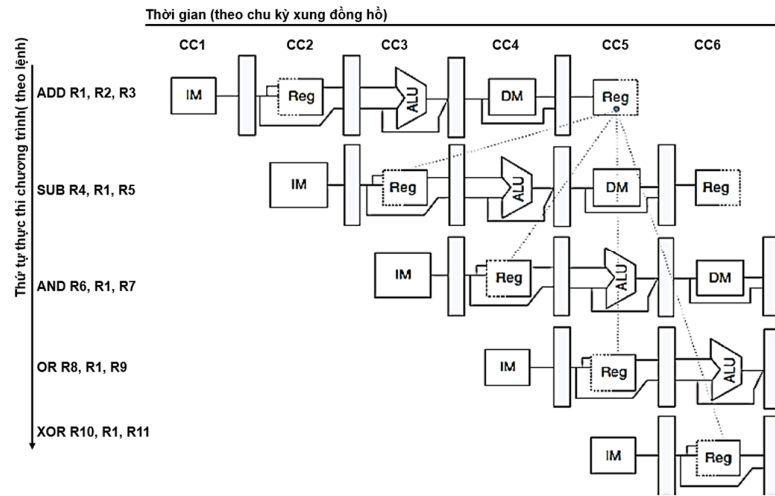
2.4.3.2. Xung đột cấu trúc

Xung đột cấu trúc xảy ra khi nhiều lệnh cố gắng truy cập cùng một tài nguyên trong cùng một chu kỳ xung nhịp, như khi hai lệnh truy cập bộ nhớ cùng lúc. Điều này gây gián đoạn và giảm hiệu suất bộ xử lý. Để giải quyết, có thể tạo "bong bóng" trong đường ống, tạm dừng một số lệnh để giải quyết xung đột, nhưng điều này cũng giảm hiệu suất do tài nguyên bị chiếm dụng. Thiết kế bộ xử lý thường biểu thị sự tạm dừng này bằng biểu đồ chỉ ra chu kỳ không hoạt động [20].

2.4.3.3. Xung đột dữ liệu

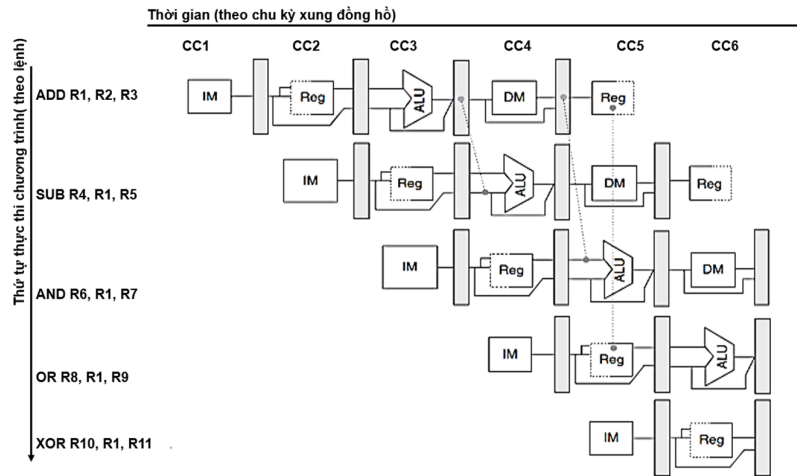
Xung đột dữ liệu là xung đột phát sinh khi một lệnh phụ thuộc vào kết quả của lệnh trước đó vẫn đang trong đường ống. Có ba loại xung đột dữ liệu chính là : RAW, WAR, WAW.

Xem xét ví dụ như Hình 2.22, xung đột dữ liệu xảy ra khi lệnh SUB đọc giá trị R1 trước khi lệnh ADD ghi giá trị mới vào đó. Điều này dẫn đến việc SUB có thể sử dụng một giá trị không chính xác. Lệnh AND cũng gặp vấn đề tương tự khi đọc giá trị R1 trước khi ADD hoàn tất việc ghi. Ngược lại, lệnh XOR và OR không gặp rủi ro này vì chúng đọc giá trị sau khi ADD ghi xong. Xung đột dữ liệu có thể gây ra các lỗi không xác định, sai kết quả hoặc chậm trễ trong quá trình thực thi lệnh. Kỹ thuật chuyển tiếp và trì hoãn có thể được sử dụng để giải quyết xung đột dữ liệu này[20].



Hình 2.22: Sự xung đột trong các lệnh dùng kỹ thuật đường ống.

❖ Chuyển tiếp



Hình 2.23: Các lệnh phụ thuộc kết quả ADD với đường dẫn chuyển tiếp.

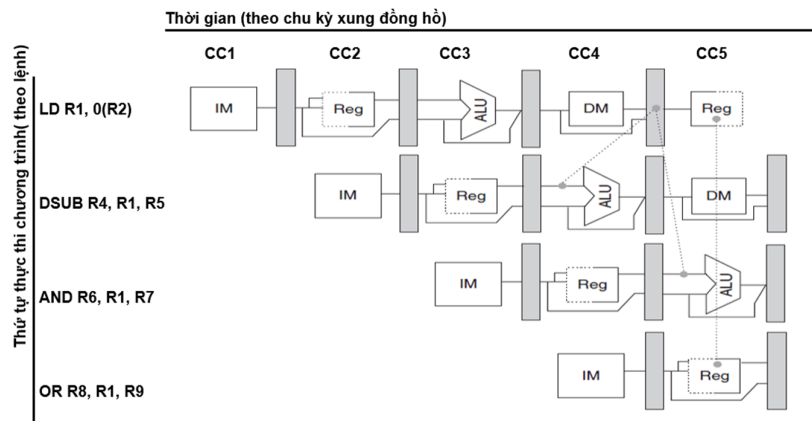
Chuyển tiếp là một kỹ thuật phần cứng đơn giản để giảm thiểu tác động của xung đột dữ liệu. Ý tưởng cơ bản là chuyển kết quả từ lệnh ADD trực tiếp đến lệnh SUB, thay vì đợi SUB đọc giá trị từ thanh ghi. Cụ thể, các bước chuyển tiếp như sau:

- Kết quả ALU từ các bộ nhớ đệm đường ống (EX/MEM và MEM/WB) luôn được đưa trở lại đầu vào ALU.

- Nếu phần cứng chuyển tiếp phát hiện rằng kết quả ALU trước đó đã ghi vào thanh ghi cần dùng cho tính toán hiện tại, nó sẽ chọn kết quả chuyển tiếp thay vì giá trị đọc từ tập tin thanh ghi.

Điều này cho phép kết quả của ADD được truyền trực tiếp đến SUB, tránh được sự chặn hoạt động do phải đợi giá trị từ thanh ghi như mô tả Hình 2.23.

❖ Chậm trễ



Hình 2.24: Lệnh LD bỏ qua kết quả tới AND và OR, không tới SUB.

		Số chu kỳ xung đồng hồ									
		1	2	3	4	5	6	7	8	9	10
LD	R1, 0(R2)	IF	ID	EX	MEM	WB					
DSUB	R4,R1,R5		IF	ID	EX	MEM	WB				
AND	R6,R1,R7			IF	ID	EX	MEM	WB			
OR	R8,R1,R9				IF	ID	EX	MEM	WB		
LD	R1, 0(R2)	IF	ID	EX	MEM	WB					
DSUB	R4,R1,R5		IF	ID	Stall	EX	MEM	WB			
AND	R6,R1,R7			IF	Stall	ID	EX	MEM	WB		
OR	R8,R1,R9				Stall	IF	ID	EX	MEM	WB	

Hình 2.25: Đường ống trước và sau khi chậm trễ

Không phải tất cả xung đột dữ liệu đều có thể giải quyết bằng cách chuyển tiếp. Trong Hình 2.24, xung đột dữ liệu không thể được giải quyết bằng chuyển tiếp vì lệnh LD cần dữ liệu vào cuối chu kỳ xung đồng hồ, còn lệnh DSUB cần dữ liệu từ đầu chu kỳ. Thay vào đó, cần dùng phần cứng "khóa liên động" để duy trì thứ tự thực thi. Khóa liên động phát hiện mâu thuẫn và chèn một "bong bóng" vào chuỗi xử lý, làm tăng độ trễ.

Ví dụ chuỗi lệnh:

LD R1, 0(R2)

DSUB R4, R1, R5

AND R6, R1, R7

OR R8, R1, R9

Khóa liên động sẽ chặn chuỗi xử lý cho đến khi mâu thuẫn được giải quyết, dẫn đến việc chen một “stall” hoặc “bubble” vào chuỗi xử lý, tăng số chu kỳ để hoàn thành chuỗi lệnh thêm một chu kỳ như Hình 2.25 [20].

2.4.3.4. Xung đột điều khiển

Xung đột điều khiển, hay xung đột nhánh, do các lệnh nhánh gây ra khi chúng kiểm soát luồng thực thi và có thể thay đổi PC. Khi lệnh nhánh được thực thi, PC thường không thay đổi cho đến khi kết thúc giai đoạn ID. Cách đơn giản nhất để xử lý nhánh là thực hiện lại việc tìm nạp lệnh theo nhánh khi phát hiện nhánh trong ID[20][21].

Các phương pháp giảm thiểu mất hiệu suất do xung đột nhánh gồm:

- Đóng băng hoặc xóa đường ống: Lệnh sau nhánh bị giữ lại hoặc xóa bỏ cho đến khi biết đích của nhánh, nhưng không giảm thiểu mất hiệu suất.
- Dự đoán không thực hiện: Đường ống tiếp tục lấy lệnh như khi nhánh không thực hiện. Nếu nhánh thực hiện, lệnh đã lấy sẽ bị hủy và lấy lệnh tại đích.
- Dự đoán thực hiện: Khi nhánh được giải mã và địa chỉ đích tính toán, giả định nhánh thực hiện và bắt đầu lấy, thực thi lệnh tại đích.
- Nhánh trễ: Chu kỳ thực thi với nhánh trễ là một, lệnh kế tiếp nằm trong khe trễ và được thực thi dù nhánh có thực hiện hay không.

Trình biên dịch sắp xếp mã nguồn để phù hợp với phần cứng, cải thiện hiệu suất. Phương pháp đóng băng hoặc xóa đường ống đơn giản nhất, nhưng dự đoán không thực hiện hiệu quả hơn, phức tạp hơn một chút, cần chú ý để không thay đổi trạng thái bộ xử lý cho đến khi biết kết quả nhánh [20].

CHƯƠNG 3 : THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1. YÊU CẦU HỆ THỐNG

- Thiết kế dựa trên kiến trúc RISC-V: Vi xử lý cần được thiết kế dựa trên kiến trúc RISC-V, cụ thể là RV32I và RV32M.
- Hệ thống sử dụng kỹ thuật đường ống 5 giai đoạn: Vi xử lý cần được thiết kế với kỹ thuật đường ống 5 giai đoạn, cho phép thực hiện nhiều lệnh cùng lúc.
- Kiến trúc Harvard: Vi xử lý cần tuân theo kiến trúc Harvard, với hai bộ nhớ riêng biệt cho dữ liệu và lệnh.
- Thiết kế bằng Verilog HDL: Vi xử lý cần được thiết kế bằng ngôn ngữ mô tả phần cứng Verilog.
- Kiểm tra và xác minh: Cần thực hiện các bài kiểm tra toàn diện để đảm bảo vi xử lý tuân thủ đầy đủ các quy định của kiến trúc RISC-V.

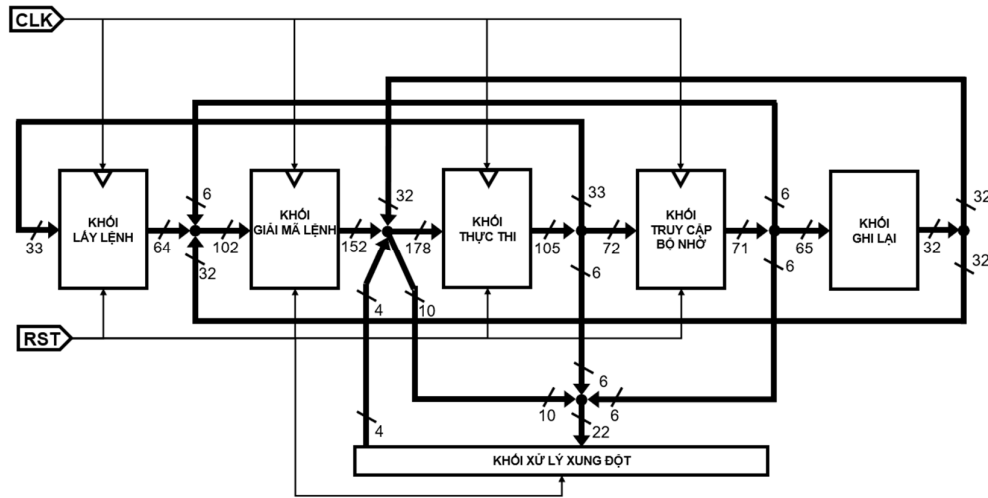
3.2. THIẾT KẾ SƠ ĐỒ KHỐI VÀ CHỨC NĂNG MỖI KHỐI

3.2.1. Sơ đồ khối hệ thống

Hình 3.1 minh họa cấu trúc tổng quan của một hệ thống vi xử lý 32 bit, sử dụng kiến trúc RISC-V và kiến trúc máy tính Harvard với kỹ thuật đường ống 5 giai đoạn. Cấu trúc này bao gồm các khối chức năng như lấy lệnh, giải mã, thực thi, truy cập bộ nhớ, ghi lại và xử lý xung đột. Các khối này được kết nối thông qua 3 loại đường bus: điều khiển, dữ liệu và địa chỉ.

Khối lấy lệnh lấy lệnh từ bộ nhớ để chuẩn bị xử lý. Đây là bước đầu tiên trong chu kỳ của một lệnh. Khối giải mã lệnh có chức năng giải mã lệnh đã lấy từ khối lấy lệnh để xác định chính xác nó cần gì và chuẩn bị các tài nguyên cần thiết để thực hiện cũng như phân tích xuất ra các tín hiệu điều khiển toàn bộ hệ thống. Khối thực thi thực hiện các phép tính hoặc các hoạt động luận lý dựa trên lệnh đã được giải mã ở khối giải mã lệnh. Khối truy cập bộ nhớ có chức năng truy cập vào bộ nhớ để lấy hoặc lưu trữ dữ liệu cần thiết cho hoặc sau quá trình thực hiện lệnh. Khối ghi lại kết quả từ bước thực thi vào một vị trí trong bộ nhớ hoặc thanh ghi.

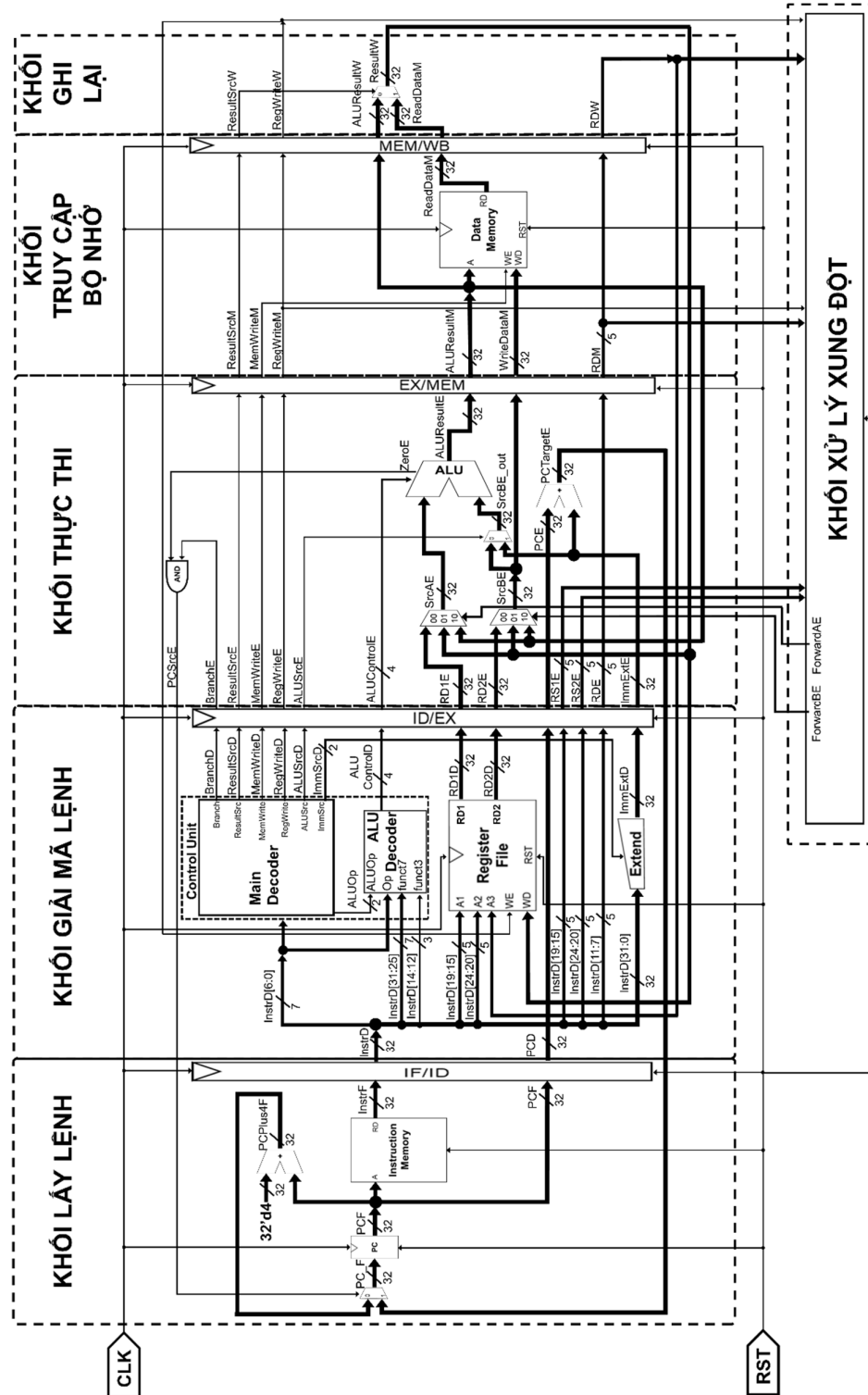
Khối xử lý xung đột giám sát và quản lý các xung đột tiềm ẩn có thể xảy ra khi lệnh tiếp theo trong hàng đợi cần đến dữ liệu hoặc tài nguyên mà lệnh hiện tại đang sử dụng.



Hình 3.1: Sơ đồ tổng quan hệ thống

Ngõ vào Khối lấy lệnh gồm bus 33 đường chứa 1 đường điều khiển và 32 đường dữ liệu từ ngõ ra Khối thực thi. Ngõ vào của Khối giải mã lệnh bao gồm một bus với 102 đường. Trong số này, có 6 đường điều khiển từ ngõ ra của Khối truy cập bộ nhớ, 32 đường dữ liệu từ Khối ghi lại, 32 đường dữ liệu và 32 đường địa chỉ từ Khối lấy lệnh. Ngõ vào của Khối thực thi bao gồm một bus với 178 đường, trong đó có 9 đường điều khiển, 64 đường dữ liệu và 69 đường địa chỉ đều từ Khối giải mã lệnh. Ngoài ra, còn có 32 đường dữ liệu từ Khối ghi lại và 4 đường điều khiển từ Khối xử lý xung đột. Ngõ vào của Khối truy cập bộ nhớ bao gồm một bus với 72 đường, trong đó có 3 đường điều khiển, 64 đường dữ liệu và 5 đường địa chỉ từ Khối thực thi. Ngõ vào của Khối ghi lại bao gồm một bus với 65 đường, trong đó có 1 đường điều khiển và 64 đường dữ liệu từ Khối truy cập bộ nhớ. Ngõ vào của Khối xử lý xung đột bao gồm một bus với 20 đường, trong đó có 10 đường địa chỉ từ Khối giải mã lệnh, 1 đường điều khiển và 5 đường địa chỉ từ Khối thực thi, và 1 đường điều khiển cùng 5 đường địa chỉ từ Khối truy cập bộ nhớ.

3.2.2. Hoạt động hệ thống



Hình 3.2: Sơ đồ chi tiết hệ thống

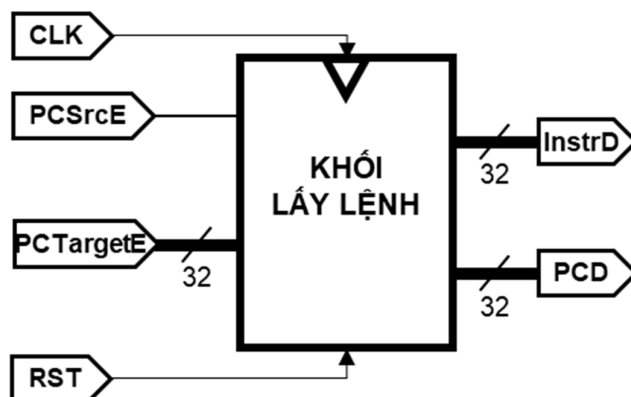
Hệ thống CPU 32 bit RISC-V sử dụng kỹ thuật đường ống 5 tầng để tối ưu hóa hiệu suất và tốc độ xử lý bằng cách chia quá trình thực hiện lệnh thành năm giai đoạn riêng biệt: lấy lệnh, giải mã lệnh, thực thi, truy cập bộ nhớ, và ghi lại.

Trong giai đoạn lấy lệnh, bộ nhớ lệnh cung cấp các lệnh từ địa chỉ do PC chỉ định, và PC được cập nhật để trỏ đến lệnh tiếp theo. Giai đoạn giải mã lệnh giải mã lệnh để xác định các thanh ghi nguồn, thanh ghi đích, và loại thao tác, đồng thời đọc giá trị từ các thanh ghi nguồn và tạo ra các tín hiệu điều khiển cần thiết. Ở giai đoạn thực thi, ALU thực hiện các thao tác tính toán hoặc luận lý dựa trên các giá trị đầu vào và tín hiệu điều khiển, và kết quả được lưu vào thanh ghi trung gian EX/MEM. Giai đoạn truy cập bộ nhớ thực hiện việc đọc hoặc ghi dữ liệu từ hoặc đến bộ nhớ dữ liệu nếu lệnh yêu cầu, và lưu kết quả vào thanh ghi MEM/WB. Cuối cùng, giai đoạn ghi lại ghi kết quả cuối cùng của lệnh vào tệp thanh ghi, cập nhật giá trị của thanh ghi đích.

Hệ thống sử dụng các khối thanh ghi trung gian giữa các giai đoạn để lưu trữ thông tin tạm thời và các đơn vị điều khiển để quản lý dòng chảy dữ liệu và điều khiển các tín hiệu. Khối xử lý xung đột đảm bảo các xung đột dữ liệu và điều khiển được xử lý một cách hiệu quả, sử dụng các kỹ thuật chuyển tiếp dữ liệu. Nhờ vào thiết kế này, hệ thống có thể thực thi nhiều lệnh cùng lúc, tăng cường hiệu suất tổng thể.

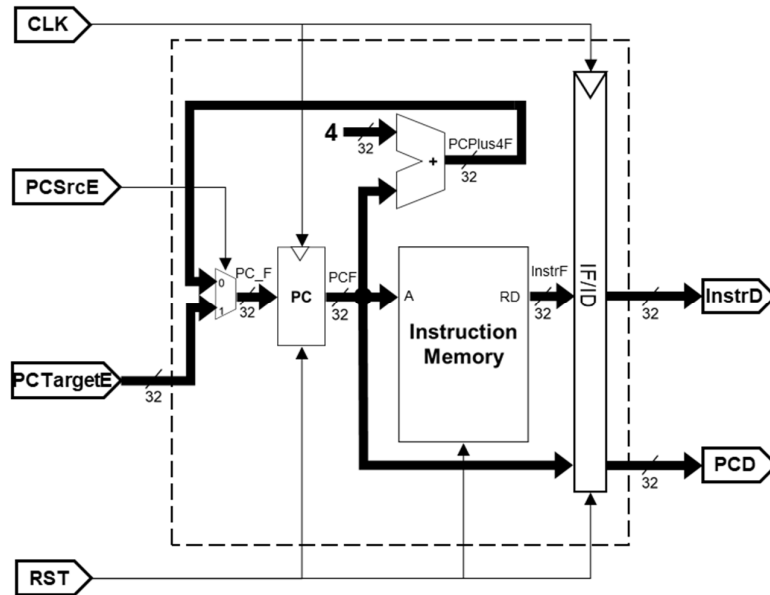
3.2.3. Khối lấy lệnh

Sơ đồ khối: gồm 4 ngõ vào và 2 ngõ ra.



Hình 3.3: Sơ đồ Khối lấy lệnh

Sơ đồ chi tiết gồm các mô đun: bộ ghép kênh chọn đầu vào của chương trình, bộ cộng, bộ đếm chương trình, bộ nhớ lệnh và thanh ghi IF/ID.



Hình 3.4: Sơ đồ chi tiết Khối lấy lệnh

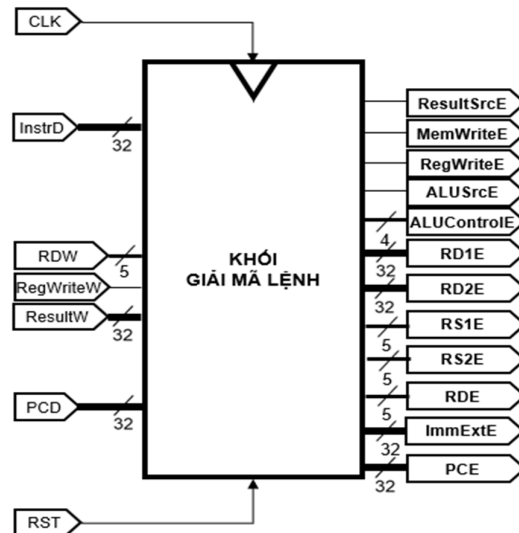
Khối lấy lệnh trong Hình 3.4 thực hiện nhiệm vụ lấy lệnh từ bộ nhớ lệnh và cập nhật bộ đếm chương trình. Dưới đây là các bước hoạt động chi tiết của khối lấy lệnh:

- Tín hiệu $PCSrcE$ và $PCTargetE$:
 - $PCSrcE$: Tín hiệu điều khiển, quyết định nguồn giá trị mới của PC .
 - $PCTargetE$: Địa chỉ đích, được sử dụng khi $PCSrcE = 1$.
 - $PCPlus4F$: Giá trị của PC được cộng thêm 4, sử dụng để trở đến lệnh kế tiếp trong bộ nhớ.
- Bộ ghép kênh chọn đầu vào cho PC .
 - Nếu $PCSrcE = 0$, chọn $PC + 4$ (tăng PC lên 4 để trở đến lệnh kế tiếp).
 - Nếu $PCSrcE = 1$, chọn giá trị từ $PCTargetE$.
- Bộ đếm chương trình lưu trữ giá trị hiện tại của bộ đếm chương trình. Giá trị này được cập nhật mỗi chu kỳ xung đồng hồ.
- Bộ nhớ lệnh, lưu trữ các lệnh của chương trình. Địa chỉ A là giá trị của PC . Đầu ra RD ($InstrF$) là lệnh tại địa chỉ tương ứng.

- Thanh ghi IF/ID là thanh ghi đường ống, lưu trữ lệnh (InstrF) và giá trị PC (PCF) để chuyển sang giai đoạn giải mã lệnh (InstrD và PCD).

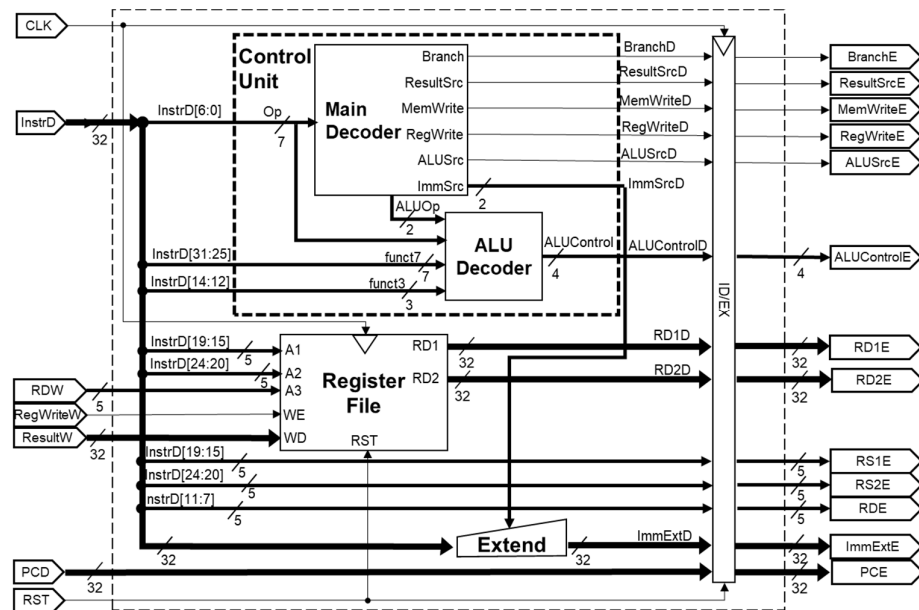
3.2.4. Khối giải mã lệnh

Sơ đồ khối: gồm 7 ngõ vào và 13 ngõ ra.



Hình 3.5: Sơ đồ Khối giải mã lệnh

Sơ đồ chi tiết gồm các mô đun: Đơn vị điều khiển có bộ giải mã chính và bộ giải mã ALU, tập thanh ghi, khối mở rộng và thanh ghi ID/EX.



Hình 3.6: Sơ đồ chi tiết Khối giải mã lệnh

Khởi giải mã lệnh ở Hình 3.6 có nhiệm vụ phân tích lệnh và tạo ra các tín hiệu điều khiển cần thiết để thực hiện lệnh. Quá trình giải mã bao gồm việc xác định loại lệnh, các tham số của lệnh, và điều khiển các đơn vị chức năng khác trong CPU. Dưới đây là mô tả chi tiết các bước hoạt động của khởi giải mã lệnh:

- Bộ giải mã chính gồm tín hiệu đầu vào là Op nhận 7 bit opcode từ lệnh ($\text{InstrD}[6:0]$) để xác định loại lệnh và tạo ra các tín hiệu điều khiển như mô tả tại Bảng:
 - Branch: Tín hiệu nhánh.
 - ResultSrc: Chọn nguồn kết quả (ALU hoặc bộ nhớ).
 - MemWrite: Tín hiệu ghi bộ nhớ.
 - RegWrite: Tín hiệu ghi thanh ghi.
 - ALUSrc: Chọn đầu vào thứ hai của ALU (thanh ghi hoặc hằng số mở rộng).
 - ImmSrc: Chọn loại hằng số mở rộng.
- Bộ giải mã ALU có tín hiệu đầu vào funct7 , funct3 và ALUOp nhận các bit chức năng từ lệnh ($\text{InstrD}[31:25]$, $\text{InstrD}[14:12]$) để tạo ra tín hiệu điều khiển ALU (ALUControlD) như mô tả Bảng 3.2.
- Bộ thanh ghi:
 - A1, A2, A3: Địa chỉ các thanh ghi nguồn và đích từ lệnh ($\text{InstrD}[19:15]$, $\text{InstrD}[24:20]$, $\text{InstrD}[11:7]$).
 - RD1, RD2: Giá trị đọc từ các thanh ghi nguồn.
 - WD: Giá trị ghi vào thanh ghi đích khi thực hiện lệnh ghi.
 - WE: Tín hiệu cho phép ghi thanh ghi.
- Khởi mở rộng hằng số từ lệnh, được tạo ra từ các bit tương ứng của lệnh như Bảng 3.3.
- Thanh ghi ID/EX lưu trữ các tín hiệu điều khiển và giá trị cần thiết để chuyển sang giai đoạn thực thi lệnh.

Bảng 3.1: Bảng trạng thái của tín hiệu điều khiển với từng loại lệnh

Instruction	Op	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
I-Type (loads)	0000011	1	00	1	0	1	0	00
S-Type (Stores)	0100011	0	01	1	1	x	0	00
B-Type (Branch)	1100011	0	10	0	0	x	1	01
I-Type (Arithmetic)	0010011	1	00	1	0	0	0	00
R-Type	0110011	1	xx	0	0	0	0	10

Bảng 3.2: Bảng trạng thái của tín hiệu điều khiển ALU

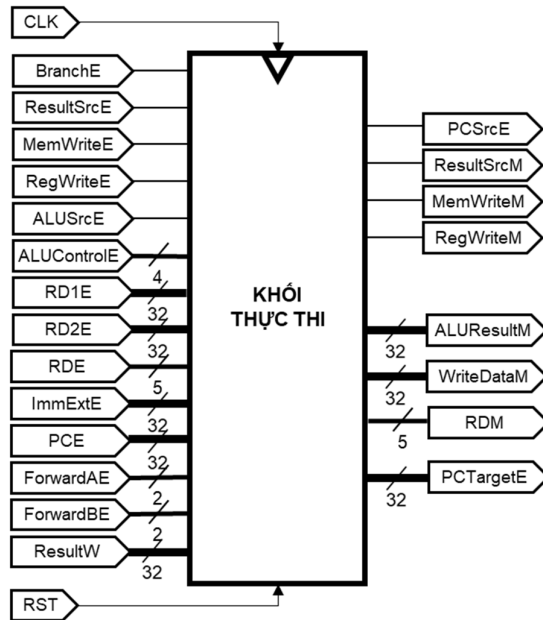
ALUOP	funct3	{op[5],funct7[5],funct7[0]}	ALUControl	Instruction
00	x	x	0000	lw,sw
01	x	x	0001	beq
10	000	000,001,010,011,100,111	0000	add
		110	0001	sub
		101	0010	mul
	001	x	0011	sll
	010	x	0100	slt
	100	xx0	0101	xor
		xx1	0110	div
	101	x0x	0111	srl
		x1x	1000	sra
	110	xx0	1001	or
		xx1	1010	rem
	111	x	1011	and

Bảng 3.3: Mã hóa ImmSrc

ImmSrc	ImmExt	Loại	Mô tả
00	{{20{Instr[31]}}, Instr[31:20]}	I	Giá trị tức thời 12 bit có dấu
01	{{20{Instr[31]}}, Instr[31:20], Instr[11:7]}	S	Giá trị tức thời 12 bit có dấu
10	{{20{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8], 1'b0}	B	Giá trị tức thời 13 bit có dấu

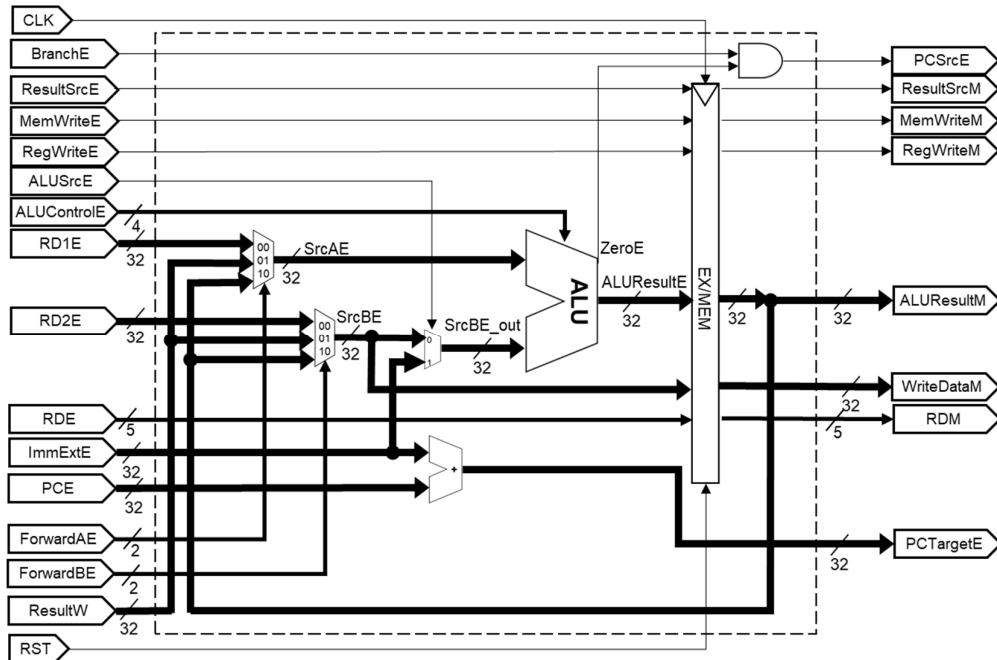
3.2.5. Khối thực thi

Sơ đồ khối: gồm 16 ngõ vào và 8 ngõ ra.



Hình 3.7: Sơ đồ Khối thực thi

Sơ đồ chi tiết gồm: Đơn vị luận lý số học, bộ cộng, bộ ghép kênh, thanh ghi EX/MEM.



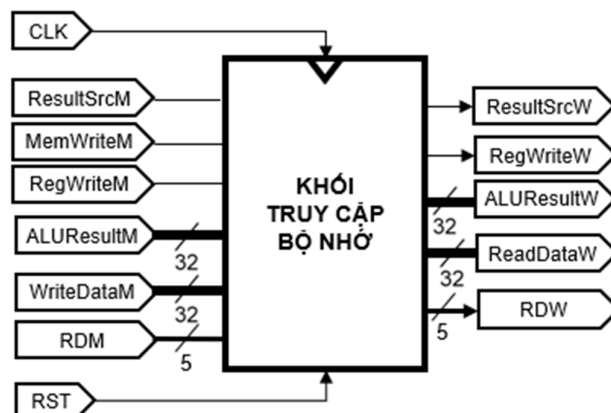
Hình 3.8: Sơ đồ chi tiết Khối thực thi

Khối thực thi ở Hình 3.8 có chức năng chính là thực hiện các phép tính và hoạt động được xác định bởi lệnh, như được giải mã từ khối giải mã lệnh. Chức năng cụ thể của khối thực thi bao gồm:

- ALU thực hiện các phép toán số học và luận lý dựa trên tín hiệu điều khiển ALUControlE. Các giá trị đầu vào cho ALU được chọn từ thanh ghi, giá trị tức thời hoặc các giá trị chuyển tiếp. Tín hiệu ZeroE chỉ ra kết quả ALU bằng không, dùng cho các quyết định nhánh.
- Xử lý xung đột: Các tín hiệu ForwardAE và ForwardBE xử lý xung đột dữ liệu bằng cách chuyển tiếp dữ liệu từ các giai đoạn đường ống sau về giai đoạn thực thi.
- Các bộ chọn lựa chọn giữa dữ liệu chuyển tiếp, dữ liệu thanh ghi hoặc các giá trị tức thời cho các phép tính của ALU.
- Tín hiệu điều khiển: BranchE, ResultSrcE, MemWriteE, RegWriteE, ALUSrcE, ALUControlE điều khiển các hành động của khối thực thi. Tín hiệu ResultW điều khiển kết quả cuối cùng.
- Tín hiệu đầu ra ALUResultM chuyển kết quả từ ALU được chuyển đến giai đoạn bộ nhớ. Tín hiệu WriteDataM ghi dữ liệu vào bộ nhớ. Tín hiệu PCTargetE địa chỉ mục tiêu cho các lệnh nhánh.

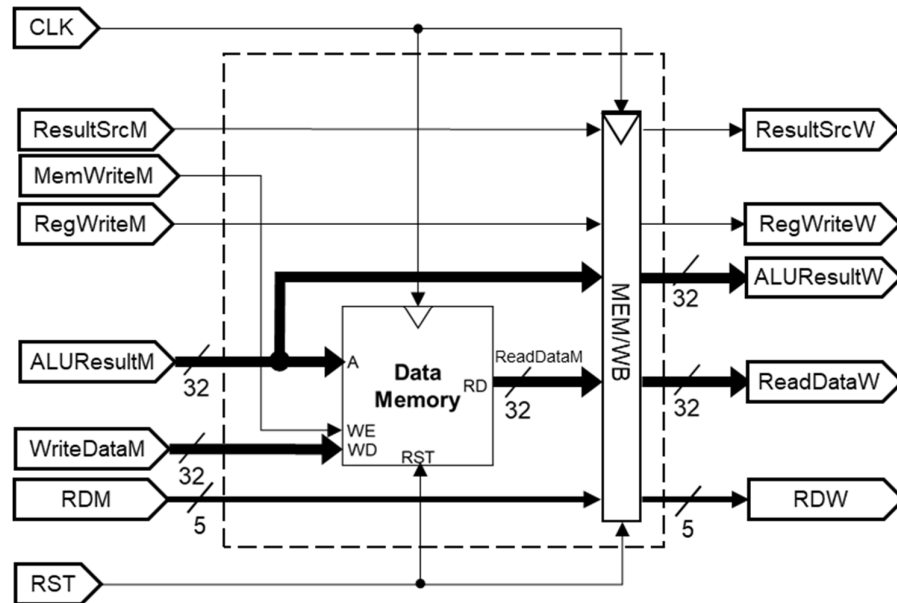
3.2.6. Khối truy cập bộ nhớ

Sơ đồ khối bao gồm có 8 ngõ vào và 5 ngõ ra:



Hình 3.9: Sơ đồ Khối truy cập bộ nhớ

Sơ đồ chi tiết gồm bộ nhớ dữ liệu và thanh ghi MEM/WB



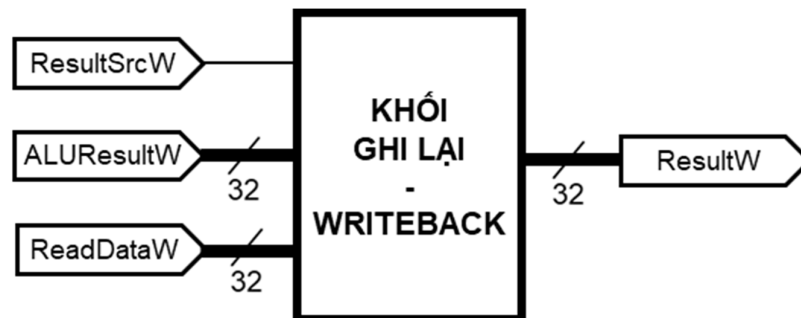
Hình 3.10: Sơ đồ chi tiết Khối truy cập bộ nhớ

Khối truy cập bộ nhớ đọc và ghi dữ liệu vào bộ nhớ chính. Khối truy cập bộ nhớ có các chức năng quan trọng sau:

- Tín hiệu MemWriteM điều khiển cho phép ghi dữ liệu vào bộ nhớ.
- Tín hiệu ALUResultM là địa chỉ từ đơn vị tính toán (ALU) để ghi vào bộ nhớ.
- Tín hiệu WriteDataM là dữ liệu để ghi vào bộ nhớ.
- Thanh ghi MEM/WB lưu trữ các tín hiệu điều khiển và giá trị cần thiết để chuyển sang giai đoạn thực thi lệnh.

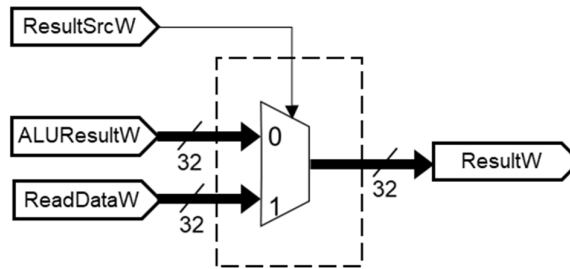
3.2.7. Khối ghi lại

Sơ đồ khối: gồm 3 ngõ vào và 1 ngõ ra.



Hình 3.11: Sơ đồ Khối ghi lại

Sơ đồ chi tiết chỉ có một bộ ghép kênh 2 ngõ vào.



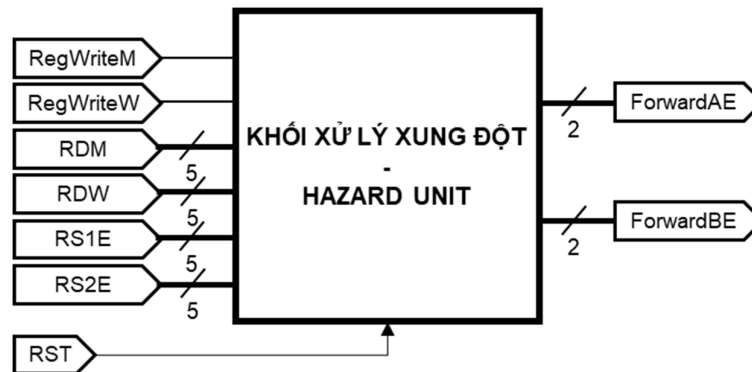
Hình 3.12: Sơ đồ chi tiết Khối ghi lại

Khối ghi lại trong sơ đồ này có chức năng chọn kết quả cuối cùng để ghi vào các thanh ghi trong hệ thống. Hoạt động của khối này có thể được giải thích như sau:

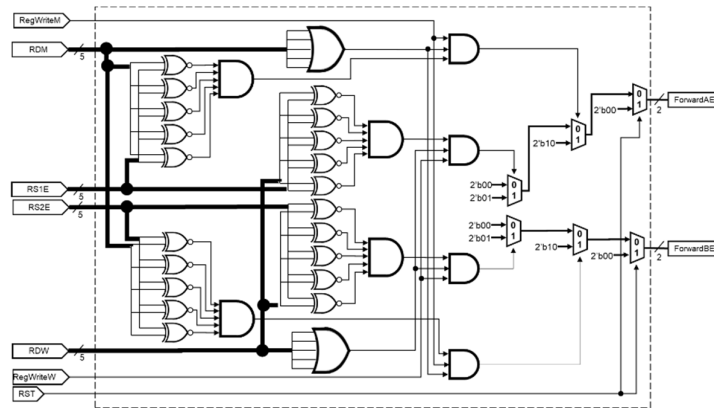
- Tín hiệu ResultSrcW điều khiển để chọn nguồn dữ liệu kết quả. Tín hiệu ALUResultW là dữ liệu kết quả từ đơn vị tính toán (ALU). Tín hiệu ReadDataW là dữ liệu được đọc từ bộ nhớ.
- Bộ chọn quyết định dữ liệu nào sẽ được truyền ra ngoài thông qua tín hiệu điều khiển ResultSrcW.

3.2.8. Khối xử lý xung đột

Khối xử lý xung đột trong sơ đồ này có nhiệm vụ phát hiện và giải quyết các xung đột xảy ra trong quá trình thực hiện lệnh.



Hình 3.13: Sơ đồ Khối xử lý xung đột



Hình 3.14: Sơ đồ chi tiết Khối xử lý xung đột

ForwardAE, ForwardBE được sử dụng để chọn nguồn dữ liệu phù hợp để tránh xung đột:

- Nếu có xung đột, khối sẽ điều khiển để chuyển tiếp dữ liệu từ giai đoạn trước đến giai đoạn hiện tại, đảm bảo rằng lệnh hiện tại có được dữ liệu chính xác mà không cần chờ đợi.
- Nếu không có xung đột, các tín hiệu này sẽ cho phép dữ liệu được sử dụng trực tiếp từ các thanh ghi nguồn.

Bảng 3.4: Các giá trị điều khiển cho các bộ ghép kênh chuyển tiếp

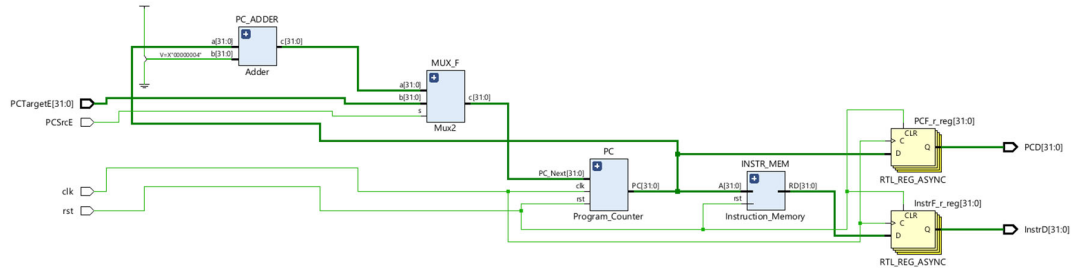
Điều khiển MUX	Nguồn	Giải thích
ForwardA = 00	ID/EX	Toán hạng ALU đầu tiên xuất phát từ Tập thanh ghi.
ForwardA = 10	EX/MEM	Toán hạng ALU đầu tiên xuất phát từ kết quả ALU trước đó.
ForwardA = 01	MEM/WB	Toán hạng ALU đầu tiên xuất phát từ bộ nhớ dữ liệu.
ForwardB = 00	ID/EX	Toán hạng ALU thứ hai xuất phát từ Tập thanh ghi.
ForwardB = 10	EX/MEM	Toán hạng ALU thứ hai xuất phát từ kết quả ALU trước đó.
ForwardB = 01	MEM/WB	Toán hạng ALU thứ hai xuất phát từ bộ nhớ dữ liệu.

CHƯƠNG 4 : KẾT QUẢ

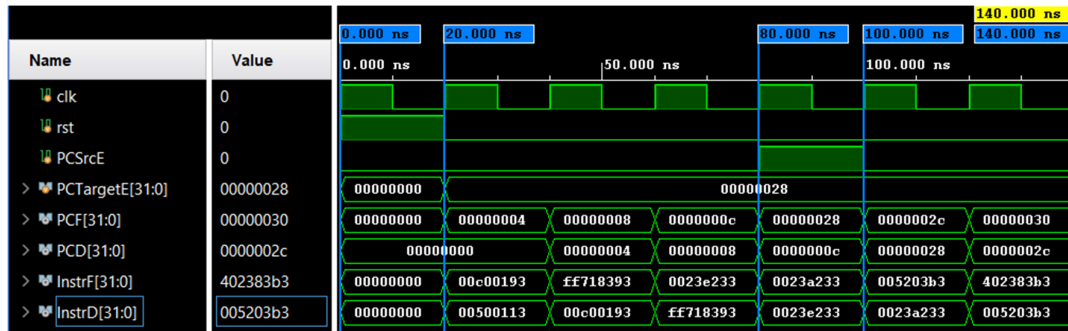
4.1. KẾT QUẢ MÔ PHỎNG DẠNG SÓNG CÁC KHỐI

4.1.1. Khối lấy lệnh

Hình 4.1 mô tả Khối lấy lệnh bao gồm các thành phần chính: bộ đếm chương trình, bộ nhớ lệnh, bộ cộng và bộ chọn. Khi hệ thống khởi động, tín hiệu rst đặt lại PC về giá trị ban đầu. Tín hiệu clk đồng bộ hóa các hoạt động. Bộ đếm chương trình xác định địa chỉ lệnh hiện tại và cập nhật địa chỉ tiếp theo thông qua bộ cộng và bộ chọn, trong khi bộ nhớ lệnh cung cấp lệnh tương ứng.



Hình 4.1: Sơ đồ RTL của Khối lấy lệnh



Hình 4.2: Dạng sóng của Khối lấy lệnh

Trong hình 4.2 mô tả chức năng của khối lấy lệnh được chia làm 4 trường hợp:

➤ **Trường hợp 1: Từ 0 – 20ns kiểm tra chức năng đặt lại hệ thống.**

Tín hiệu rst ở mức 1, do đó các ngõ ra của khối được đặt về mức 0.

➤ **Trường hợp 2: Từ 20 – 80ns kiểm tra chức năng tăng địa chỉ PC.**

Tín hiệu rst ở mức 0, tín hiệu PCSrcE ở mức 0, cho phép giá trị PC tăng tuần tự lần lượt là PCF = 32'h00000004, 32'h00000008, 32'h0000000c, tương ứng với các giá trị PC đó là lệnh InstrF = 32'h00c00193, 32'hff718393,

32'h0023e233. Vì tín hiệu PCD và InstrD là ngõ ra của thanh ghi với ngõ vào là PCF và InstrF nên các giá trị của ngõ ra sẽ chậm hơn so với ngõ vào 1 chu kỳ xung đồng hồ.

➤ **Trường hợp 3: Từ 80 – 100ns kiểm tra chức năng nhảy địa chỉ.**

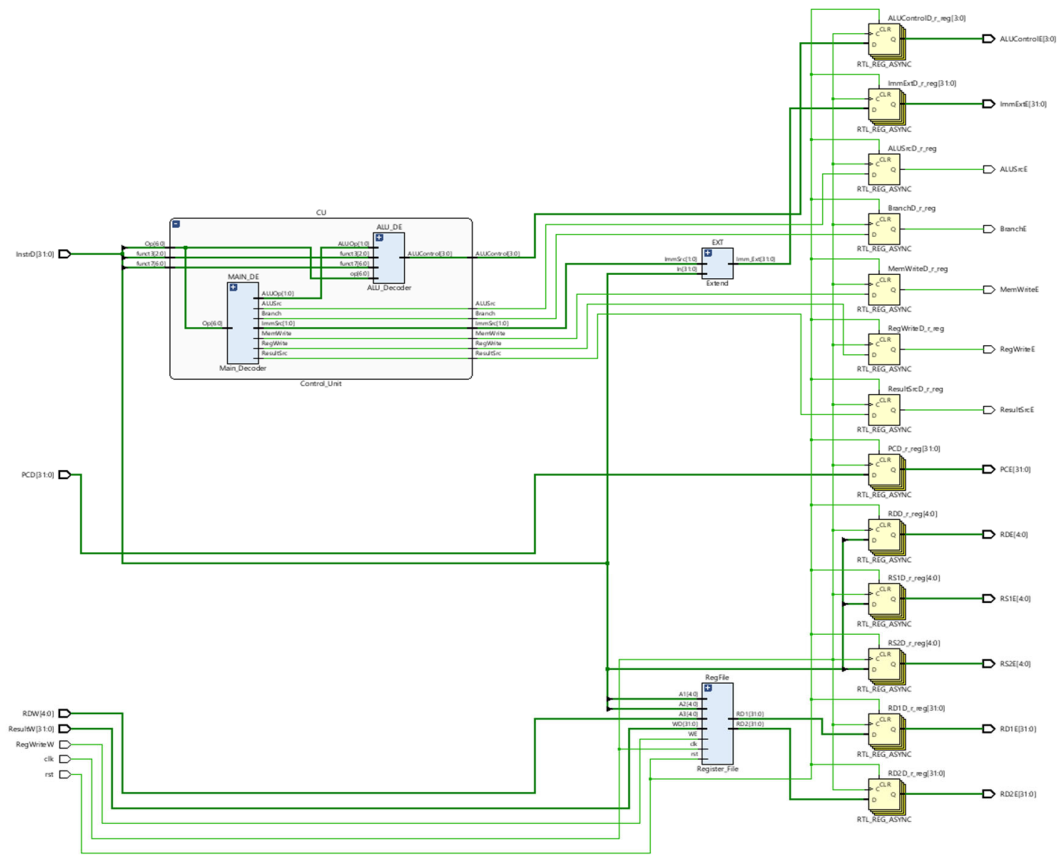
Tín hiệu rst ở mức 0, tín hiệu PCSrcE ở mức 1, cho phép giá trị PC nhận giá trị đích. Giá trị đích PCTargetE = 32'h00000028 nên giá trị PC = 32'h00000028, tương ứng giá trị PC là lệnh InstrF = 32'h23a233. Vì tín hiệu PCD và InstrD là ngõ ra của thanh ghi với ngõ vào là PCF và InstrF nên các giá trị của ngõ ra sẽ chậm hơn so với ngõ vào 1 chu kỳ xung đồng hồ.

➤ **Trường hợp 4: Từ 100 – 140ns kiểm tra chức năng tăng địa chỉ PC sau khi kết thúc lệnh nhảy.**

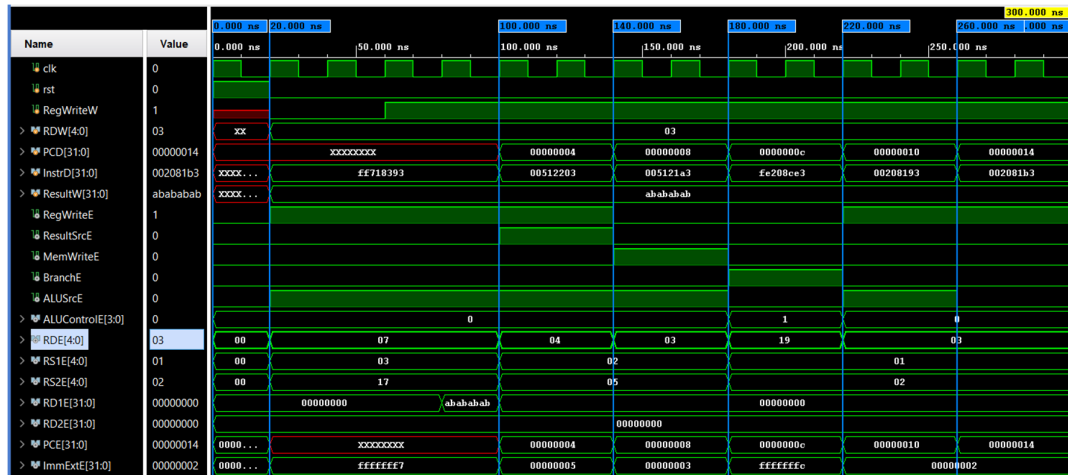
Tín hiệu rst ở mức 0, tín hiệu PCSrcE ở mức 0, cho phép giá trị PC tăng tuần tự lần lượt là PCF = 32'h0000002c, 32'h00000030. Lệnh tương ứng là InstrF = 32'h005203b3, 32'h402393b3. Vì tín hiệu PCD và InstrD là ngõ ra của thanh ghi với ngõ vào là PCF và InstrF nên các giá trị của ngõ ra sẽ chậm hơn so với ngõ vào 1 chu kỳ xung đồng hồ.

4.1.2. Khối giải mã lệnh

Hình 4.3 mô tả sơ đồ RTL khối giải mã lệnh gồm các thành phần chính: Đơn vị điều khiển chứa bộ giải mã chính và bộ giải mã ALU, tập hợp thanh ghi và các tín hiệu điều khiển. Khi một lệnh được tải vào Đơn vị điều khiển (CU), CU sử dụng bộ giải mã chính để phân tích lệnh và tạo ra các tín hiệu điều khiển. Các tín hiệu này được gửi đến bộ giải mã ALU, tập hợp thanh ghi, và các thanh ghi điều khiển. Bộ giải mã ALU xác định các phép toán cần thực hiện. Tín hiệu điều khiển từ CU điều phối các hoạt động như thực hiện phép toán trong ALU, đọc/ghi dữ liệu từ/đến bộ nhớ, và cập nhật các thanh ghi. Tập hợp thanh ghi lưu trữ các giá trị tạm thời và cung cấp dữ liệu cho các đơn vị khác khi cần.



Hình 4.3: Sơ đồ RTL của Khối giải mã lệnh



Hình 4.4: Dạng sóng của Khối giải mã lệnh

Hình 4.4 mô tả chức năng của Khối giải mã lệnh được chia làm 8 trường hợp:

➤ **Trường hợp 1: Từ 0 – 20ns kiểm tra chức năng đặt lại hệ thống.**

Tín hiệu rst ở mức 1, do đó các ngõ ra của khối được đặt về mức 0.

➤ **Trường hợp 2: Từ 20 – 100ns kiểm tra chức năng ghi vào tệp thanh ghi.**

- Tín hiệu rst ở mức 0.
- Lệnh ngõ vào InstrD = 32'hff718393 (addi x7, x3, -9) tương ứng cộng giá trị -9 và giá trị tại thanh ghi x3 lưu vào thanh ghi x7.
- Giá trị và địa chỉ ghi
 - ResultW = 32'habababab là giá trị cần ghi vào thanh ghi
 - RDW = 5'h3 là địa chỉ thanh ghi đích x3.
- Từ 20 – 60ns, tín hiệu RegWriteW = 0 không cho phép ghi vào tệp thanh ghi nên ngõ ra RD1D = 0 không có giá trị nào được ghi vào thanh ghi x3.
- Từ 60 – 100ns, tín hiệu RegWriteW = 1 cho phép ghi vào tệp thanh ghi nên ngõ ra RD1D = 32'habababab, giá trị được ghi vào thanh ghi x3.

➤ **Trường hợp 3: Từ 100 – 140ns kiểm tra tín hiệu điều khiển lệnh loại tải.**

- Tín hiệu rst ở mức thấp.
- Lệnh ngõ vào InstrD = 32'h00512203 (lw x4 5(x2)) tương ứng lấy giá trị từ địa chỉ bộ nhớ tính bằng giá trị trong thanh ghi x2 cộng với giá trị tức thời 5 và nạp giá trị này vào thanh ghi x4.
- Các tín hiệu điều khiển:
 - RegWriteE = 1 cho phép ghi vào thanh ghi.
 - ResultSrcE = 1 lấy kết quả từ bộ nhớ.
 - MemWriteE = 0 không ghi vào bộ nhớ.
 - BranchE = 0 không nhảy.
 - ALUSrcE = 1 sử dụng giá trị tức thời.
 - ALUControlE = 4'b0000 tương ứng phép cộng.
 - ImmExtE = 32'h00000005 tương ứng mở rộng theo kiểu I
- Địa chỉ thanh ghi:
 - RS1E = 5'h2 tương ứng thanh ghi nguồn x2.
 - RDE = 5'h4 tương ứng thanh ghi đích x4
- Giá trị RD1E = 0 vì không có giá trị nào được ghi vào thanh ghi x2.

➤ **Trường hợp 4: Từ 140 – 180ns kiểm tra tín hiệu điều khiển lệnh loại S.**

- Tín hiệu rst ở mức thấp.
 - Lệnh ngõ vào InstrD = 32'h005121a3 (sw x5 3(x2)) lấy giá trị từ thanh ghi x5 và lưu vào địa chỉ bộ nhớ tính bằng giá trị trong thanh ghi x2 cộng với giá trị tức thời 3.
 - Các tín hiệu điều khiển:
 - RegWriteE = 0 không cho phép ghi vào thanh ghi.
 - ResultSrcE = 0 không lấy kết quả từ bộ nhớ.
 - MemWriteE = 1 cho phép ghi dữ liệu vào bộ nhớ.
 - BranchE = 0 không nhảy.
 - ALUSrcE = 1 sử dụng giá trị tức thời.
 - ALUControlE = 4'b0000 tương ứng phép cộng.
 - ImmExtE = 32'h00000003 tương ứng mở rộng theo kiểu S
 - Địa chỉ thanh ghi:
 - RS1E = 5'h2 tương ứng thanh ghi chứa địa chỉ cơ sở x2.
 - RS2E = 5'h5 tương ứng thanh ghi chứa dữ liệu cần lưu trữ x5
 - Giá trị RD1E và RD2E đều bằng 0 vì không có giá trị nào được ghi vào thanh ghi x2 và x5.
- **Trường hợp 5: Từ 180 – 220ns kiểm tra tín hiệu điều khiển lệnh loại B.**
- Tín hiệu rst ở mức thấp.
 - Lệnh ngõ vào InstrD= 32'hfe208ce3 (beq x1, x2, -8) tương ứng sẽ nhảy đến giá trị PC đích nếu hai thanh ghi x1 và x2 bằng nhau.
 - Các tín hiệu điều khiển:
 - RegWriteE = 0 không cho phép ghi vào thanh ghi.
 - ResultSrcE = 0 không lấy kết quả từ bộ nhớ.
 - MemWriteE = 0 không cho phép ghi dữ liệu vào bộ nhớ.
 - BranchE = 1 thực hiện nhảy nếu điều kiện so sánh đúng.
 - ALUSrcE = 0 sử dụng giá trị từ thanh ghi
 - ALUControlE = 4'b0001 thực hiện phép trừ để kiểm tra bằng (x1 - x2).
 - ImmExtE = 32'hfffffffc tương ứng -8 mở rộng theo kiểu B

- Địa chỉ thanh ghi:
 - RS1E=5'h1 tương ứng thanh ghi chứa giá trị cần so sánh x1.
 - RS2E=5'h2 tương ứng thanh ghi chứa giá trị cần so sánh x2
- Giá trị RD1E và RD2E đều bằng 0 vì không có giá trị nào được ghi vào thanh ghi x1 và x2.

➤ **Trường hợp 6: Từ 220 – 260ns kiểm tra tín hiệu điều khiển lệnh loại I.**

- Tín hiệu rst ở mức thấp.
- Lệnh ngõ vào InstrD = 32'h00208193 (addi x3, x1, 2) tương ứng cộng giá trị tại thanh ghi x1 với giá trị tức thời 2 lưu vào địa chỉ thanh ghi x3.
- Các tín hiệu điều khiển:
 - RegWriteE = 1 cho phép ghi vào thanh ghi.
 - ResultSrcE = 0 không lấy kết quả từ bộ nhớ.
 - MemWriteE = 0 không cho phép ghi dữ liệu vào bộ nhớ.
 - BranchE = 0 không nhảy.
 - ALUSrcE = 1 sử dụng giá trị tức thời.
 - ALUControlE = 4'b0000 thực hiện phép cộng.
 - ImmExtE = 32'h00000002 tương ứng mở rộng theo kiểu I
- Địa chỉ thanh ghi:
 - RS1E = 5'h1 tương ứng thanh ghi nguồn x1.
 - RDE = 5'h3 tương ứng thanh ghi đích x3.
- Giá trị RD1E = 0 vì không có giá trị nào được ghi vào thanh ghi x1

➤ **Trường hợp 7: Từ 260 – 300ns kiểm tra tín hiệu điều khiển lệnh loại R.**

- Tín hiệu rst ở mức thấp.
- Lệnh ngõ vào InstrD= 32'h002081b3 (addi x3, x1, x2) tương ứng cộng giá trị tại thanh ghi x1 và x2 sau đó lưu vào địa chỉ tại thanh ghi x3.
- Các tín hiệu điều khiển:
 - RegWriteE = 1 cho phép ghi vào thanh ghi.
 - ResultSrcE = 0 không lấy kết quả từ bộ nhớ.
 - MemWriteE = 0 không cho phép ghi dữ liệu vào bộ nhớ.

- BranchE = 0 không nhảy.
- ALUSrcE = 1 sử dụng giá trị tức thời.
- ALUControlE = 4'b0000 thực hiện phép cộng.
- ImmExtE không sử dụng.
- Địa chỉ thanh ghi:
 - RS1E = 5'h1 tương ứng thanh ghi nguồn x1.
 - RS2E = 5'h2 tương ứng thanh ghi nguồn x2.
 - RDE = 5'h3 tương ứng thanh ghi đích x3.
- Giá trị RD1E = RD2E = 0 vì không có giá trị nào được ghi vào thanh ghi x1 và x2.

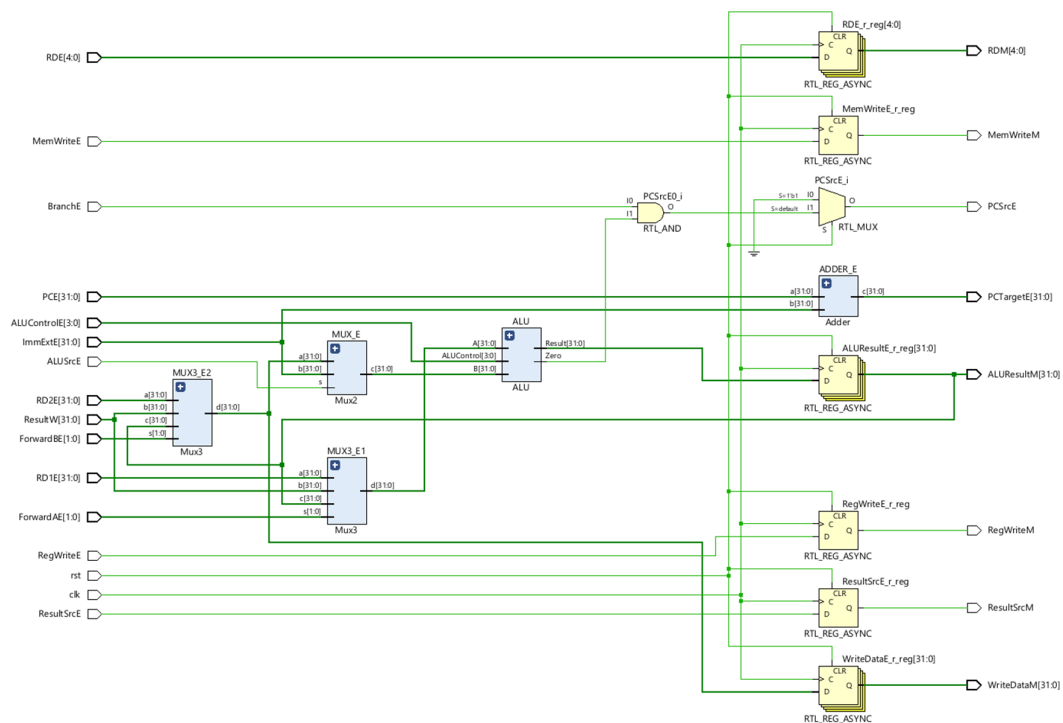
➤ **Trường hợp 8: Từ 100 – 300ns kiểm tra chức năng ghi thanh ghi ID/EX.**

Tín hiệu PCD là ngõ vào thanh ghi lần lượt được xuất ra thanh ghi bằng ngõ ra PCE tương ứng:

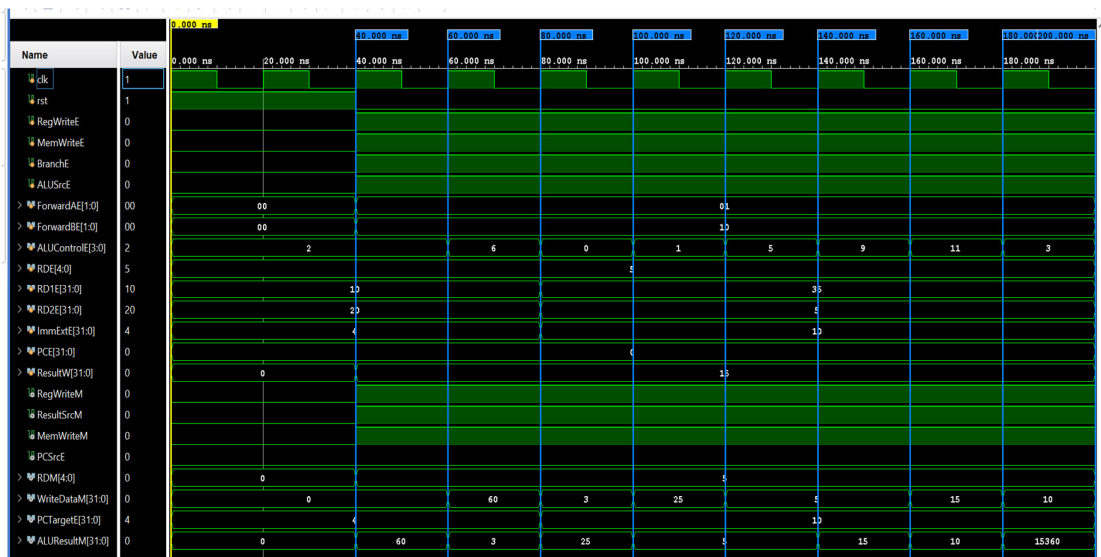
- Từ 100 – 140ns, PCD = 32'h00000004 = PCE.
- Từ 140 – 180ns, PCD = 32'h00000008 = PCE.
- Từ 180 – 220ns, PCD = 32'h0000000c = PCE.
- Từ 220 – 260ns, PCD = 32'h00000010 = PCE.
- Từ 260 – 300ns, PCD = 32'h00000014 = PCE.

4.1.3. Khối thực thi

Hình 4.5 cho thấy sơ đồ RTL trên mô tả khối thực thi bao gồm các thành phần chính như bộ luận lý số học, bộ chọn ngõ vào, bộ chọn 2 ngõ vào và bộ cộng. Khối thực thi có nhiệm vụ thực hiện các phép tính số học, luận lý và tính toán địa chỉ bộ nhớ, thực thi các lệnh và xử lý các ngoại lệ, quyết định đến hiệu suất chung của đường ống.



Hình 4.5: Sơ đồ RTL của Khối thực thi



Hình 4.6: Dạng sóng của Khối thực thi

Kết quả mô phỏng của khối thực thi kiểm tra các chức năng tính toán của bộ vi xử lý 32 bit được thể hiện qua Hình 4.6 như sau:

➤ **Trường hợp 1: Từ 0 – 40ns kiểm tra chức năng đặt lại.**

Tín hiệu rst ở mức 1, do đó các ngõ ra của khối được đặt về mức 0.

➤ **Trường hợp 2: Từ 40 – 200ns kiểm tra chức năng tính toán của ALU.**

- Tín hiệu rst ở mức 0, RegWriteE = 1, ResultSrcE = 1, MemWriteE = 1, BranchE = 1, ALUSrcE = 1, ForwardAE = 2'b01, ForwardBE = 2'b10, ResultW = 15, WriteDataM = 25.

- **Từ 40ns – 60ns: ALUControlE = 4'b0010 (Phép nhân)**

- Tính toán trong ALU:

- SrcAE = ResultW = 15 (ForwardAE = 2'b01)
 - SrcBE = ALUResultW = 25 (ForwardBE = 2'b10)
 - SrcBE_out = ImmExtE = 4 (ALUSrcE = 1)
 - ALU thực hiện phép nhân: SrcAE * SrcBE_out = 15 * 4 = 60
 - ALUResultE = 60
 - ZeroE = 0 (vì ALUResultE không phải là 0)

- Các giá trị đầu ra:

- RegWriteM = 1
 - ResultSrcM = 1
 - MemWriteM = 1
 - ALUResultM = 60
 - WriteDataM = SrcBE = 25
 - RDM = 5
 - PCSrcE = 0 (vì ZeroE = 0 và BranchE = 1)

- **Từ 60ns – 80ns: Thay đổi ALUControlE = 4'b0110 (Phép chia)**

- Tính toán trong ALU:

- SrcAE = ResultW = 15 (ForwardAE = 2'b01)
 - SrcBE = ALUResultW = 25 (ForwardBE = 2'b10)
 - SrcBE_out = ImmExtE = 4 (ALUSrcE = 1)
 - ALU thực hiện phép chia: SrcAE / SrcBE_out = 15 / 4 = 3
 - ALUResultE = 3
 - ZeroE = 0 (vì ALUResultE không phải là 0)

- Các giá trị đầu ra:

- RegWriteM = 1
 - ResultSrcM = 1
 - MemWriteM = 1
 - ALUResultM = 3
 - WriteDataM = SrcBE = 25
 - RDM = 5
 - PCSrcE = 0 (vì ZeroE = 0 và BranchE = 1)
- **Từ 80ns – 100ns: Thay đổi ALUControlE = 4'b0000 (Phép cộng) và các giá trị đầu vào RDE1 = 35, RDE2 = 5, ImmExtE = 10.**
 - Tính toán trong ALU:
 - SrcAE = ResultW = 15 (ForwardAE = 2'b01)
 - SrcBE = RD2E = 5 (ForwardBE = 2'b10)
 - SrcBE_out = ImmExtE = 10 (ALUSrcE = 1)
 - ALU thực hiện phép cộng: SrcAE + SrcBE_out = 15 + 10 = 25
 - ALUResultE = 25
 - ZeroE = 0 (vì ALUResultE không phải là 0)
 - Các giá trị đầu ra:
 - RegWriteM = 1
 - ResultSrcM = 1
 - MemWriteM = 1
 - ALUResultM = 25
 - WriteDataM = SrcBE = 5
 - RDM = 5
 - PCSrcE = 0 (vì ZeroE = 0 và BranchE = 1)
 - **Từ 100ns – 120ns: Thay đổi ALUControlE = 4'b0001 (Phép trừ) và các giá trị đầu vào RDE1 = 35, RDE2 = 5, ImmExtE = 10.**
 - Tính toán trong ALU:
 - SrcAE = ResultW = 15 (ForwardAE = 2'b01)
 - SrcBE = RD2E = 5 (ForwardBE = 2'b10)

- SrcBE_out = ImmExtE = 10 (ALUSrcE = 1)
- ALU thực hiện phép trừ: SrcAE - SrcBE_out = 15 - 10 = 5
- ALUResultE = 5
- ZeroE = 0 (vì ALUResultE không phải là 0)
- Các giá trị đầu ra:
 - RegWriteM = 1
 - ResultSrcM = 1
 - MemWriteM = 1
 - ALUResultM = 25
 - WriteDataM = SrcBE = 25
 - RDM = 5
 - PCSrcE = 0 (vì ZeroE = 0 và BranchE = 1)
- **Từ 120ns – 140ns: Thay đổi ALUControlE = 4'b0101 (Phép XOR) và các giá trị đầu vào RDE1 = 35, RDE2 = 5, ImmExtE = 10.**
 - Tính toán trong ALU:
 - SrcAE = ResultW = 15 (ForwardAE = 2'b01)
 - SrcBE = RD2E = 5 (ForwardBE = 2'b10)
 - SrcBE_out = ImmExtE = 10 (ALUSrcE = 1)
 - ALU thực hiện phép XOR: SrcAE XOR SrcBE_out = 15 XOR 10 = 5
 - ALUResultE = 5
 - ZeroE = 0 (vì ALUResultE không phải là 0)
 - Các giá trị đầu ra:
 - RegWriteM = 1
 - ResultSrcM = 1
 - MemWriteM = 1
 - ALUResultM = 5
 - WriteDataM = SrcBE = 5
 - RDM = 5

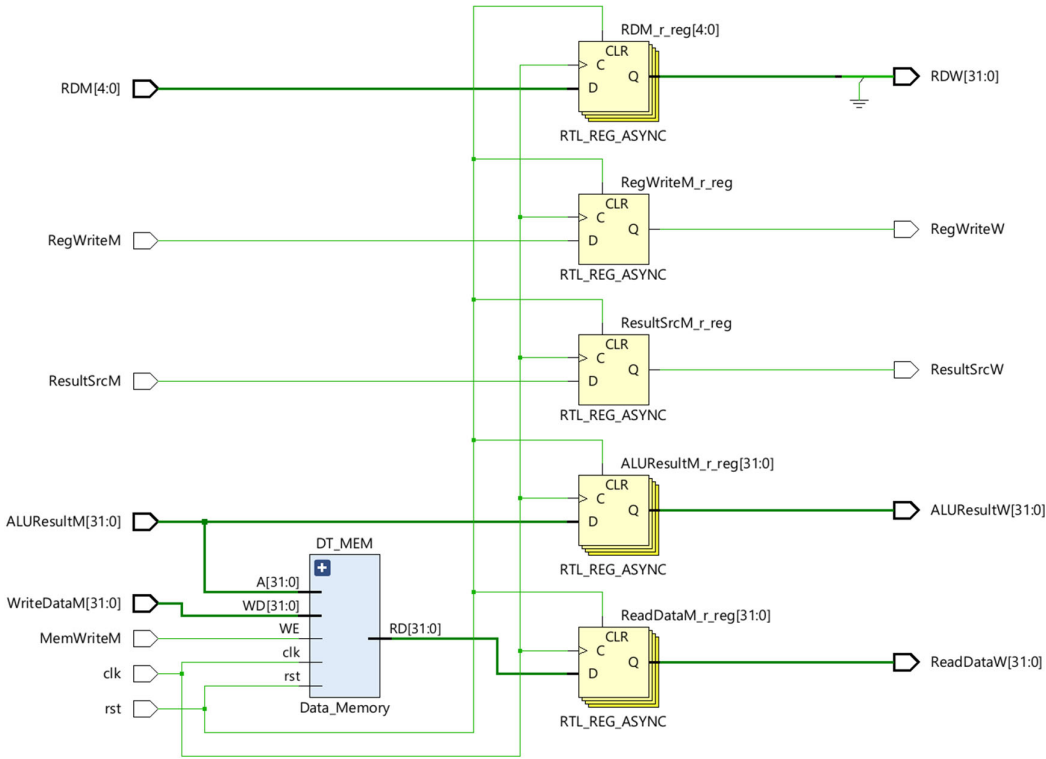
- PCSrcE = 0 (vì ZeroE = 0 và BranchE = 1)
- **Từ 140ns – 160ns: Thay đổi ALUControlE = 4'b1001 (Phép OR) và các giá trị đầu vào RDE1 = 35, RDE2 = 5, ImmExtE = 10.**
 - Tính toán trong ALU:
 - SrcAE = ResultW = 15 (ForwardAE = 2'b01)
 - SrcBE = RD2E = 5 (ForwardBE = 2'b10)
 - SrcBE_out = ImmExtE = 10 (ALUSrcE = 1)
 - ALU thực hiện phép OR: SrcAE OR SrcBE_out = 15 OR 10 = 15
 - ALUResultE = 15
 - ZeroE = 0 (vì ALUResultE không phải là 0)
 - Các giá trị đầu ra:
 - RegWriteM = 1
 - ResultSrcM = 1
 - MemWriteM = 1
 - ALUResultM = 15
 - WriteDataM = SrcBE = 5
 - RDM = 5
 - PCSrcE = 0 (vì ZeroE = 0 và BranchE = 1)
- **Từ 160ns – 180ns: Thay đổi ALUControlE = 4'b1011 (Phép AND) và các giá trị đầu vào RDE1 = 35, RDE2 = 5, ImmExtE = 10.**
 - Tính toán trong ALU:
 - SrcAE = ResultW = 15 (ForwardAE = 2'b01)
 - SrcBE = RD2E = 5 (ForwardBE = 2'b10)
 - SrcBE_out = ImmExtE = 10 (ALUSrcE = 1)
 - ALU thực hiện phép AND: SrcAE AND SrcBE_out = 15 AND 10 = 10
 - ALUResultE = 10
 - ZeroE = 0 (vì ALUResultE không phải là 0)
 - Các giá trị đầu ra:

- RegWriteM = 1
 - ResultSrcM = 1
 - MemWriteM = 1
 - ALUResultM = 10
 - WriteDataM = SrcBE = 15
 - RDM = 5
 - PCSrcE = 0 (vì ZeroE = 0 và BranchE = 1)
- **Từ 180ns – 200ns: Thay đổi ALUControlE = 4'b0011 (Phép dịch) và các giá trị đầu vào RDE1 = 35, RDE2 = 5, ImmExtE = 10.**
 - **Tính toán trong ALU:**
 - SrcAE = ResultW = 15 (ForwardAE = 2'b01)
 - SrcBE = RD2E = 5 (ForwardBE = 2'b10)
 - SrcBE_out = ImmExtE = 10 (ALUSrcE = 1)
 - ALU thực hiện phép dịch: SrcAE dịch SrcBE_out = 15 dịch 10 = 15360
 - ALUResultE = 15360
 - ZeroE = 0 (vì ALUResultE không phải là 0)
 - **Các giá trị đầu ra:**
 - RegWriteM = 1
 - ResultSrcM = 1
 - MemWriteM = 1
 - ALUResultM = 15360
 - WriteDataM = SrcBE = 10
 - RDM = 5
 - PCSrcE = 0 (vì ZeroE = 0 và BranchE = 1)

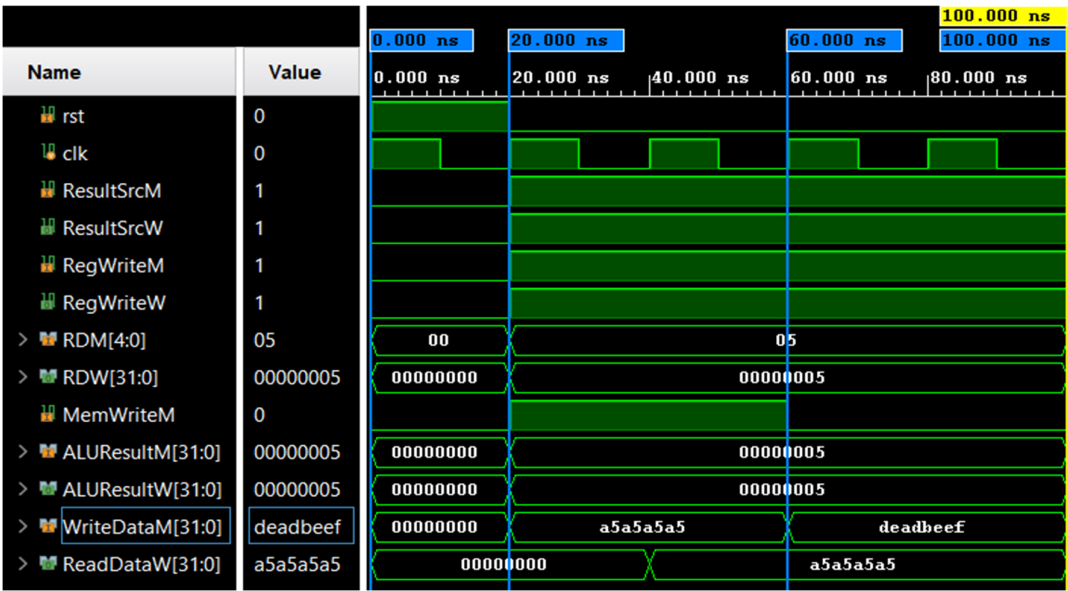
4.1.4. Khối truy cập bộ nhớ

Hình 4.7 cho thấy sơ đồ RTL trên mô tả khối truy cập bộ nhớ có bộ Data_Memory. Khối truy cập bộ nhớ thực hiện các hoạt động liên quan đến bộ nhớ. Cụ thể, khối này sẽ xử lý các lệnh tải và lưu trữ, truy xuất dữ liệu từ bộ nhớ

hoặc lưu dữ liệu vào bộ nhớ, đảm bảo tính nhất quán của dữ liệu trong quá trình tương tác với bộ nhớ, ảnh hưởng lớn đến hiệu suất chung của đường ống.



Hình 4.7: Sơ đồ RTL của Khối truy cập bộ nhớ



Hình 4.8: Dạng sóng của Khối truy cập bộ nhớ

Kết quả mô phỏng của Khối truy cập bộ nhớ truy xuất dữ liệu từ bộ nhớ hoặc lưu dữ liệu vào bộ nhớ được thể hiện qua Hình 4.8 như sau:

➤ **Trường hợp 1: Từ 0 – 20ns kiểm tra chức năng đặt lại.**

Tín hiệu rst ở mức 1, do đó các ngõ ra của khối được đặt về mức 0.

➤ **Trường hợp 2: Từ 20 – 60ns kiểm tra chức năng đọc ghi dữ liệu bộ nhớ.**

- Tín hiệu rst = 0.
- Các tín hiệu điều khiển và dữ liệu:
 - MemWriteM = 1 cho phép ghi dữ liệu vào bộ nhớ.
 - ALUResultM = 32'd5 là địa chỉ bộ nhớ để ghi.
 - WriteDataM = 32'hA5A5A5A5 là dữ liệu cần ghi vào bộ nhớ.
- Các giá trị:
 - ReadDataW = 32'ha5a5a5a5 cho thấy địa chỉ được lưu vào bộ nhớ và trích xuất ra chậm 1 xung đồng hồ vì bộ nhớ cần 1 chu kỳ xung để hoàn thành quá trình đọc dữ liệu xuất ra.

➤ **Trường hợp 3: Từ 60 – 100ns kiểm tra chức năng cho phép ghi dữ liệu vào bộ nhớ.**

- Tín hiệu rst = 0.
- Các tín hiệu điều khiển và dữ liệu:
 - MemWriteM = 0 không cho phép ghi dữ liệu vào bộ nhớ.
 - ALUResultM = 32'd5 là địa chỉ bộ nhớ để ghi.
 - WriteDataM = 32'hDEADBEEF là dữ liệu cần ghi vào bộ nhớ.
- Các giá trị:
 - ReadDataW = 32'ha5a5a5a5 cho thấy dữ liệu mới không được lưu vào bộ nhớ do tín hiệu MemWriteM = 0 không cho phép ghi dữ liệu vào bộ nhớ.

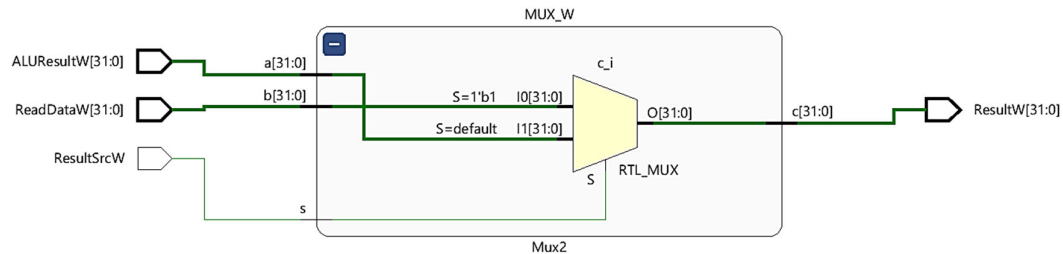
➤ **Trường hợp 4: Từ 20 – 80ns kiểm tra chức năng cho thanh ghi.**

- Các tín hiệu ResultSrcM, RegWriteM, RDM đều được xuất ra ngõ ra qua thanh ghi MEM/WB tương ứng các giá trị:
 - ResultSrcM = ResultSrcW = 1.

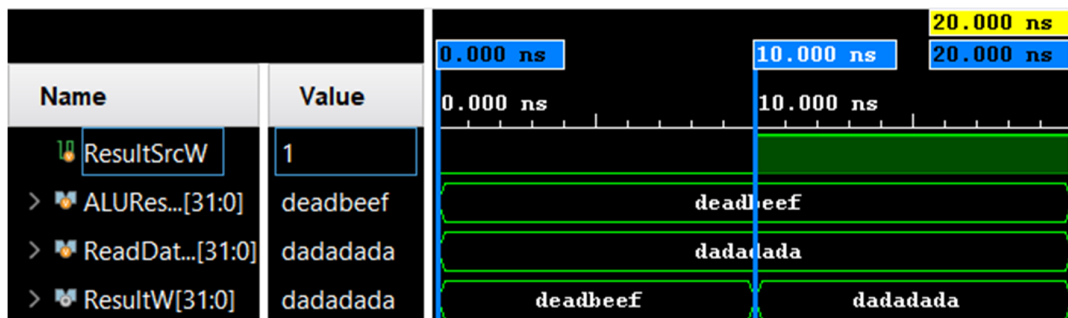
- $\text{RegWriteM} = \text{RegWriteW} = 1$.
- $\text{RDM} = \text{RDW} = 5'h05$.

4.1.5. Khối ghi lại

Khối ghi lại có trách nhiệm nhận kết quả từ các khối khác, ghi kết quả vào các thanh ghi đích, và cập nhật các thanh ghi trạng thái, hoàn tất chu kỳ thực thi lệnh.



Hình 4.9: Sơ đồ RTL của Khối ghi lại

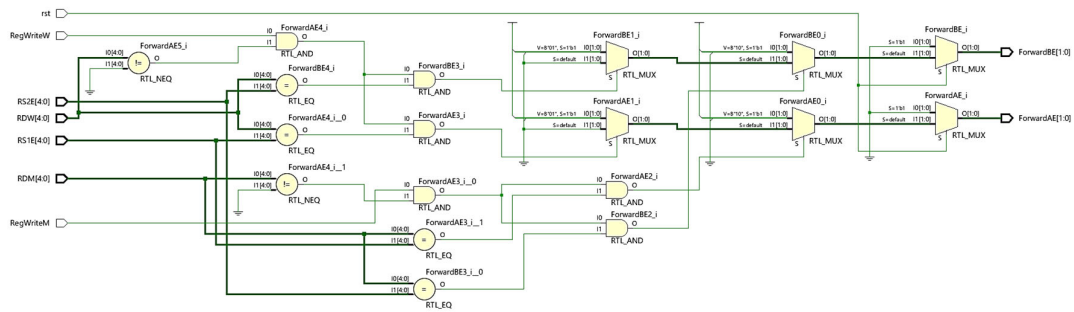


Hình 4.10: Dạng sóng của Khối ghi lại

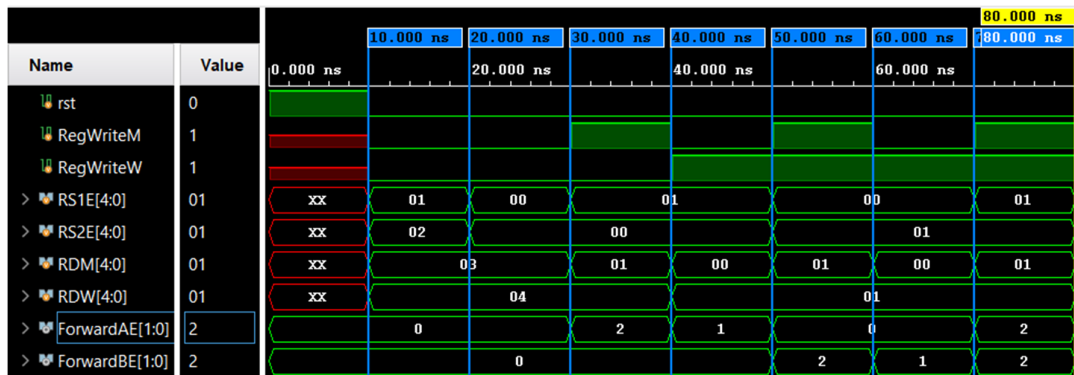
- **Từ 0 – 10ns:** Tín hiệu $\text{ResultSrcW} = 0$ chọn giá trị ALU làm đầu ra, $\text{ResultW} = \text{ALUResultW} = 32'h\text{deadbeef}$.
- **Từ 10 – 20ns:** Tín hiệu $\text{ResultSrcW} = 1$ chọn giá trị bộ nhớ làm đầu ra, $\text{ResultW} = \text{ReadDataW} = 32'h\text{dadadada}$.

4.1.6. Khối xử lý xung đột

Khối xử lý xung đột có nhiệm vụ phát hiện và xử lý các tình huống xung đột trong đường ống như dữ liệu, điều khiển và cấu trúc thực hiện các chiến lược xử lý như chuyển tiếp để duy trì tính nhất quán của dữ liệu và tối ưu hóa hiệu suất của đường ống.



Hình 4.11: Sơ đồ RTL của Khối xử lý xung đột



Hình 4.12: Dạng sóng của Khối xử lý xung đột

- **Trường hợp 1: Từ 0ns – 10ns kiểm tra tín hiệu reset của hệ thống**
 Khi reset (rst = 1) ở mức cao, tất cả các tín hiệu đều đặt về 2'b00, không có chuyển tiếp nào xảy ra
- **Trường hợp 2: Từ 10ns – 30ns không có tín hiệu nào được chuyển tiếp**
 Từ 10 – 20ns, tín hiệu đầu vào là: rst = 0, RegWriteM = RegWriteW = 0, không có ghi vào thanh ghi ở giai đoạn MEM hoặc WB, nên không có chuyển tiếp.
 Từ 20 – 30ns, tín hiệu đầu vào là: rst = 0, RDW = RDM = 0 không có chuyển tiếp vì cả hai thanh ghi đều là thanh ghi x0.
- **Trường hợp 3: Từ 30ns – 50ns Chuyển tiếp với toán tử thứ nhất.**
 Từ 30 – 40ns, tín hiệu đầu vào là: rst = 0, RegWriteM = 1, RDM = RS1E = 5'h1 và khác không nên ForwardAE = 2'b10, chuyển tiếp từ giai đoạn truy cập bộ nhớ.

Từ 40 – 50ns, tín hiệu đầu vào là: $\text{rst} = 0$, $\text{RegWriteW} = 1$, $\text{RDW} = \text{RS1E} = 5'h1$ và khác không nên $\text{ForwardAE} = 2'b01$, chuyển tiếp từ giai đoạn ghi lại.

➤ **Trường hợp 4: Từ 50ns – 70ns Chuyển tiếp với toán tử thứ hai.**

Từ 50 – 60ns, tín hiệu đầu vào là: $\text{rst} = 0$, $\text{RegWriteM} = 1$, $\text{RDM} = \text{RS2E} = 5'h1$ và khác không nên $\text{ForwardBE} = 2'b10$, chuyển tiếp từ giai đoạn truy cập bộ nhớ.

Từ 60 – 70ns, tín hiệu đầu vào là: $\text{rst} = 0$, $\text{RegWriteW} = 1$, $\text{RDW} = \text{RS2E} = 5'h1$ và khác không nên $\text{ForwardBE} = 2'b01$, chuyển tiếp từ giai đoạn ghi lại.

➤ **Trường hợp 5: Từ 70ns – 80ns có tín hiệu được chuyển tiếp từ cả 2 giai đoạn (Giai đoạn truy cập bộ nhớ có mức độ ưu tiên hơn).**

Tín hiệu đầu vào là: $\text{rst} = 0$, $\text{RegWriteM} = \text{RegWriteW} = 1$, $\text{RS1E} = \text{RS2E} = 5'h1$, $\text{RDM} = \text{RDW} = 5'h1$. Khi cả RegWriteM và RegWriteW đều bằng 1 và cả RDM và RDW đều bằng RS1E và RS2E , tín hiệu ForwardAE và ForwardBE được đặt là $2'b10$ để chuyển tiếp từ giai đoạn truy cập bộ nhớ vì giai đoạn truy cập bộ nhớ có độ ưu tiên cao hơn.

4.2. KẾT QUẢ MÔ PHỎNG TOÀN HỆ THỐNG

Sau khi hoàn thành việc thiết kế các khối thành phần, nhóm đã tiến hành kết nối đường dây để được ra sơ đồ RTL của bộ vi xử lý 32 bit hoàn chỉnh như Hình 4.13 mô tả các khối chức năng chính và cách chúng tương tác trong quá trình thực thi lệnh. Đầu tiên, khối lấy lệnh sẽ lấy lệnh từ bộ nhớ lệnh. Lệnh này sau đó được gửi đến khối giải mã để giải mã và tạo ra các tín hiệu điều khiển cần thiết. Các tín hiệu này được truyền đến khối thực thi, nơi các phép toán số học hoặc logic được thực hiện dựa trên lệnh và các tín hiệu điều khiển. Nếu lệnh yêu cầu truy cập bộ nhớ, khối truy cập bộ nhớ sẽ thực hiện các hoạt động đọc hoặc ghi dữ liệu từ bộ nhớ dữ liệu. Cuối cùng, kết quả của lệnh được ghi vào các thanh ghi thông qua khối ghi kết quả. Sơ đồ này cho thấy một cách rõ ràng các kết nối giữa các khối, minh họa cho quá trình sử dụng kỹ thuật đường ống, nơi nhiều lệnh được thực hiện

[illegible]

Summary

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	0.22 W
Design Power Budget:	Not Specified
Process:	typical
Power Budget Margin:	N/A
Junction Temperature:	26.1°C
Thermal Margin:	58.9°C (12.1 W)
Ambient Temperature:	25.0 °C
Effective θ_{JA} :	4.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power

Dynamic:	0.149 W (68%)
Clocks:	0.024 W (16%)
Signals:	0.062 W (41%)
Logic:	0.048 W (32%)
DSP:	0.001 W (1%)
I/O:	0.014 W (10%)
Device Static:	0.071 W (32%)

Hình 4.14 cung cấp một bản tóm tắt về ước tính tiêu thụ điện năng của bộ vi xử lý 32 bit. Tổng công suất tiêu thụ trên chip là 0.22 W, nhiệt độ điểm nổi là 26.1°C

và biên nhiệt là 58.9°C (12.1 W). Nhiệt độ môi trường là 25.0°C với hiệu quả tản nhiệt là 4.9°C/W. Công suất tiêu thụ trên chip được chia thành hai phần chính: công suất động và công suất tĩnh. Công suất động là 0.149 W, chiếm 68% tổng công suất. Trong đó, các đồng hồ (Clocks) tiêu thụ 0.024 W (16%), các tín hiệu (Signals) tiêu thụ 0.062 W (41%), logic tiêu thụ 0.048 W (32%), DSP tiêu thụ 0.001 W (1%), và I/O tiêu thụ 0.014 W (10%). Công suất tĩnh của thiết bị là 0.071 W, chiếm 32% tổng công suất.

Bảng 4.1: Mã Assembly dùng mô phỏng chương trình

Nhãn	Mã Assembly	Mã HEX	Nhãn	Mã Assembly	Mã HEX
label:	addi x1, x0, 5	0x00500093	end:	lw x14, 4(x3)	0x0041A703
	addi x2, x0, 5	0x00500113		sw x4, 8(x3)	0x0041A423
	addi x3, x0, 15	0x00F00193		beq x1, x2, tag	0x00208663
	addi x4, x0, 60	0x03C00213		addi x5, x0, -15	0xFF100293
	add x7, x1, x2	0x002083B3		addi x6, x0, 255	0xFF00313
	sub x8, x1, x2	0x40208433	tag:	add x5, x3, x4	0x004182B3
	mul x9, x1, x2	0x022084B3		sub x7, x5, x3	0x403283B3
	div x10, x1, x2	0x0220C533		j end	0xFF1FF06F
	rem x11, x1, x2	0x0220E5B3			
	or x12, x1, x2	0x0020E633			
	sll x13, x1, x2	0x002096B3			

Kết quả mô phỏng dạng sóng ngõ ra của bộ vi xử lý 32 bit sẽ giúp chúng ta hiểu rõ hơn về cách các tín hiệu và dữ liệu di chuyển qua các giai đoạn khác nhau trong quá trình thực thi lệnh. Bằng cách kiểm tra này chúng ta có thể kiểm tra để đảm bảo rằng các lệnh được thực thi chính xác và hiệu quả, cũng như phát hiện và sửa các lỗi có trong việc thiết kế. Kết quả mô phỏng được trình bày trong các Hình 4.15, Hình 4.16 và Hình 4.17 dựa vào Bảng 4.1, Bảng 4.2

➤ **Trường hợp 1: kiểm tra chức năng đặt lại.**

Từ 0 – 20ns, tín hiệu rst ở mức 1 nên tất cả ngõ ra được đặt về mức 0.

➤ **Trường hợp 2: kiểm tra các lệnh loại I khởi tạo giá trị.**

Từ 20 – 120ns thực hiện lệnh loại I tính toán gồm 5 chu kỳ xung:

- Chu kỳ xung thứ nhất, Khối lấy lệnh lấy lệnh InstrD = 32'h00500093 là lệnh cộng, tương ứng địa chỉ PCD = 32'h00000000.
- Chu kỳ xung thứ hai, Khối giải mã giải mã lệnh ra thành các tín hiệu điều khiển: BranchE = 0, ResultSrcE = 0, MemWriteE = 0, RegWriteE = 1, ALUSrcE = 1, ALUControlE = 4'h0; các địa chỉ thanh ghi RS1E = 5'0, RDE = 5'h1; giá trị RD1E = 32'd0, ImmExtE = 32d5, PCE = 32'h00000000.
- Chu kỳ xung thứ ba, Khối thực thi thực hiện tính toán phép cộng. Vì ALUSrcE = 1 nên chọn ngõ vào thứ hai ALU lấy từ ImmExtE = 32'd5. Lệnh không bị xung đột dữ liệu (Forward AE = 0) nên ngõ vào thứ nhất ALU lấy từ thanh ghi RD1E = 32'd0. Ngõ ra ALUResultM = RD1E + ImmExtE = 32'd5.
- Chu kỳ xung thứ tư, Khối truy cập bộ nhớ chuyển kết quả ALUResultM tới Khối ghi lại thông qua thanh ghi MEM/WB. Lệnh tính toán loại I không cần ghi vào bộ nhớ.
- Chu kỳ xung thứ năm, Khối ghi lại ghi giá trị vào Tập thanh ghi. Vì ResultSrcW = 0 nên ngõ ra ResultW chọn ngõ vào là ALUResultW = 32'd5 ghi vào thanh ghi x1.
- Khối xử lý xung đột không phát hiện xung đột nên ForwardAE = ForwardBE = 0.
- Tương tự với các lệnh có địa chỉ PCD = 0x00000004, 0x00000008, 0x0000000c thực hiện lần lượt phép cộng thanh ghi x2 với 32'd5, thanh ghi x3 với 32'd15, thanh ghi x4 với 32'd60 trong khoảng thời gian từ 40 – 140ns, 60 – 160ns, 80 – 180ns.

➤ **Trường hợp 3: kiểm tra các lệnh loại R tính toán phép tính số học và luận lý**

Từ 100 – 200ns thực hiện lệnh cộng loại R tính toán gồm 5 chu kỳ xung:

- Chu kỳ xung thứ nhất, Khối lấy lệnh lấy lệnh InstrD = 32'h002083b3 là lệnh cộng, tương ứng địa chỉ PCD = 32'h00000010.

- Chu kỳ xung thứ hai, Khối giải mã giải mã lệnh ra thành các tín hiệu điều khiển: BranchE = 0, ResultSrcE = 0, MemWriteE = 0, RegWriteE = 1, ALUSrcE = 0, ALUControlE = 4'h0; các địa chỉ thanh ghi RS1E = 5'1, RS2E = 5'h2, RDE = 5'h7; giá trị RD1E = 32'd5, RD2E = 32'd0, PCE = 32'h0000010.
- Chu kỳ xung thứ ba, Khối thực thi thực hiện tính toán phép cộng. Lệnh không bị xung đột dữ liệu (Forward AE = 0, ForwardBE = 0) và ALUSrcE = 0 nên cả hai ngõ vào ALU lấy từ thanh ghi RD1E = 32'd5 và RD2E = 32'd0. Ngõ ra ALUResultM = RD1E + RD2E = 32'd5.
- Chu kỳ xung thứ tư, Khối truy cập bộ nhớ chuyển kết quả ALUResultM tới Khối ghi lại thông qua thanh ghi MEM/WB. Lệnh tính toán loại R không cần ghi vào bộ nhớ.
- Chu kỳ xung thứ năm, Khối ghi lại ghi giá trị vào Tập thanh ghi. Vì ResultSrcW = 0 nên ngõ ra ResultW chọn ngõ vào là ALUResultW = 32'd5 ghi vào thanh ghi x7.
- Khối xử lý xung đột không phát hiện xung đột nên ForwardAE = ForwardBE = 0.
- Phép tính trừ, nhân, chia lấy nguyên, chia lấy dư, or, dịch được thực hiện tương tự trong khoảng thời gian lần lượt là:
 - Từ 120 – 220ns thực hiện phép trừ $x8 = x1 - x2 = 32'd5 - 32'd5 = 32'd0$.
 - Từ 140 – 240ns thực hiện phép nhân $x9 = x1 * x2 = 32'd5 * 32'd5 = 32'd25$.
 - Từ 160 – 260ns thực hiện phép chia lấy phần nguyên $x10 = x1 / x2 = 32'd5 / 32'd5 = 32'd1$.
 - Từ 180 – 280ns thực hiện phép chia lấy phần dư $x11 = x1 \% x2 = 32'd5 \% 32'd5 = 32'd0$.
 - Từ 200 – 300ns thực hiện phép or $x12 = x1 | x2 = 32'd5 | 32'd5 = 32'd5$.

- Từ 220 – 320ns thực hiện phép dịch trái $x13 = x1 \ll x2 = 32'd5$
 $\ll 32'd5 = 32'd160$.

➤ **Trường hợp 4: kiểm tra các lệnh tải.**

Từ 240 – 340ns thực hiện lệnh tải tính toán gồm 5 chu kỳ xung:

- Chu kỳ xung thứ nhất, Khối lấy lệnh lấy lệnh InstrD = 32'h0041a703 là lệnh tải, tương ứng địa chỉ PCD = 32'h0000002c.
- Chu kỳ xung thứ hai, Khối giải mã giải mã lệnh ra thành các tín hiệu điều khiển: BranchE = 0, ResultSrcE = 1, MemWriteE = 0, RegWriteE = 1, ALUSrcE = 1, ALUControlE = 4'h0; các địa chỉ thanh ghi RS1E = 5'h3, giá trị RD1E = 32'd15, ImmExt = 32'd4.
- Chu kỳ xung thứ ba, Khối thực thi thực hiện tính toán địa chỉ bộ nhớ cần truy cập. Lệnh không bị xung đột dữ liệu (Forward AE = 0, ForwardBE = 0) và ALUSrcE = 1 nên lấy ngõ vào ALU thứ nhất từ thanh ghi RD1E = 32'd15 và ImmExtE = 32'd4. Ngõ ra ALUResultM = RD1E + ImmExtE = 32'd19.
- Chu kỳ xung thứ tư, Khối truy cập bộ nhớ chuyển kết quả ALUResultM tới Khối ghi lại thông qua thanh ghi MEM/WB. Lệnh tính toán tải không cần ghi vào bộ nhớ.
- Chu kỳ xung thứ năm, Khối ghi lại ghi giá trị vào Tệp thanh ghi. Vì ResultSrcW = 1 nên ngõ ra ResultW chọn ngõ vào là ReadDataW = 32'd0 tương ứng địa chỉ 32'd19 và ghi vào thanh ghi x14.
- Khối xử lý xung đột không phát hiện xung đột nên ForwardAE = ForwardBE = 0.

➤ **Trường hợp 5: kiểm tra các lệnh lưu trữ.**

Từ 260 – 360ns thực hiện lệnh tải tính toán gồm 5 chu kỳ xung:

- Chu kỳ xung thứ nhất, Khối lấy lệnh lấy lệnh InstrD = 32'h041a423 là lệnh lưu trữ, tương ứng địa chỉ PCD = 32'h00000030.
- Chu kỳ xung thứ hai, Khối giải mã giải mã lệnh ra thành các tín hiệu điều khiển: BranchE = 0, ResultSrcE = 0, MemWriteE = 1, RegWriteE = 0,

$ALUSrcE = 1$, $ALUControlE = 4'h0$; các địa chỉ thanh ghi $RS1E = 5'h3$, $RS2E = 5'h4$, giá trị $RDE = 32'd8$, $RD1E = 32'd15$, $RD2E = 32'd60$, $ImmExtE = 32'd8$.

- Chu kỳ xung thứ ba, Khối thực thi thực hiện tính toán địa chỉ bộ nhớ cần truy cập. Lệnh không bị xung đột dữ liệu ($ForwardAE = 0$, $ForwardBE = 0$) và $ALUSrcE = 1$ nên lấy ngõ vào ALU thứ nhất từ thanh ghi $RD1E = 32'd15$ và $ImmExt = 32'd4$. Ngõ ra $ALUResultM = RD1E + ImmExtE = 32'd23$.
- Chu kỳ xung thứ tư, Khối truy cập bộ nhớ truy cập vào địa chỉ $32'd23$ tại bộ nhớ và dữ liệu từ thanh ghi x4 là $RD2E = 32'd60$ được lưu vào bộ nhớ tại địa chỉ $32'd23$
- Chu kỳ xung thứ năm, Khối ghi lại không dùng vì không có hoạt động ghi vào thanh ghi trong giai đoạn này.
- Khối xử lý xung đột không phát hiện xung đột nên $ForwardAE = ForwardBE = 0$.

➤ **Trường hợp 6: kiểm tra các lệnh loại B.**

Từ 280 – 380ns thực hiện lệnh tải tính toán gồm 5 chu kỳ xung:

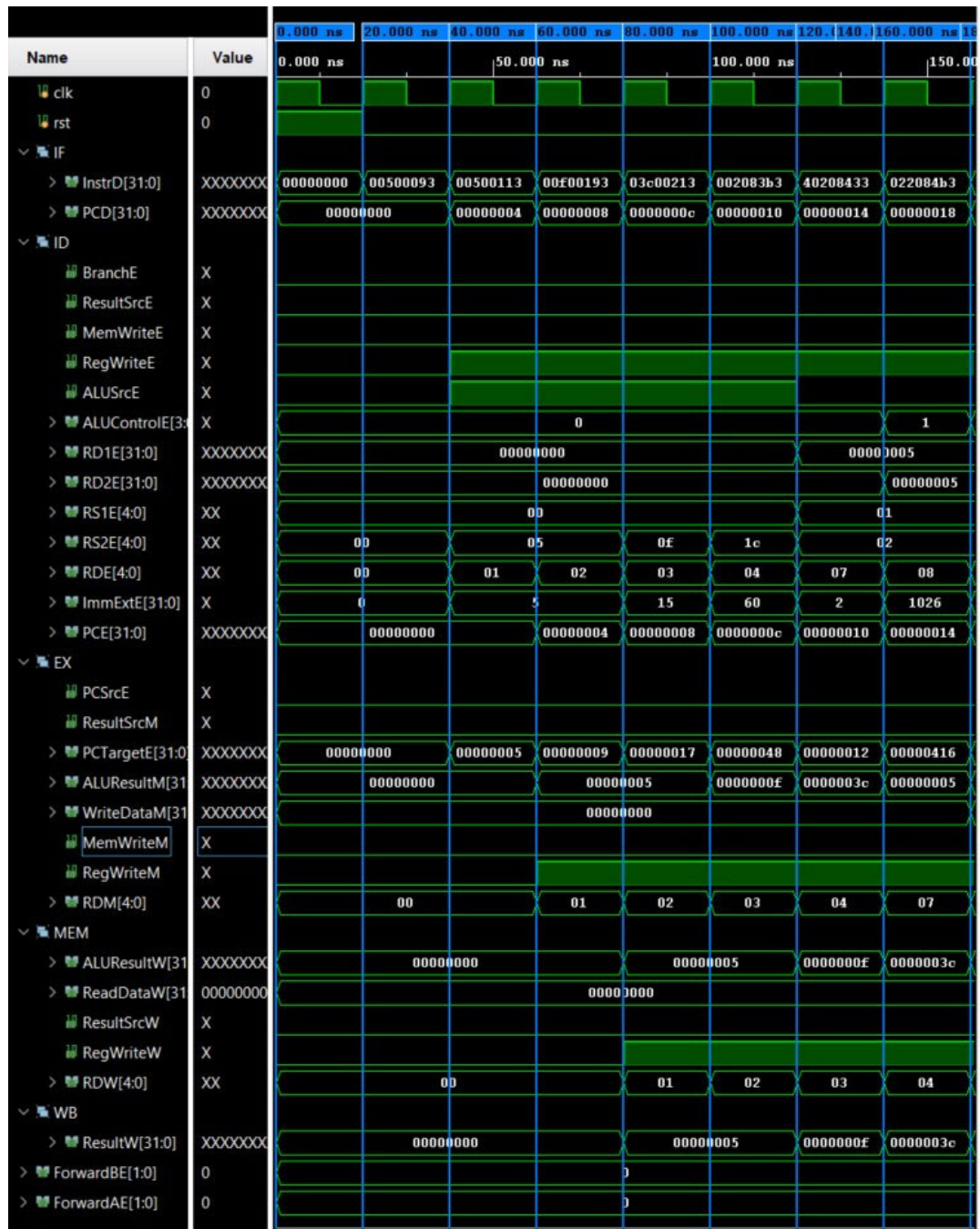
- Chu kỳ xung thứ nhất, Khối lấy lệnh lấy lệnh $InstrD = 32'h00208663$ là lệnh nhảy, tương ứng địa chỉ $PCD = 32'h00000034$.
- Chu kỳ xung thứ hai, Khối giải mã giải mã lệnh ra thành các tín hiệu điều khiển: $BranchE = 1$, $ResultSrcE = 0$, $MemWriteE = 0$, $RegWriteE = 0$, $ALUSrcE = 0$, $ALUControlE = 4'h1$; các địa chỉ thanh ghi $RS1E = 5'h1$, $RS2E = 5'h2$, giá trị $RDE = 32'd12$, $RD1E = 32'd5$, $RD2E = 32'd5$, $ImmExtE = 32'd12$.
- Chu kỳ xung thứ ba, Khối thực thi thực hiện so sánh giá trị của x1 và x2 bằng cách lấy $x1 - x2 = 32'd5 - 32'd5 = 32'd0$. $x1 - x2 = 0$ tương ứng $x1 = x2$ nên tính toán địa chỉ đích của nhánh $PctargetE = PCE + ImmExtE = 32'h00000034 + 32'h00000012 (32'd12) = 32'h00000040$.
- Chu kỳ xung thứ tư, vì là lệnh nhảy nên không thực hiện truy cập bộ nhớ

- Chu kỳ xung thứ năm, vì là lệnh nhảy nên không thực hiện ghi vào thanh ghi.
 - Khối xử lý xung đột không phát hiện xung đột nên ForwardAE = ForwardBE = 0.
 - Sau 1 chu kỳ, giá trị PCD = PCTargetE = 32'h00000040.
- ⇒ Khoảng thời gian tiếp theo hệ thống tiếp tục lấy lệnh như mã assemble PCD = 32'h000000044, PCD = 32'h00000048.
- ⇒ Bảng 4.2 tóm tắt lại kết quả của hệ thống với tín hiệu ResultW là giá trị được lưu vào tệp thanh ghi và tín hiệu RDW là địa chỉ lưu giá trị ResultW. Đối với lệnh lưu trữ, tín hiệu ALUResultW đại diện cho địa chỉ và WriteDataM đại diện cho giá trị ghi vào bộ nhớ.

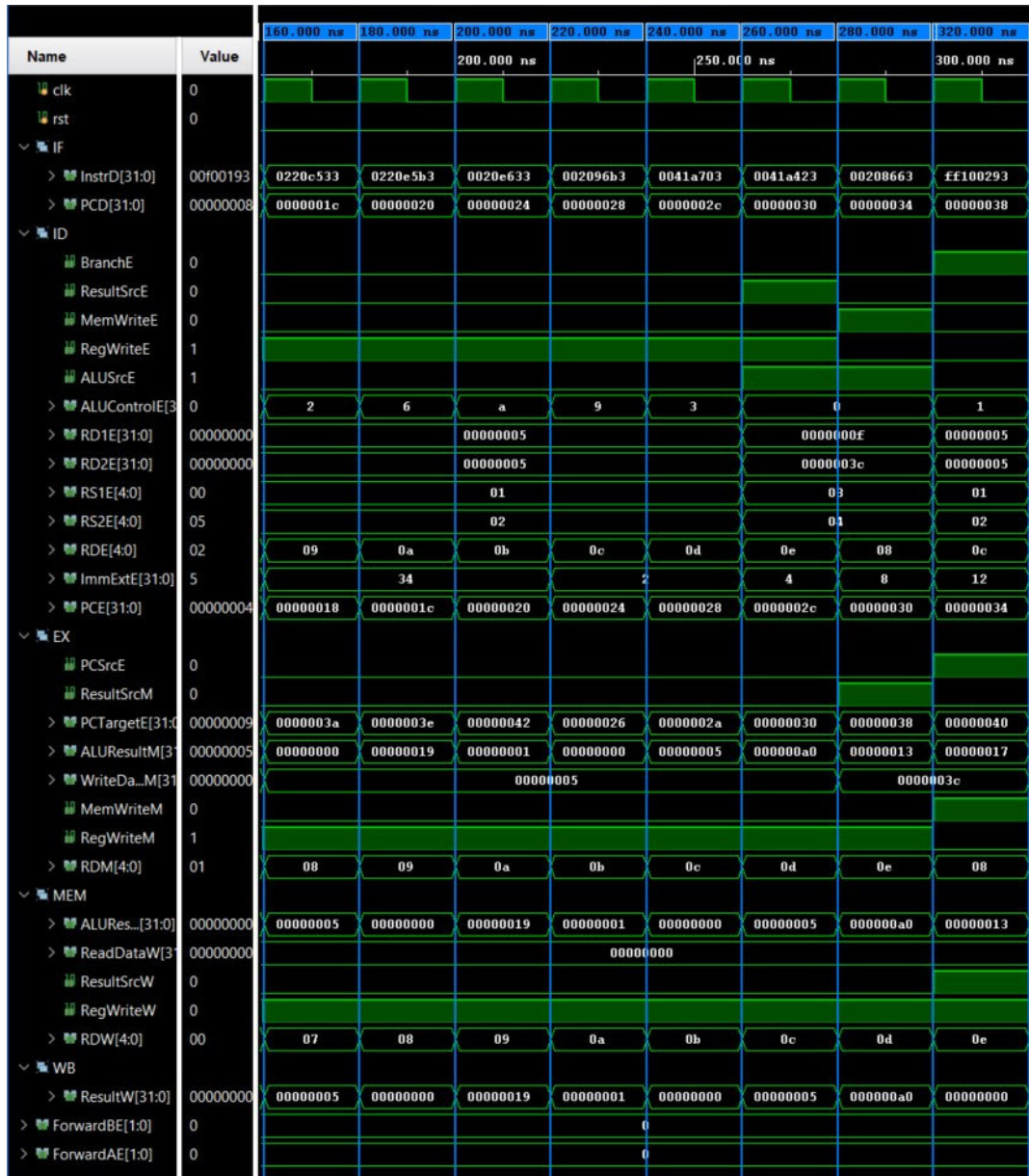
Bảng 4.2: Tóm tắt kết quả

Trường hợp	Thời gian	Hoạt động
Đặt lại hệ thống	0 - 20ns	Tất cả các ngõ ra được đặt về mức 0
Lệnh loại I khởi tạo giá trị	20 - 120ns	Lệnh addi x1, x0, 5. Phép cộng : ALUResultM = RD1E + ImmExtE = 32'd5. ResultW = 32'd5; RDW = 5'd1.
	40 - 140ns	Lệnh addi x2, x0, 5. Phép cộng : ALUResultM = RD1E + ImmExtE = 32'd5. ResultW = 32'd5; RDW = 5'd2.
	60 - 160	Lệnh addi x3, x0, 15. Phép cộng : ALUResultM = RD1E + ImmExtE = 32'd15. ResultW = 32'd15; RDW = 5'd3.
	80 - 180ns	Lệnh addi x4, x0, 60. Phép cộng : ALUResultM = RD1E + ImmExtE = 32'd60. ResultW = 32'd60; RDW = 5'd4.
Lệnh loại R	100-200ns	Lệnh add x7, x1, x2. Phép cộng : $x7 = x1 + x2 = 32'd5 + 32'd0 = 32'd5$.

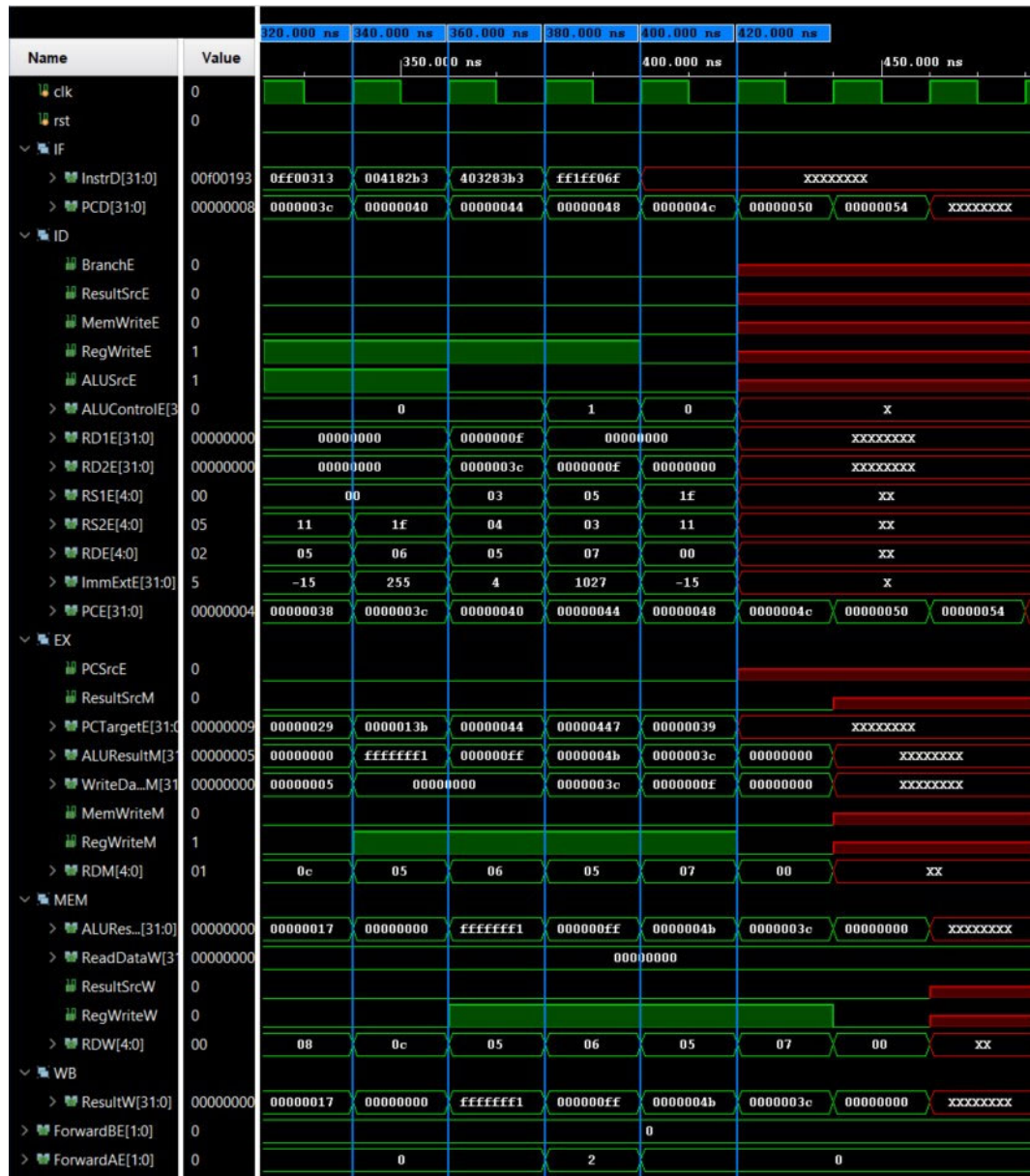
		ResultW = 32'd5; RDW = 5'd7.
	120-220ns	Lệnh sub x8, x1, x2. Phép trừ: $x8 = x1 - x2 = 32'd5 - 32'd5 = 32'd0$. ResultW = 32'd0; RDW = 5'd8.
	140-240ns	Lệnh mul x9, x1, x2. Phép nhân: $x9 = x1 * x2 = 32'd5 * 32'd5 = 32'd25$. ResultW = 32'd25; RDW = 5'd9.
	160-260ns	Lệnh div x10, x1, x2. Phép chia lấy nguyên: $nx10 = x1 / x2 = 32'd5 / 32'd5 = 32'd1$. ResultW = 32'd1; RDW = 5'd10.
	180-280ns	Lệnh rem x11, x1, x2. Phép chia lấy dư: $x11 = x1 \% x2 = 32'd5 \% 32'd5 = 32'd0$. ResultW = 32'd0; RDW = 5'd11.
	200-300ns	Lệnh or x12, x1, x2. Phép OR: $x12 = x1 x2 = 32'd5 32'd5 = 32'd5$. ResultW = 32'd5; RDW = 5'd12.
	220-320ns	Lệnh sll x13, x1, x2. Phép dịch trái: $x13 = x1 << x2 = x1 << x2 = 32'd160$. ResultW = 32'd160; RDW = 5'd13.
Lệnh tải	240-340ns	Lệnh lw x14, 4(x3). Tải giá trị vào thanh ghi: ALUResultM = RD1E + ImmExtE = 32'd19. ResultW = 32'd19; RDW = 5'd14.
Lệnh lưu trữ	260-360ns	Lệnh sw x15, 8(x3). Lưu giá trị vào bộ nhớ: ALUResultM = RD1E + ImmExtE = 32'd23. ALUResultW = 32'd23; WriteDataM = 5'd60.
Lệnh nhảy	280-380ns	Lệnh beq x1, x2, tag. Lệnh nhảy có điều kiện: ALUResult = RD1E - RD2E = 32'd0. PCSrcE = 1, PCTargetE = 32'd40.



Hình 4.15: Dạng sóng của bộ vi xử lý 32 bit (từ 0 – 160ns)



Hình 4.16: Dạng sóng của bộ vi xử lý 32 bit (từ 160 – 320ns)



Hình 4.17: Dạng sóng của bộ vi xử lý 32 bit (từ 320 – 480ns)

CHƯƠNG 5 : KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. KẾT LUẬN

Sau quá trình thực hiện đồ án thiết kế bộ vi xử lý 32 bit dựa trên kiến trúc tập lệnh RISC-V và kiến trúc Harvard, nhóm đã đạt được các yêu cầu sau:

- Thiết kế được một bộ vi xử lý 32 bit
- Mô phỏng được các chức năng của các khối còn trong bộ vi xử lý 32 bit.
- Mô phỏng được toàn bộ hệ thống vi xử lý 32 bit với một số lệnh cơ bản.

5.2. HƯỚNG PHÁT TRIỂN

Đề tài thực hiện “Thiết kế bộ vi xử lý 32 bit” được nhóm thực hiện đạt được các mục tiêu và yêu cầu thiết kế ban đầu đề ra nhưng còn hạn chế về khả năng và thời gian thực hiện nên kết quả thực hiện có thể thiếu sót một vài trường hợp chưa được tối ưu, nhóm đưa ra một số hướng phát triển như sau:

- Mô phỏng thêm nhiều lệnh hơn ở kiến trúc tập lệnh RISC-V.
- Thiết kế bộ vi xử lý với các phép tính số học có dấu chấm động.
- Nhúng lên kit FPGA để kiểm tra tín hiệu và chức năng thực tế.

TÀI LIỆU THAM KHẢO

- [1] – Hoàng Trung Hiếu, (2024), “*Những xu hướng công nghệ triển vọng năm 2024*”, *Báo Thế giới và Việt Nam*, <https://baoquocte.vn/nhung-xu-huong-cong-nghe-trien-vong-nam-2024-255740.html>.
- [2] – Tạp chí Thông tin & Truyền thông, (2023), “*Ngành công nghệ vi mạch và cơ hội phát triển ở Việt Nam*”, <https://ictvietnam.vn/nganh-cong-nghe-vi-mach-va-co-hoi-phat-trien-o-viet-nam-57472.html>.
- [3] – Báo chính phủ, (2023), “*Phát triển công nghiệp điện tử, vi mạch bán dẫn là đầu tư cho tương lai*” <https://baochinhphu.vn/phat-trien-cong-nghiep-dien-tu-vi-mach-ban-dan-la-dau-tu-cho-tuong-lai-102230906214454513.htm>.
- [4] – David A. Patterson, John L. Hennessy [2017], “*Computer Organization and Design RISC-V Edition: The Hardware Software Interface*”, Morgan Kaufmann 2017.
- [5] – Hyun Woo Oh, Kwon Neung Cho và Seung Eun Lee, (2020), “*Design of 32-bit Processor for Embedded Systems*”, 2020 International SoC Design Conference (ISOC), IEEE, pp.306–307.
- [6] – Mamun Bin Ibne Reaz, Md. Shabiul Islam và Mohd. S. Sulaiman, (2002), “*A single clock cycle MIPS RISC processor design using VHDL*”, ICONIP '02, Proceedings of the 9th International Conference on Neural Information Processing, Computational Intelligence for the E-Age (IEEE Cat. No.02EX575). IEEE, pp. 199–203.
- [7] – Safaa S. Omran và Hadeel S. Mahmood, (2013), “*Hardware Modelling of a 32-bit Single Cycle RISC Processor using VHDL*”, in ICIT 2013, The 6th International Conference on Information Technology.
- [8] – Poduel B. và cộng sự, (2015) “*Design and Implementation of Synthesizable 32-bit Four Stage Pipelined RISC Processor in FPGA Using Verilog/VHDL*,” Nepal journal of science and technology, Vol. 15, No.1, pp. 81–88.
- [9] – I. Thanga Dharsni, K. S. Pande and M. K. Panda, (2022), “*Optimized Hazard Free Pipelined Architecture Block for RV32I RISC-V Processor*” 2022 3rd International Conference on Smart Electronics and Communication (ICOSEC), Trichy, India.
- [10] – Synopsys, *What is RISC-V? – How Does it Work?*, <https://www.synopsys.com/glossary/what-is-risc-v.html>
- [11] – RISC-V, *History – RISC-V International* .<https://riscv.org/about/history/#origin>
- [12] – Sharma, S. (2024), *RISC-V Architecture: A Comprehensive Guide to the Open-Source ISA*, <https://www.wevolver.com/article/risc-v-architecture-a-comprehensive-guide-to-the-open-source-isa>

- [13] – Waterman, A. (2016), *Design of the RISC-V Instruction Set Architecture*. (Doctoral dissertation), EECS Department, University of California, Berkeley, 15 – 77.
- [14] – Nguyễn Văn Linh, (2019), *THIẾT KẾ BỘ NHỚ ĐỆM LỆNH CHO VI XỬ LÝ RISC-V*, Luận văn Thạc sĩ, Đại học Quốc gia TP.HCM, Trường Đại học Bách Khoa.
- [15] – Waterman, A. & Asanovic, K. (2017). *The RISC-V Instruction Set Manual*. SiFive Inc, tập 1, 27-102.
- [16] – Admin (2023). Kiến trúc RISC, CISC là gì? Sự khác biệt giữa RISC và CISC CPU. Yeuphancung. <https://yeuphancung.com/tim-hieu-kien-truc-risc-cisc/>
- [17] – Patterson, D. A., & Hennessy, J. L. (2018). *Computer Organization and Design: The Hardware/Software Interface: RISC-V Edition*. Waltham, MA: Morgan Kaufmann, 262–317.
- [18] – TS.Phạm Công Thắng, *Kỹ thuật đường ống và kiến trúc nâng cao*, Trường Đại học Bách Khoa Đại học Đà Nẵng, chương 11.
- [19] – Awati, R. (2022). *What is pipelining?*. TechTarget. <https://www.techtarget.com/whatis/definition/pipelining>
- [20] – Hennessy, J.L., Patterson, D.A. và Krste Asanović (2012). *Computer architecture : a quantitative approach*. Waltham, MA: Morgan Kaufmann.,C1 – C82
- [21] – GeeksforGeeks. (2016). *Computer Organization and Architecture | Pipelining | Set 2 (Dependencies and Data Hazard)*. <https://www.geeksforgeeks.org/computer-organization-and-architecture-pipelining-set-2-dependencies-and-data-hazard/>.
- [22] – Hoang Xuan Dau. (2010). *Bài giảng kiến trúc máy tính*. Học viện Bưu chính viễn thông.

PHỤ LỤC

A. Kiến trúc tập lệnh RV32I/M

31	26 25 24	20 19 15 14	12 11	7 6	0	
funct7	rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]		rs1	funct3	rd	opcode	I-type
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode	B-type
imm[31:12]				rd	opcode	U-type
imm[20 10:1 11 19:12]				rd	opcode	J-type

RV32I Base Instruction Set

imm[31:12]				rd	0110111	LUI
imm[31:12]				rd	0010111	AUIPC
imm[20 10:1 11 19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU
imm[11:0]		rs1	000	rd	0000011	LB
imm[11:0]		rs1	001	rd	0000011	LH
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:0]		rs1	100	rd	0000011	LBU
imm[11:0]		rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI
0000000	rs2	rs1	000	rd	0110011	ADD

31	26 25 24	20 19 15 14	12 11	7 6	0	
funct7	rs2	rs1	funct3	rd	opcode	R-type
	imm[11:0]	rs1	funct3	rd	opcode	I-type

RV32I Base Instruction Set (tiếp theo)

0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0100000	rs2	rs1	101	rd	0110011	SRA
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND
0000	pred	succ	00000	000	00000	FENCE
0000	0000	0000	00000	001	00000	FENCE.I
000000000000			00000	000	00000	ECALL
000000000001			00000	000	00000	EBREAK
csr		rs1	001	rd	1110011	CSRRW
csr		rs1	010	rd	1110011	CSRRS
csr		rs1	011	rd	1110011	CSRRC
csr		zimm	101	rd	1110011	CSRRWI
csr		zimm	110	rd	1110011	CSRRSI
csr		zimm	111	rd	1110011	CSRRCI

RV32M Standard Extension

0000001	rs2	rs1	000	rd	0110011	MUL
0000001	rs2	rs1	001	rd	0110011	MULH
0000001	rs2	rs1	010	rd	0110011	MULHSU
0000001	rs2	rs1	011	rd	0110011	MULHU
0000001	rs2	rs1	100	rd	0110011	DIV
0000001	rs2	rs1	101	rd	0110011	DIVU
0000001	rs2	rs1	110	rd	0110011	REM
0000001	rs2	rs1	111	rd	0110011	REMU

B. Quy ước gọi hàm

Quy ước gọi hàm trong RISC-V là một tập hợp các quy tắc và thỏa thuận về cách các hàm được gọi, cách các đối số được truyền và cách giá trị được trả về

trong kiến trúc máy tính RISC-V. Dưới đây là một số điểm chính của quy ước gọi hàm cho kiến trúc RV32I và RV32M:

- Truyền đối số qua thanh ghi: Khi có thể, các đối số được truyền qua các thanh ghi. Tối đa tám thanh ghi số nguyên (a0-a7) và tối đa tám thanh ghi số học dấu phẩy động (fa0-fa7) được sử dụng để truyền đối số.
- Truyền đối số qua ngăn xếp: Nếu có nhiều hơn tám đối số, các đối số còn lại sẽ được truyền qua ngăn xếp.
- Trả về giá trị: Giá trị trả về của hàm, nếu là kiểu số nguyên hoặc con trỏ, thường được trả về qua thanh ghi a0 (hoặc a1 nếu giá trị trả về là một cấu trúc lớn).
- Lưu trữ trạng thái: Các thanh ghi s0-s11 được sử dụng để lưu trữ trạng thái qua các lời gọi hàm.

C. Link Github:

Tất cả các code được đính kèm trong liên kết Github dưới:

<https://github.com/nbtinh2002/32-bit-RISC-V-CPU.git>